



ソフトウェアエンジニアリング入門

2025.04.07

目次

- 1 はじめに
- 2 ビジネスにおけるソフトウェア開発の特徴
- 3 ソフトウェアエンジニアリングとは？
- 4 ソフトウェアシステムの開発で行なうこと
- 5 グループワーク
 - 要求分析と仕様化ワークショップ
 - 重篤度の判断ワークショップ

1 はじめに

1.1 目的

1.2 到達目標

1.3 全体構成

1.4 フリーでの開発に当てはめる際に意識して欲しいこと

1.5 進行の目安

1.1 目的

- ソフトウェア開発を専門とする組織にて、ソフトウェア開発を実施するに当たり、開発の進め方、各活動の必要性を学ぶ
- 各自がエンジニアとして仕事をしていく中で、自社の開発の中のどの位置付けの仕事をしているのかがエンジニアリングとしてわかり、他の仕事との関係性を考慮して生産性高く高品質なアウトプットができる

エンジニアリング実践者 = エンジニア

1.2 到達目標

- ソフトウェア開発をどのように進めていけばよいのか説明することができる
- ソフトウェア開発の各活動の必要性を説明することができる
- 自分たちの開発プロセスを、一般的なソフトウェアエンジニアリングで行う活動と対比して説明できる

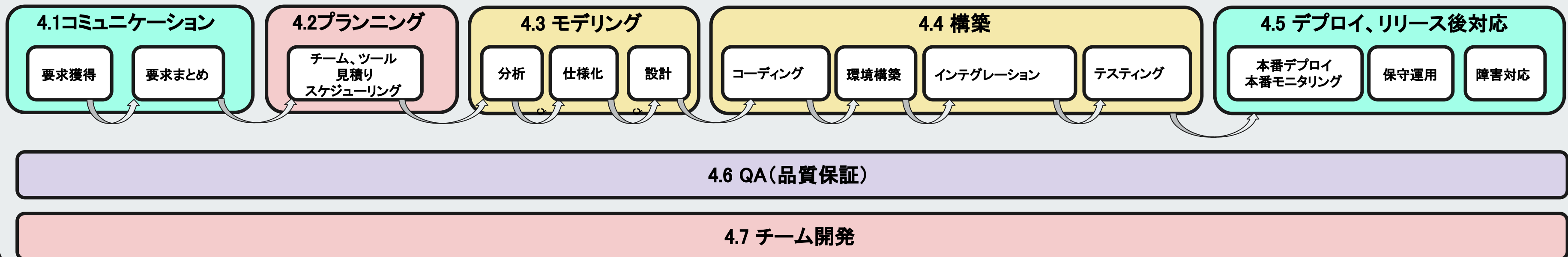
**研修での説明を2日間で完璧に理解する必要はなく、
現場に入ってから改めてこれを見直して理解を深めればOK**

1.3 全体構成

2.ビジネスにおけるソフトウェア開発の特徴

3.ソフトウェアエンジニアリングとは何か？

4.ソフトウェア開発にて行うこと



1.4 フリーでの開発に当てはめる際に意識して欲しいこと

- フリーのエンジニアはDevOps時代のエンジニアなので、今日説明する開発プロセスの活動全部に何かしら関与するってことを意識してもらいたい
 - ただし、ここで説明する一連の活動は、短時間で理解しやすいよう重要なことだけ取り上げる
 - 具体的にすると常にちょっとずつ変わったりするものなので、汎用的にしている
 - 汎用的なエンジニアリングをフリーの開発に当てはめる時は、汎用的なものを具体的に考えてみるのが良い
- フリーの開発プロセスは日々進化しているので、ここでのフリーの具体例の説明は、現時点のスナップショットであることを肝に銘じる

DevOps(デブオプス)とは、ソフトウェアの開発と運用を一体化して、開発プロセスを効率化し、製品を迅速かつ安定してリリースするための考え方や手法

1.5 進行の目安

一日目 4h

- 10:00–11:15
 - 1.はじめに
 - 2.ビジネスにおけるソフトウェア開発の特徴
 - 3.ソフトウェアエンジニアリングとは？
- 11:15–11:30 休憩
- 11:30–13:00
 - 4.ソフトウェアシステムの開発で行なうこと
 - 4.1.コミュニケーション(要求獲得、要求取りまとめ)
 - 4.2.プランニング
 - 4.3.モデリング(分析、仕様化、設計)
 - 5.1グループワーク:グループワークの説明と準備
- 13:00–14:00 休憩
- 14:00–15:00
 - 5.1グループワーク:要求の仕様化

1.5 進行の目安

二日目 4h

- 12:00–13:00
 - 4.ソフトウェアシステムの開発で行なうこと
 - 4.4.構築(コーディング、環境構築、インテグレーション、テスト)
- 13:00–14:00 休憩
- 14:00–15:30
 - 4.ソフトウェアシステムの開発で行なうこと
 - 4.5.デプロイとリリース対応
 - 4.6.品質保証(QA)
 - 4.7.チーム開発
 - 5.2.グループワーク: 重篤度のグループワークの説明と準備
- 15:30–15:45 休憩
- 15:45–17:00
 - 5.2.グループワーク: 重篤度
 - まとめ

2 ビジネスにおけるソフトウェア開発の特徴

2.1 研究室や個人レベルでの開発との違い

2.2 ビジネスにおけるソフトウェア開発の特徴

2.3 ビジネスにおけるソフトウェア開発で言えること

2.4 ソフトウェア開発の本質

2.1 研究室や個人レベルでの開発との違い(i.e. 難しさ)

4つの要因とそれらにある課題にバランスよく対処していかなければならない

顧客

規模

品質

納期

2.2 ビジネスにおけるソフトウェア開発の特徴—顧客

- 顧客が存在すること
 - 利用ユーザー、マーケット
 - 自分が顧客ではない
- 顧客が存在するということは要求があるということ
 - 顧客にとって価値がある、使ってくれるもの
 - 要求はその源流
- 要求は...
 - 最初から全部わからない、かつ曖昧
 - 最初なんて、何が売れるかなんてわからない
 - 変わる
 - マーケット/競合の状況により and/or ものをみたら/知れば知るほど
- 上記にもかかわらず、QCDゴールのコミットメントが必要になる

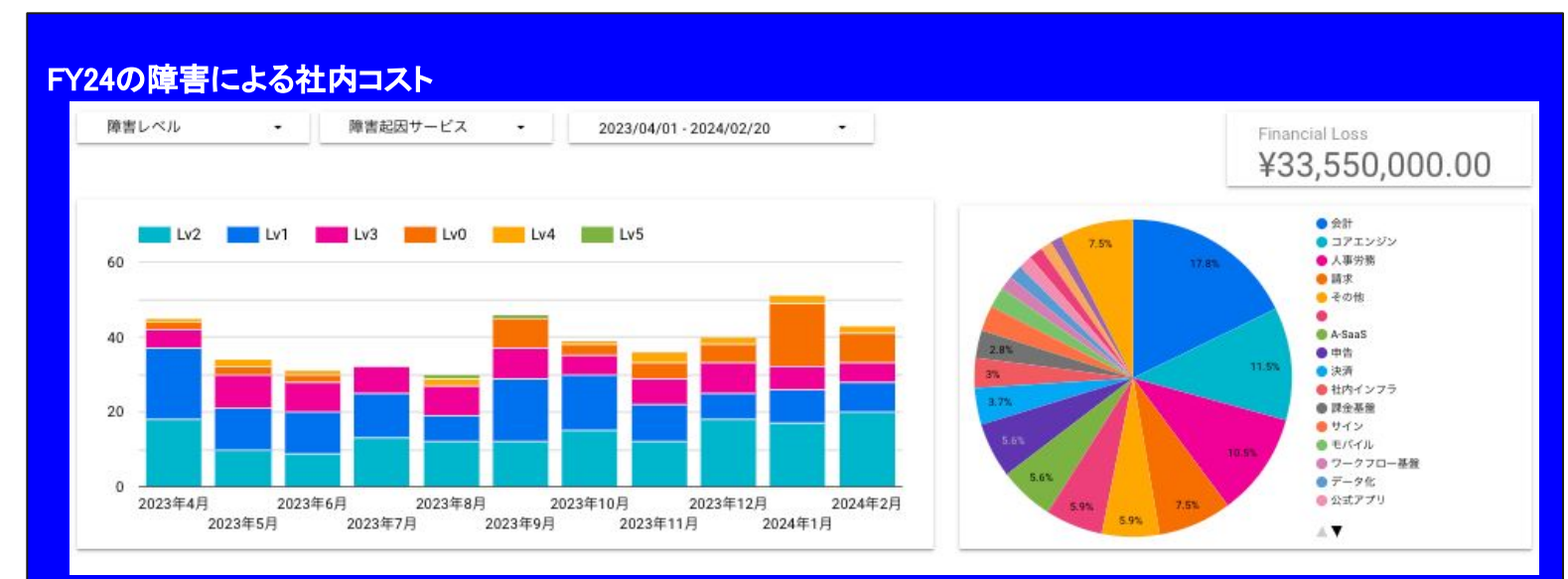
顧客

Q: Quality: 何を・どのような状態で
C: Cost: どの位のコストをかけて
D: Delivery: いつまでに

2.2 ビジネスにおけるソフトウェア開発の特徴—品質

- 途中でエラーを起こしたり、落ちたりしてはいけない ... 原則
 - きちんとした信頼性を確保しなければならない
 - ユーザーが増えれば増えるほど信頼性が求められる
 - 使ってもらう時間/期間が増えるほど信頼性が求められる
- 適切に信頼性が確保できないと...
 - 売れない
 - 社会的影響(レピュテーション)
 - 割り込み対応による追加作業の増大
- 法律や規制への対応
 - 満足しないと市場に出せない
- ソフトウェアの品質確保の難しさ
 - 目に見えない *人工物* 大規模 * 複雑

品質



2.2 ビジネスにおけるソフトウェア開発の特徴ー規模

- 規模の増大化

- 一人でこなせる規模には限界がある
 - ユーザー数、使ってもらう時間/期間が増えるほど
- 体制も大規模化
 - 大きくなればなるほど、制御(ex. 変化への対応)にはオーバーヘッドを要するようになる

規模

- 機能拡大、連携、統合による複雑化

- つまり、要求の高度化・複雑化→ 要求の問題がより深刻なものに
 - ユーザーにとって複雑になりすぎると使ってくれなくなることも

- ソフトウェアコードの指数関数的な複雑化

- 技術的負債



2.2 ビジネスにおけるソフトウェア開発の特徴ー納期

- 開発は納期(適切なリリースタイミング)が必要
 - 納期(D)はQ(品質)C(コスト)とセットでコミットする必要がある
- 納期も品質の一部
 - 同じものでも時期を逸したら価値はなくなる/落ちる
 - できた時が納期ではない
- 開発期間は短縮化の傾向 ⇔ 大規模化
 - 典型的に要求やコストと対立
- ソフトウェアの見積りの難しさ
 - 不明確・不確定な要求 * 目に見えない・人工物
- ソフトウェアの作りは納期にも大きく影響する

納期

フリーで言えば、確定申告や年末調整に合わせたリリースをしないと無意味
(例:フリー創業時の確定申告に間に合わなかった)

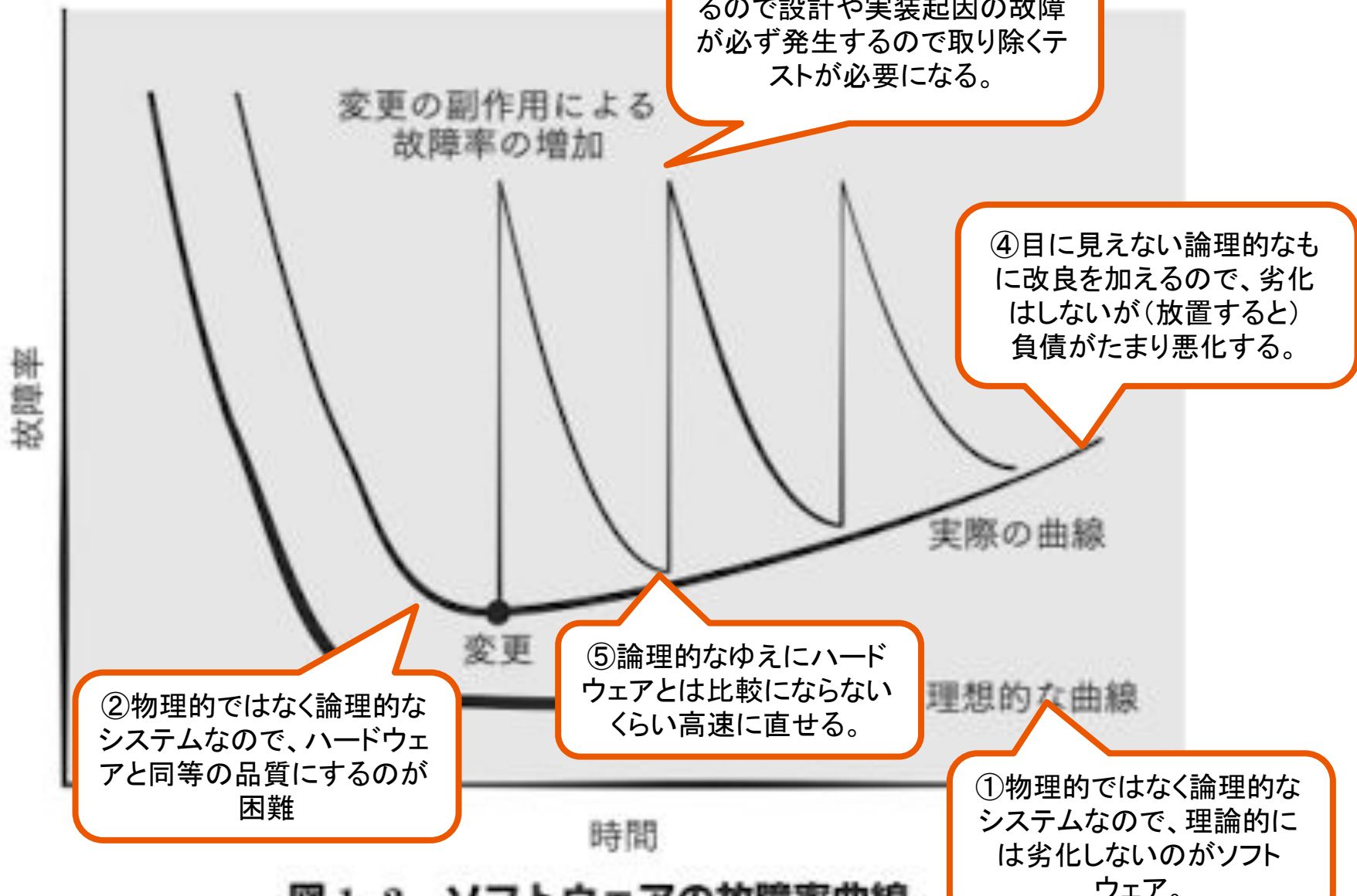
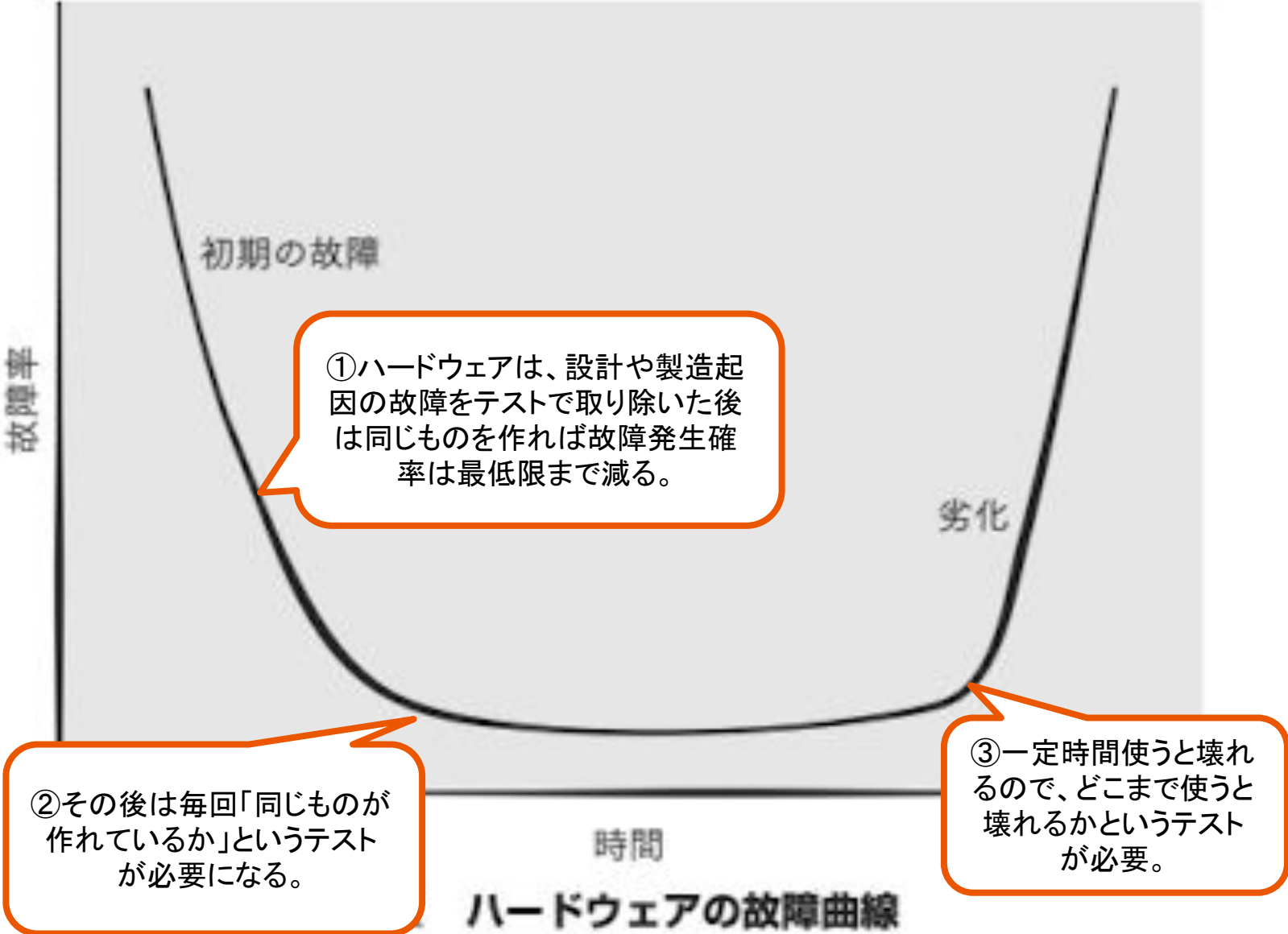
2.3 ビジネスにおけるソフトウェア開発に言えること

- 要求が曖昧、短期間でリリースといった難しい状況の中でも、ビジネス成功のためは確度高く開発を成功させなければならない
- 各個人に能力があっても、それぞれがバラバラに作っては、成功とは程遠いものになってしまう
- ギャンブルでも、星占いでもない、合理的なやり方が必要
「エンジニアリング」

日曜大工で棚を作る方法と大きなビルは、建てる方法が違う
それぞれには、それぞれに適した方法が必要
＝エンジニアリング

2.4 ソフトウェア開発の本質（製造業開発との比較）

● 製造業など他のエンジニアリングとの決定的な違いを理解する



3 ソフトウェアエンジニアリングとは？

3.1 エンジニアリングであるためのポイント

3.2 エンジニアリングの三要素

3.3 エンジニアリングの三要素の関係

3.4 本講義で取り上げる内容ープロセス

3.5 プロセスとテスト/レビューの関係

ここではソフトウェア開発に限定して説明するので、全般的に「ソフトウェア」を省略してエンジニアリングと記載しますが、
全てソフトウェアエンジニアリングのことだと思って見て行ってください

3.1 エンジニアリングであるためのポイント

ソフトウェアエンジニアリングの定義

ソフトウェアエンジニアリング基礎知識体系 -SWEBOK V3.0-

- (成功の確率を上げるため)体系的で、訓練されていて、定量的なやり方の適用

エンジニアリングを学問として学ぶことを**工学**と呼ぶが、ここではあくまで現場で実践していくことにこだわるので**エンジニアリング**と呼ぶ

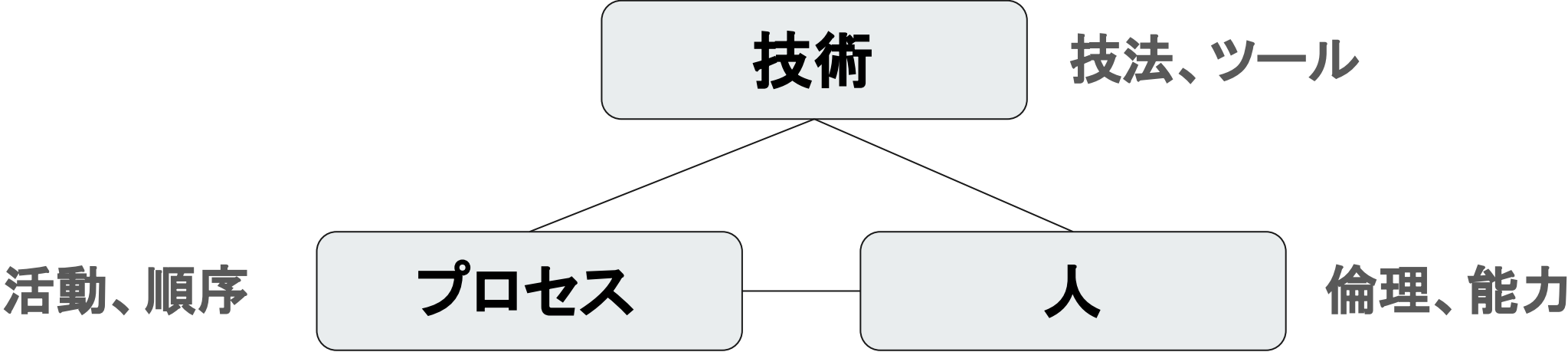
エンジニアリングであるためのポイント

- 一般に正当と認められた知識、合理的で科学的な基盤に基づいた知識の体系に基づいたアプローチであること
 - 開発チームにおいて
 - 体系的なやり方の適用
 - エンジニア個人にとって
 - 上記知識/アプローチの習得、実践への的確な適用

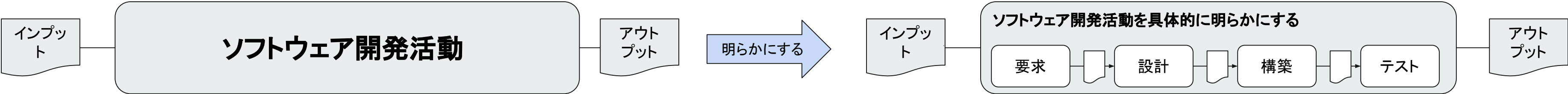
(大事なことからもう一度書きます)

エンジニアリング実践者
=エンジニア

3.2 エンジニアリングの三要素



開発での活動をブラックボックスではなく、具体的にあきらかにして共有可能にするのが**プロセス**



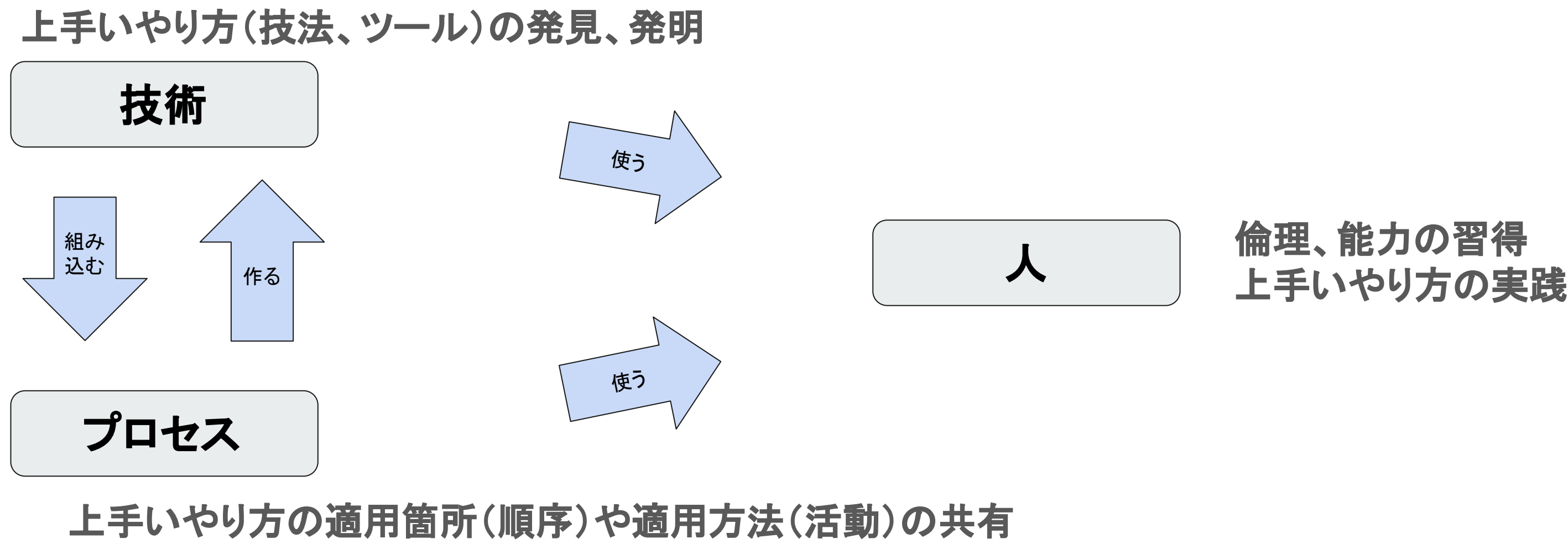
開発での活動にて、効率をよくしたり、同じ間違いを繰り返さないような上手いやり方が**技術**

技法や手法：開発言語、設計技法、仕様記述言語、モデリング言語、テスト技法...

ツール：エディター（VSCodeなど）、構成管理（GitHubなど）、CI/CD（CircleCIなど）、環境（Dockerなど）...



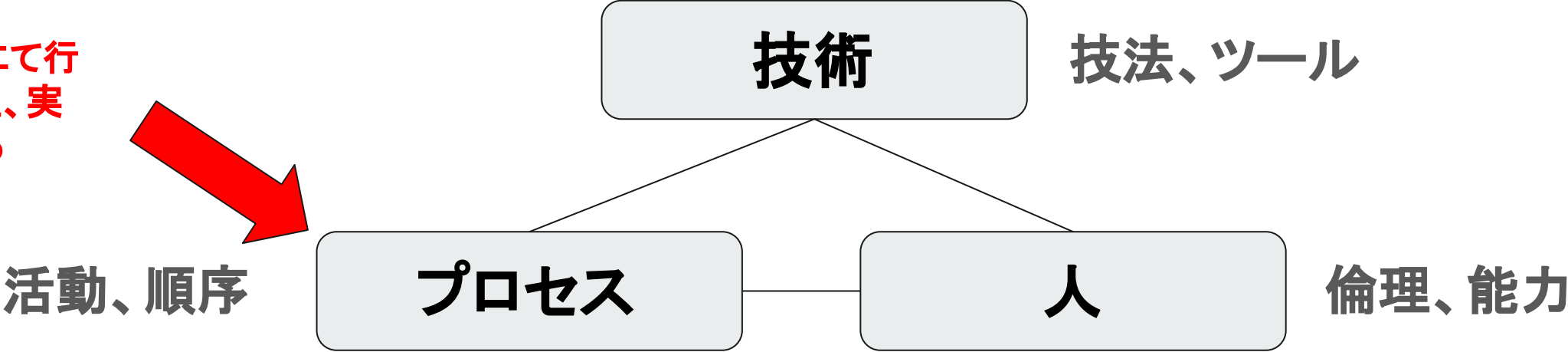
3.3 エンジニアリングの三要素の関係



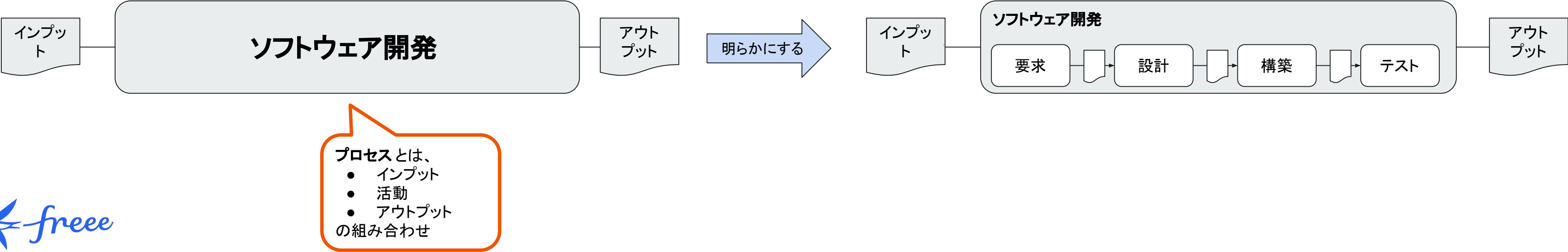
三要素が噛み合って初めて現場で実践ができる

3.4 本研修で取り上げる内容...プロセス

本研修では、プロセスにて行われる各活動の必要性、実施内容などをとりあげる



[再掲]開発での活動をブラックボックスではなく、具体的にあきらかにして共有可能にするのが
プロセス

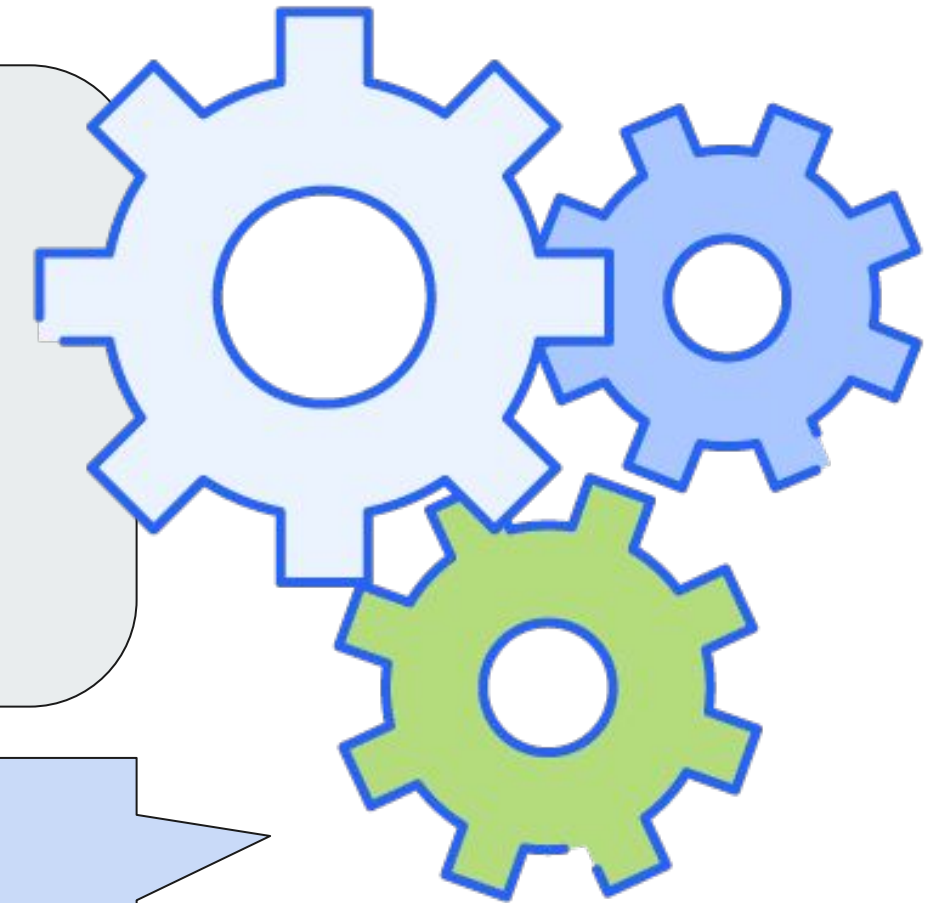


[参考]ソフトウェア開発にとってプロセスが大事な理由



人に依存するところが大(目に見えない)

- 要求は人から生まれる
- 仕様も設計も人が考える
- 構築も人の手で行う



空中戦



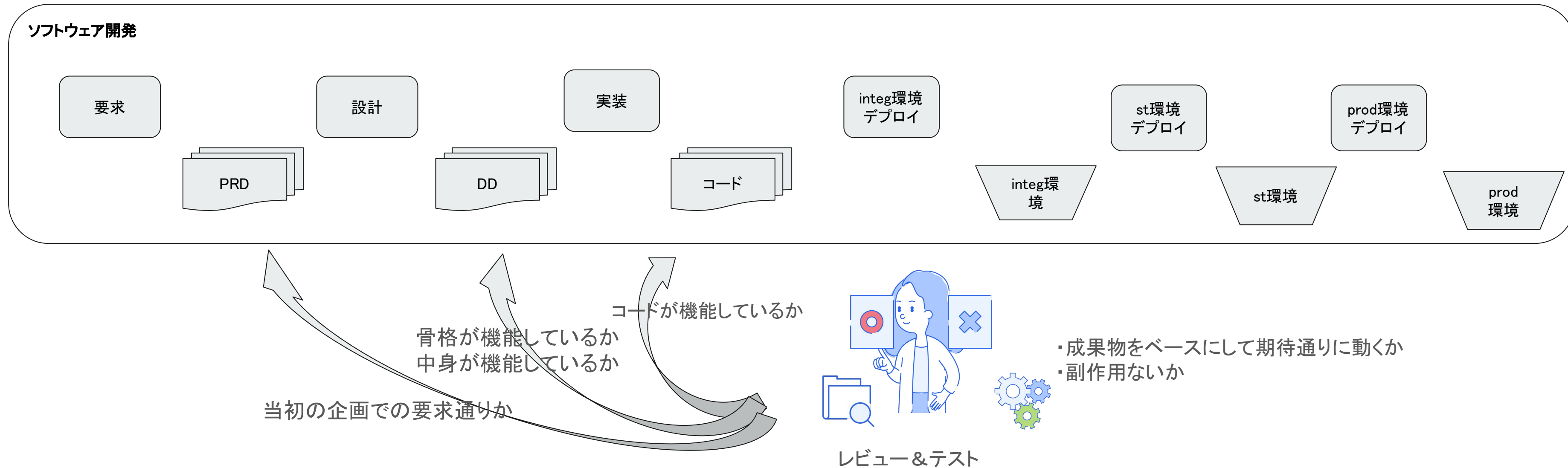
ソフトウェアは複雑な概念構造物
物理法則に縛られなく
自由度が高い

目に見えないから
どうにでもなる
「誤解」しやすい

「プロダクトの品質はそれを開発するのに用いられるプロセスに支配される(プロセスが悪い状態では高品質なプロダクトは届け続けられない)」
という考え方に基づいている

3.5 プロセスとテスト/レビューの関係

プロセスの中の活動と活動の間には成果物がある



プロセスの節目単位でアウトプットが大丈夫かを確認

[参考]実践ソフトウェアエンジニアリング序文から抜粋

21世紀において取り組むに値するソフトウェアを構築するためには、次の現実を認識しなければならない。

・ソフトウェアは、人々の生活のほぼすべてに深く入り込んでいる。とあるアプリケーションが提供する機能やフィーチャに対して興味をもつ人の数も劇的に増大している。**ソフトウェアによる解決策が構築される前に問題を理解する労力を費やすべき** である。

・個人や企業、政府により求められる情報技術の要求は、年々複雑さを増し続けている。今や大規模なチームでプログラムが作られている。予測可能であり、必要なものが揃えられ、コンピュータ環境にて実装された高度なソフトウェアは、電子機器や医療機器、自動車まであらゆるものに組み込まれている。**設計が中枢を担う活動** となる。

・個人や企業、政府において、日々の活動や統制に加えて、戦略的、戦術的な決定判断をソフトウェアに頼る状況が増えてきている。ソフトウェアが問題を起こしたならば、人々や企業が少し不便になることもあれば、致命的な結果にもなることもある。**ソフトウェアは高品質であるべき** だ。

・特定のアプリケーションにて得られる価値が高まることにより、アプリケーションのユーザー数増加に加え寿命も伸びる。アプリケーションのユーザー数と使用時間の増加につれて、さらなる適用先と拡張の要求が発生する。**ソフトウェアはメンテナンス(保守)可能であるべき** だ。

ロジャー・プレスマン; ブルース・マキシム. 実践ソフトウェアエンジニアリング (第9版) (p. 2). (Function). Kindle Edition.



4 ソフトウェアシステムの開発で行なうこと

4.1 コミュニケーション

4.2 プランニング

4.3 モデリング

4.4 構築

4.5 デプロイ、リリース後対応

4.6 QA(品質保証)

4.7 チーム開発

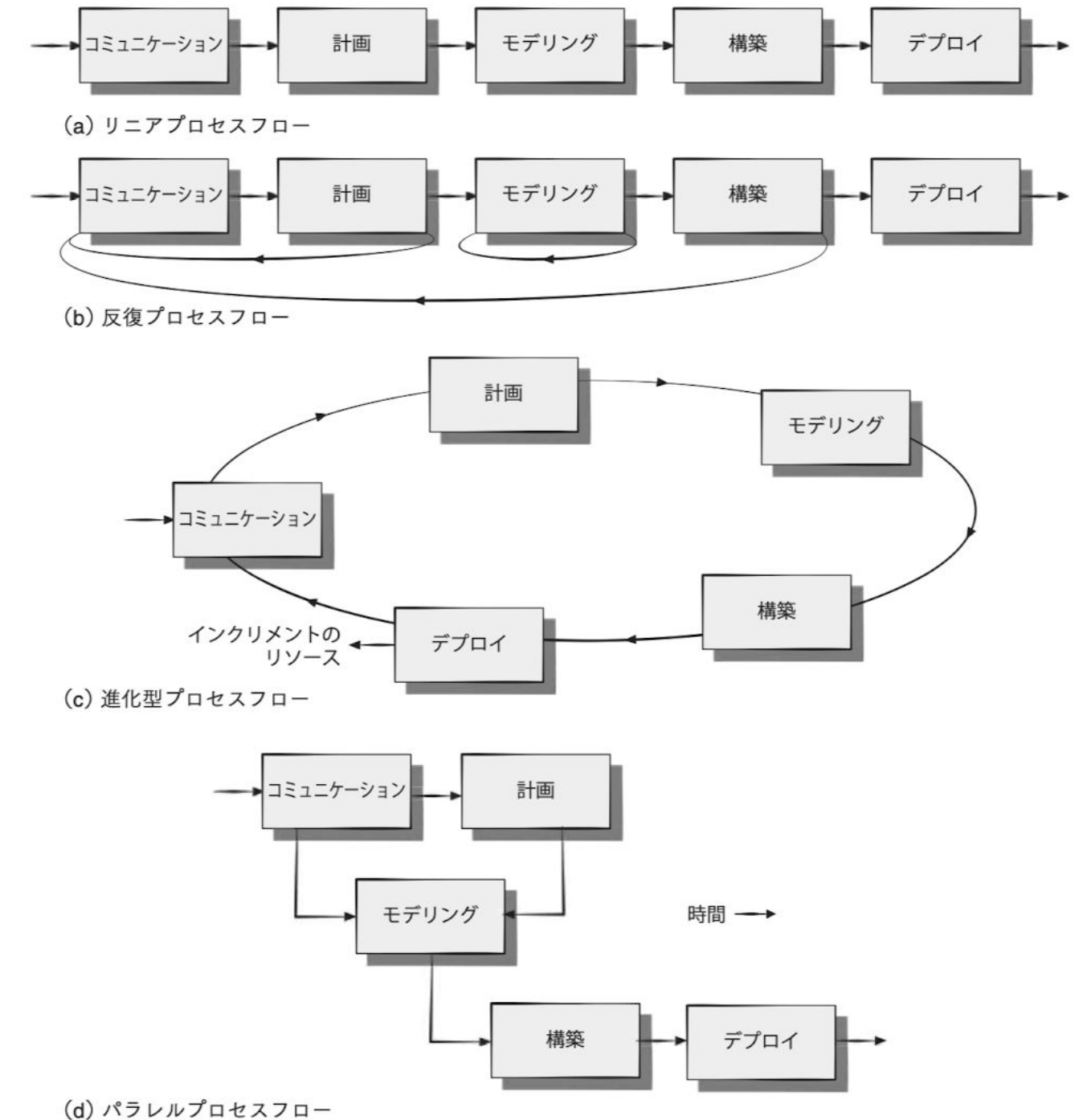


図 2.2 プロセスフロー

ロジャー・プレスマン; ブルース・マキシム. 実践ソフトウェアエンジニアリング (第9版)

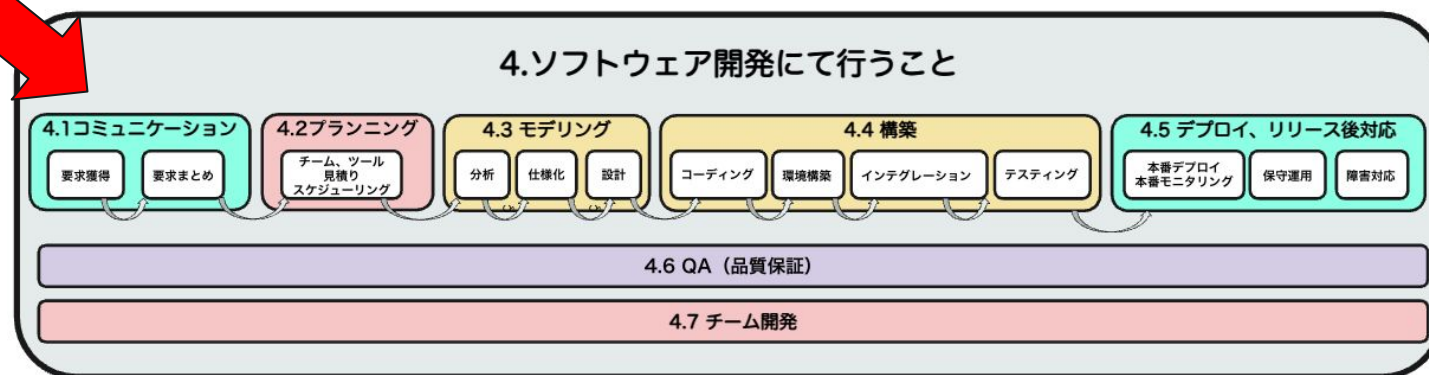
開発で行なうことは、アプローチによって
さまざまなプロセスフローで行なわれる

4.1 コミュニケーション

4.1.1 システム要求の獲得

4.1.2 システム要求のとりまとめ

4.1.3 フリーの開発でやっていること



技術的な業務を開始する前に、顧客やステークホルダーとのコミュニケーション および協業は極めて重要である。ステークホルダーのプロジェクトにおける目的を理解することや要望を聞くことは、ソフトウェアの機能やフィーチャを決めるために役に立つ。

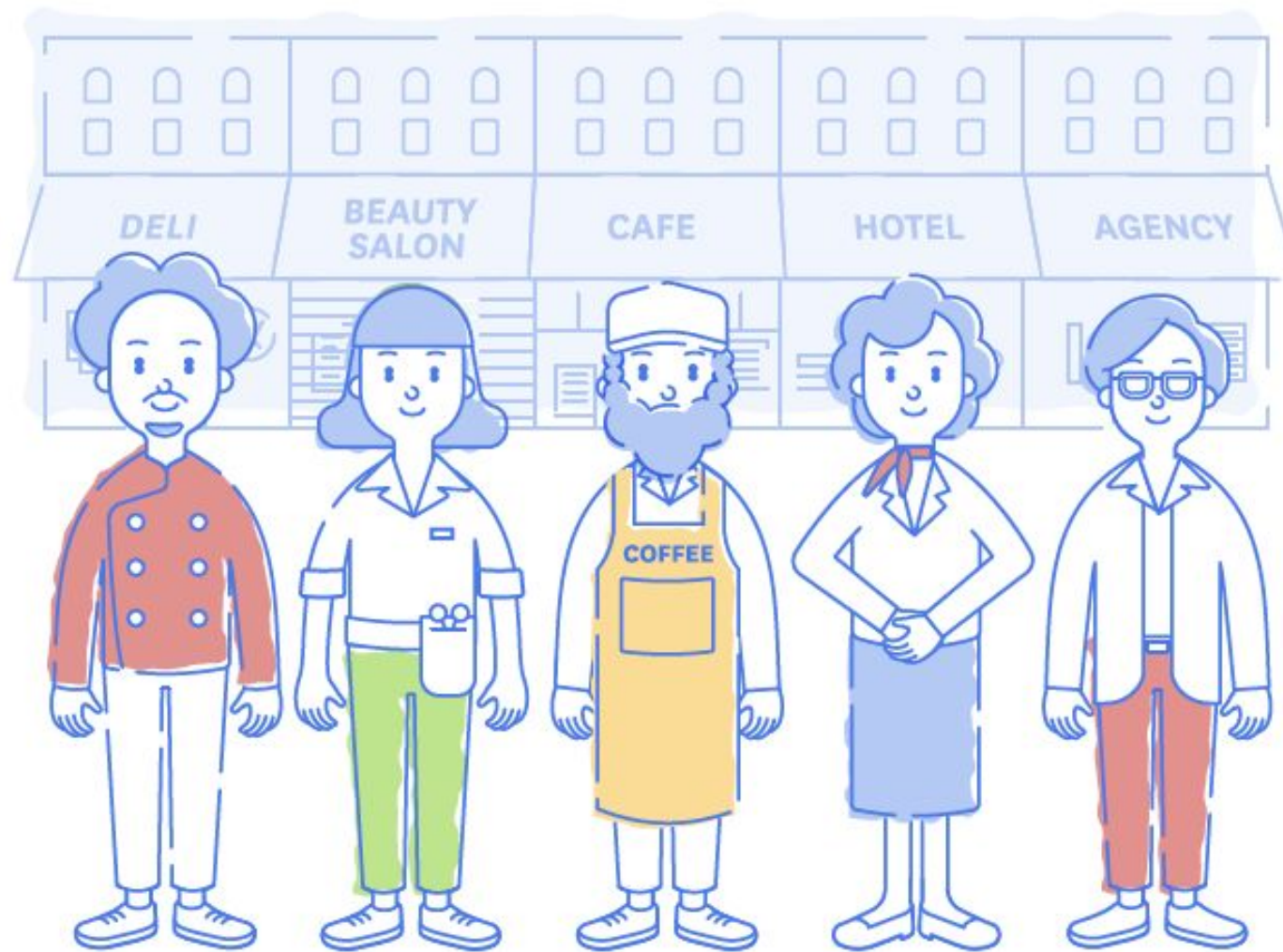
ロジャー・プレスマン; ブルース・マキシム. 実践ソフトウェアエンジニアリング (第9版) (p. 8). (Function). Kindle Edition.

4.1.1 システム要求の獲得

システム要求とは、関係者がシステムに実現してもらいたいこと

- 関係者のことをステークホルダーともいう

freee会計のスマホアプリだとすると...



機能要求

レシート撮影して勝手に登録を全部すませたい

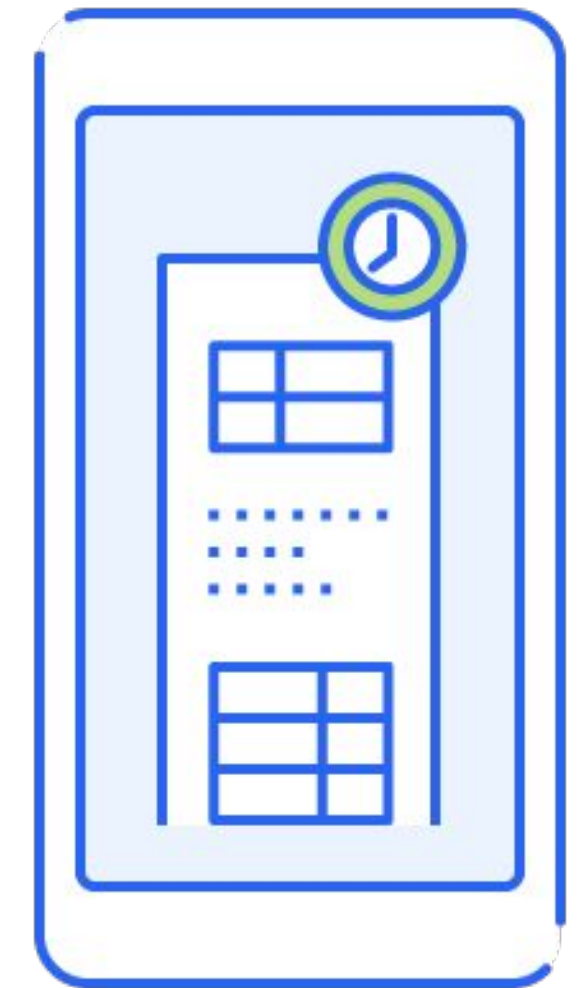
最新の預金残高を把握したい

自分のやらなきゃいけないタスクを把握したい

非機能要求

登録はサクサクできるようにしたい

直感的に何をすればよいかわかるようにしたい

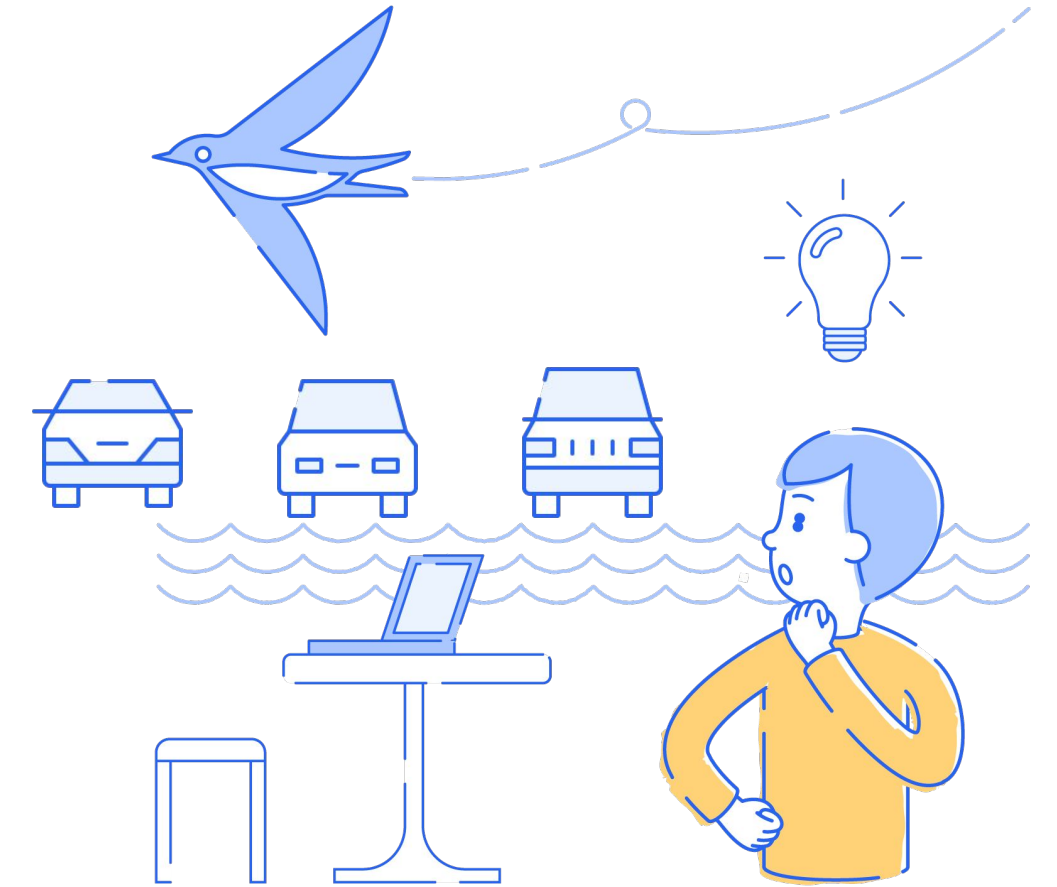
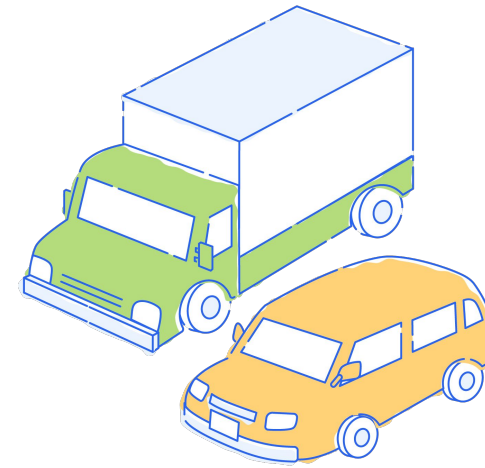


4.1.1 システム要求の獲得

新規プロダクトの企画立案

東京都内の中古車販売会社にて、ある上層部の着想

- 2つ大展示場とフランチャイズ方式の販売店舗にてビジネスを展開しているが、競合が増えてきて、もっとバリューをだしたい
- そのためには、顧客への迅速対応を目指すため、従来の手作業の受注業務や在庫車情報をシステムで管理すればよいのでは？
- 「誰に価値があるか？」「売り上げがどれくらいありそうか？」「利益が出るか？」といったことをまとめた企画を通して、プロジェクトを立ち上げる



4.1.1 システム要求の獲得

関係者のシステムへの要求を収集する

要求を収集するには以下の活動が必要

- 関係者が誰か特定し、
- 要望(システム化することがまだ合意できてない状態)を収集する
 - ブレインストーミング、インタビュー、プロトタイピング(紙芝居などでやりたいこと聞き出す)

在庫係



どの車から整備しなきゃいけないのか(納期順に)知りたい

顧客係



お客様のほしい中古車が販売可能かすぐに提示したい

経営層



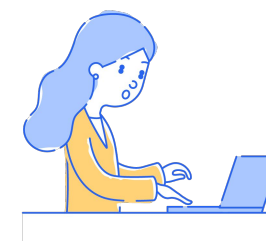
無駄な在庫を減らして利益率をあげたいし、売り上げも上げたい

他システム関係者



売り上げが立ったらすぐに計上できるように連携してほしい

開発エンジニア



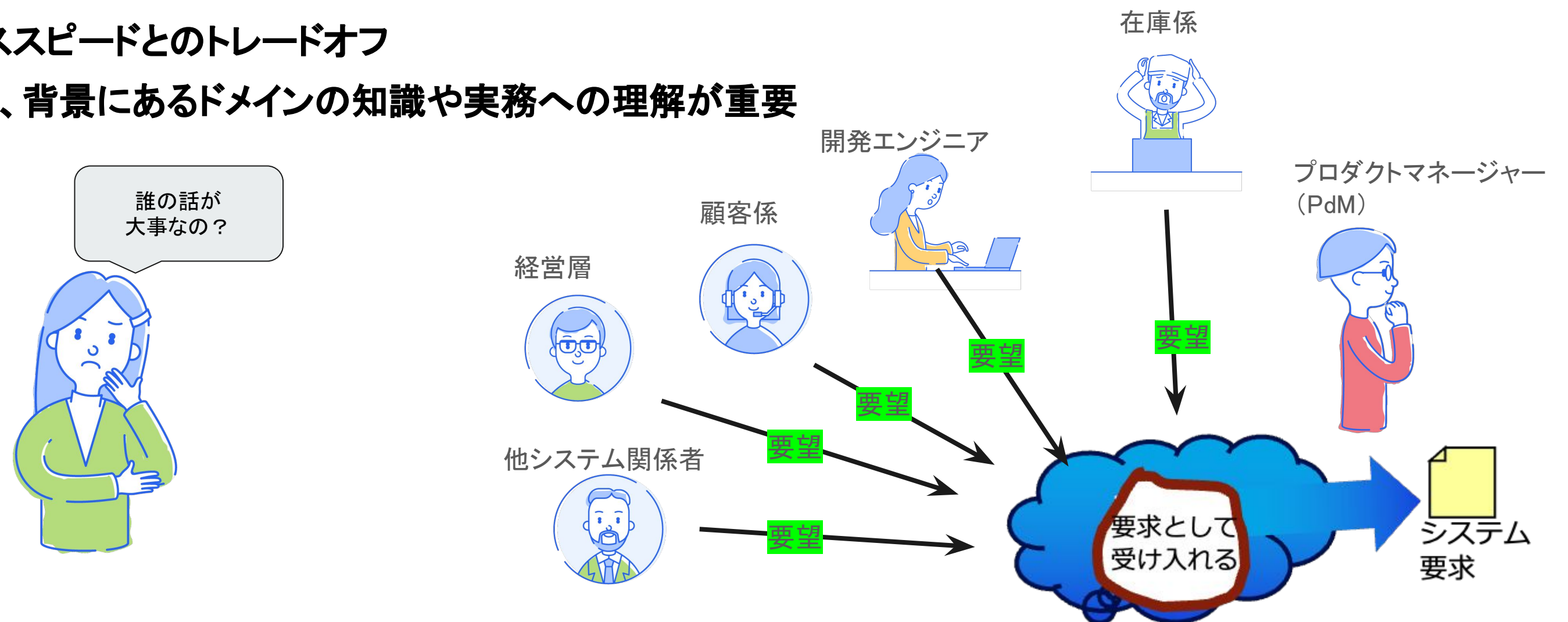
連携システムを増やして複雑ににならないようにしてほしい

4.1.2 システム要求のとりまとめ

関係者のシステムへの要求をまとめる

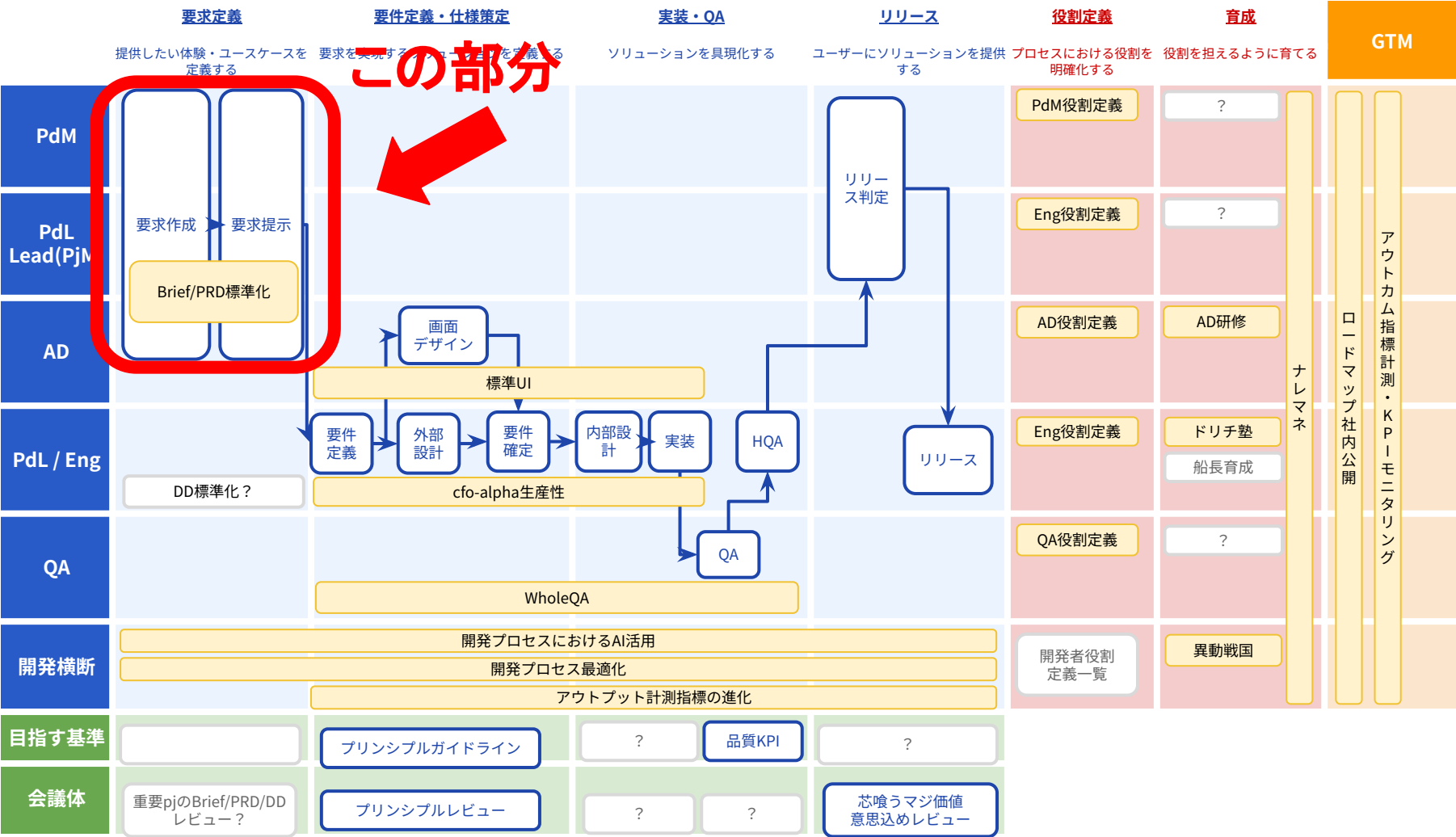
要求をまとめるには以下の活動が必要

- 関係者の要望が本来の企画内容からして妥当なのかを考慮して、
- システム要求(システム化することが合意できている要望)を取捨選択
 - 取捨選択するには、要求の衝突に対する折衝やトレードオフが必要
 - 例:リリーススピードとのトレードオフ
 - 取捨選択には、背景にあるドメインの知識や実務への理解が重要



4.1.3 フリーの開発でやってること(コミュニケーション)

FY25開發生産性PJから



2024年10月の時点でまとめた開発プロセスから

- フリーの開発プロセス(外部公開なし)
- 主要成果物
 - ロードマップ
 - 開発計画
 - Brief
 - (新規)サービス名
 - (新規)slack ch
 - (新規)UT結果
 - (新規)サービスコード
 - プロジェクトコード
 - PRD

フリー社内のモデリングに属する活動についてのお役立ちリンク(外部公開なし)

- Brief PRD

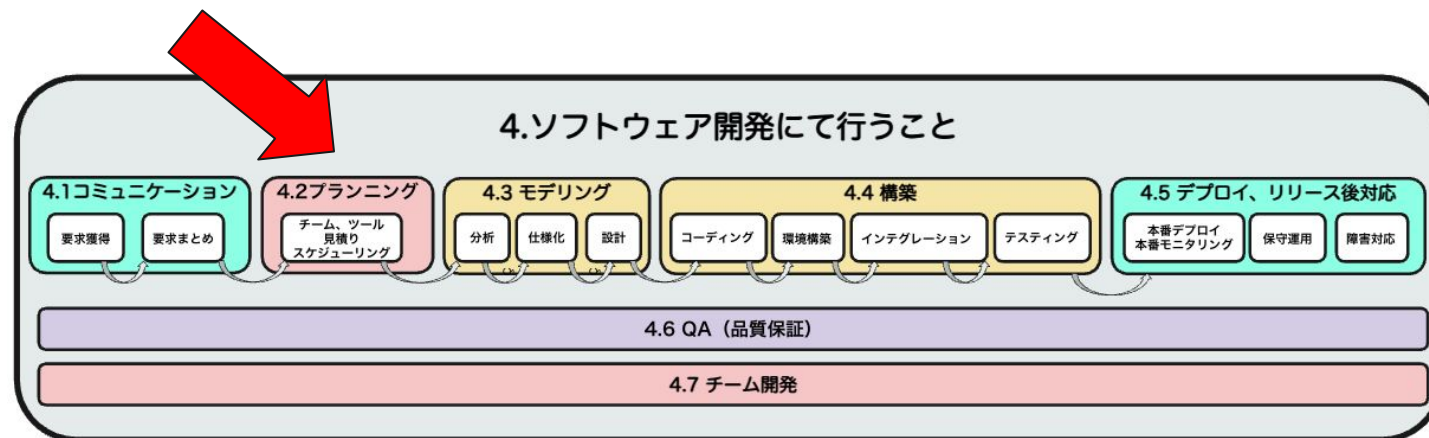
4.2 プランニング

4.2.1 プランニングの必要性

4.2.2 アプローチ選定

4.2.3 見積り

4.2.4 スケジューリング



ソフトウェアのプロジェクトは複雑な旅程に相当する。計画のアクティビティではチームをガイドする「地図」を作り出す。ソフトウェアプロジェクト計画と呼ばれる地図には、ソフトウェアエンジニアリング業務に関する技術的なタスク、発生しそうなリスク、必要なリソース、作成すべき成果物、スケジュールを記述する。

ロジャー・プレスマン; ブルース・マキシム. 実践ソフトウェアエンジニアリング (第9版) (p. 8). (Function). Kindle Edition.

4.2.1 プランニングの必要性

プランニングとは、プロジェクト成功の地図作り

- 規模やリスクが大きいソフトウェア開発では、ゴールへの道筋を明確化しないと手戻りが増える
- チームが同じ方向性を共有するための「コンパス」の役割を果たす
 - 計画を決め切って必ず守るというより「チーム全体が進め方に合意する」のが大事
 - プロジェクト開始前に大まかな見通しを立て、途中で状況に合わせて修正していくのがよいやり方
 - プロジェクトマネジメントで行う「モニタリングとコントロール」が相当する
- 主な検討項目
 - アプローチ選定（プロセスフローの選択と、適した体制やツールの選択）
 - 見積り（工数・コスト・リスク）
 - スケジューリング（いつ何を行うか）



4.2.2 アプローチ選定

プロセスフローの選択

- リニアプロセス(例:ウォーターフォール開発)
 - 計画を理解しやすい
 - 要求が明確なプロジェクトでうまくきやすい
 - 変化への対応が考慮されていない
 - テストがプロセスの後半になってしまい、工数、期間が膨らむ
- 反復/進化型プロセス(例:アジャイル開発)
 - 開発リスクがマネジメントされる
 - 拡張性があるプロダクトに非常に適している
 - リスク分析の成否がプロジェクトの運命を決める
 - プロジェクトのマネジメントは、開発チームの成熟度に依存する
- ハイブリッド型(複数のプロセスを融合した開発)
 - 上流で大まかな要件を固めつつ、詳細部分はイテレーティブに開発できる
 - 文化的・組織的な衝突の可能性があり、マネジメント力が必要となる

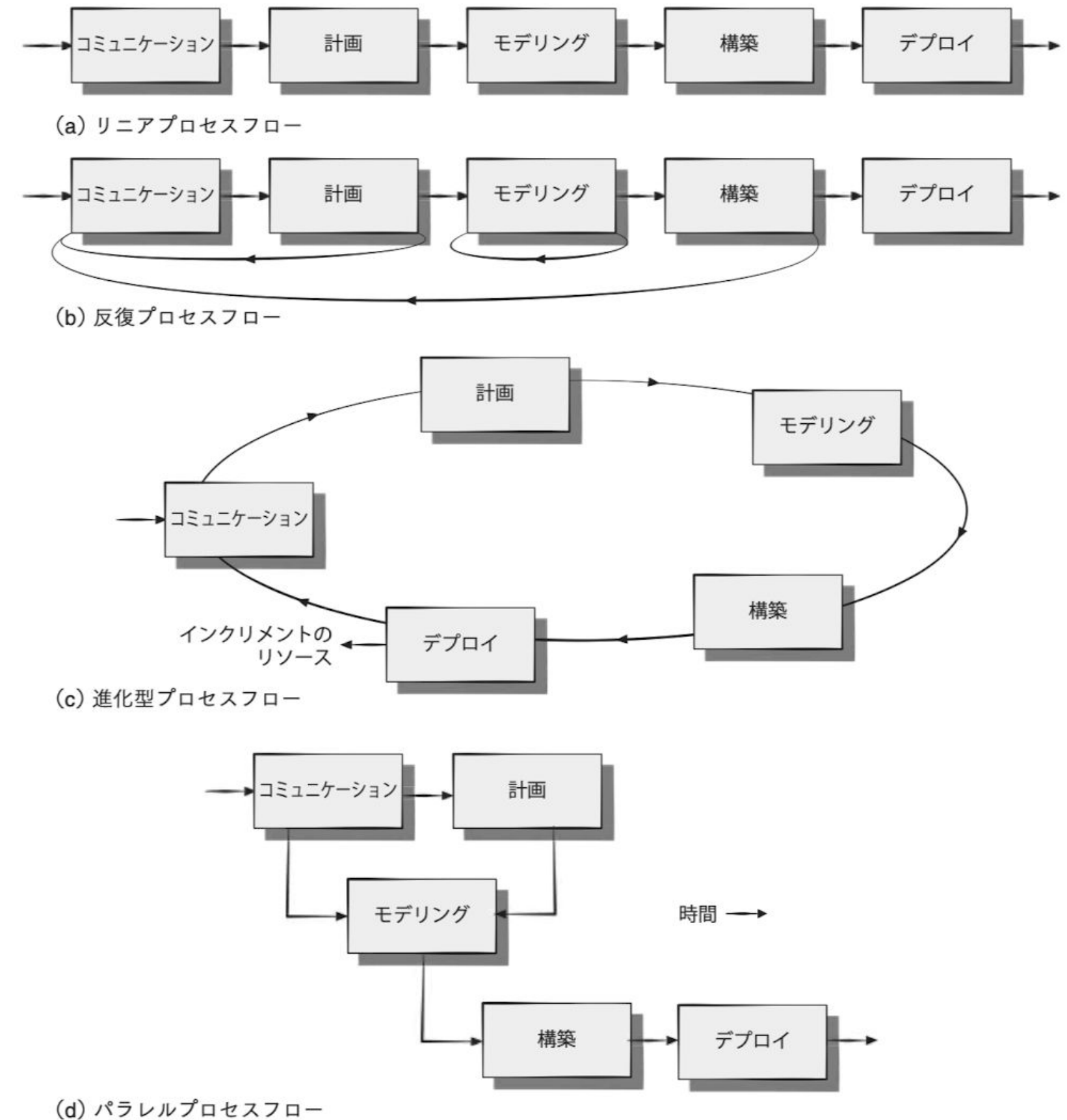


図 2.2 プロセスフロー

4.2.2 アプローチ選定

プロセスフローに合わせた、チーム編成・役割分担・利用ツールの選択

- チーム体制
 - プロダクトの特性に応じて人数・アサインする人に求められる専門領域を決める
 - アジャイル開発ならクロスファンクショナルなチームを組成
- 役割分担
 - エンジニア、デザイナー、PdM、テックリード、QA、スクラムマスターなど
 - アジャイル開発では、メンバーが役割を重複して持つことも多い
 - 「4.7.4チーム開発」にて説明しているチームのステージによって役割を考慮する
- ツール・環境選定
 - リポジトリ管理（GitHubなど）、CI/CD（GitHub Actionsなど）、タスク管理（Jiraなど）
 - 開発プロセスフローに合うツールや環境を揃え、プロセスフローの中でどう使うかを明確化する
 - チームが自律的に動けるように必要な技術スタックを定義する

技術スタック = プロジェクトで採用するプログラミング言語、フレームワーク、ツール、インフラなどをまとめたもの

4.2.3 見積り

見積りはなぜ難しい？ どう進める？

- ソフトウェア見積りの難しさ
 - 要求があいまい、変更が多い
 - コードベースや技術要素が複雑

コードベース = 追加開発する際の前提になる既存のコード群のこと

- 主な見積り手法

- トップダウン見積り: 過去実績や大まかな類推でざっくり推測
 - 比率による見積り: 例. 開発工数とテスト工数が4:1という実績があれば、5人日の工数のうちテストは1人日
- ボトムアップ見積り: より具体的なタスクに分割し、各タスクの工数を積み上げる
 - プランニングポーカー: ストーリーポイントをチームでディスカッションしながら相対見積り
 - 外挿(実績ベースの見積り): 実績ある過去データからの見積り(例: バーンダウンチャート)

比率は、プロセスフローによって変わる、例えば、コーディングの工数は、アジャイル開発では開発全体の50%になることもある(ウォーターフォールだと20-40%)

- リスクを考慮したバッファ

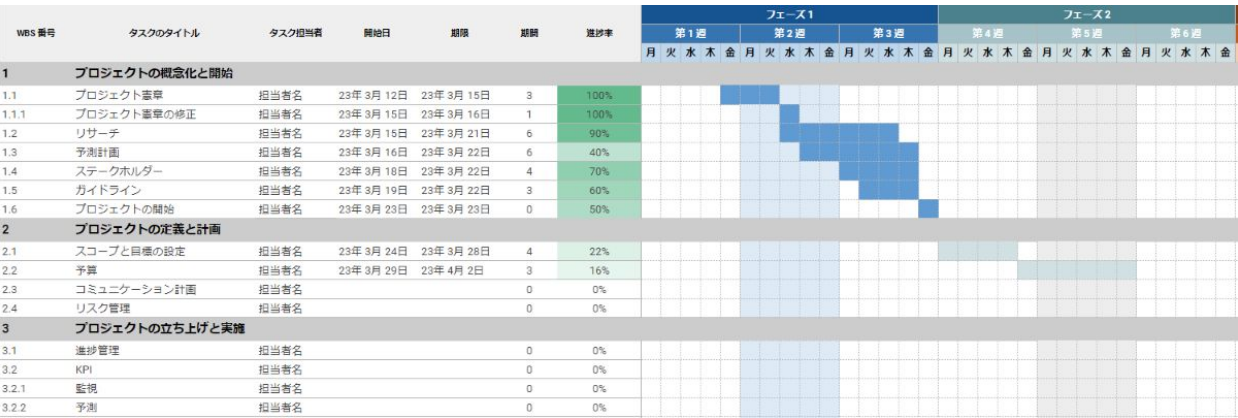
- 不確定要素に備えて一定の工数バッファを設ける
- リスク(技術的難易度、外部依存、スキル不足など)を洗い出して対処を考える

リスク = 現時点では問題になっていない将来に良くない自体が起きる可能性のこと。

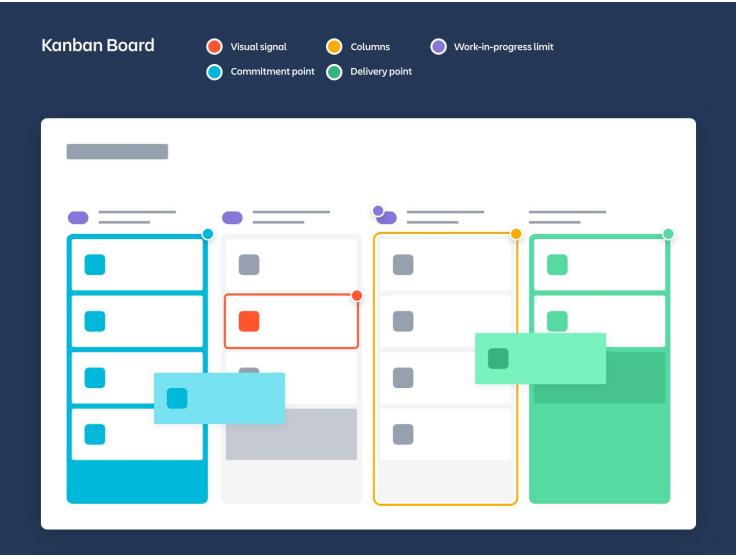
4.2.4 スケジューリング

スケジュールの可視化ツール

- チームの習熟度やプロジェクト特性に合わせて使い分ける or 可視化ツールを組み合わせる
- スケジュールは一度作って終わりではなく、随時アップデートする
- 可視化ツール
 - ガントチャート
 - ウォーターフォール開発によく使われ、タスクの依存関係や期間を視覚的に把握する
 - 長期・大規模案件で全体像を捉えるのに有効だが、変更時の更新が面倒な場合も
 - スプリント/イテレーション
 - アジャイル開発で1~4週間サイクルを区切りとする
 - その期間で確実にリリース可能な機能を完成させる
 - カンバン(看板)ボード
 - To Do / Doing / Doneの3列などでタスクの進捗をリアルタイムに管理
 - 個々のタスクを「WIPリミット(仕掛かり数制限)」で制御し、流量を安定化



ガントチャート



カンバンボード

4.2.4 スケジューリング

マイルストーンの設定

- **マイルストーンとは、リリースなど主要な出来事や期日のタイミングを設定して、スケジュールの節目を明確化すること**
 - **全員が重要な締め切りやアウトプットを意識できる**
 - 最も重要なマイルストーンから逆算して設定していく
 - 柔軟にリスケジュールができるよう、常に数週間先を見越して余裕を持たせる
 - **納期と品質のトレードオフ**
 - 開発スピードと品質をどうバランスさせるか
 - スcopeを削る(MVPの発想)か、納期をずらすかなどの調整
 - **チーム外との連携**
 - 他チームや他社サービスへの連絡や会議の時間も含める
 - 連携先の都合で遅延する 可能性を見込んだリスク管理をする



MVP = 必要最小限の製品 (Minimum Viable Product) の略

4.3 モデリング

4.3.1 要求分析

4.3.2 要求の仕様化

4.3.3 設計

4.3.4 フリーの開発でやってること



庭師や橋梁架設者、航空機技術者、大工、建築家はいずれもモデルを用いて毎日の業務をこなしている。スケッチをすることで物ごとの全体像を理解する。構造的にはどうか、要素部分をどのようにつなぎ合わせるか、等について考察する。より深く問題を理解し、どのように問題を解くかを考えるために、必要に応じて対象を深く詳細まで掘り下げて描いたスケッチを書き直す。同様にエンジニアはソフトウェアの要求、実現のための設計をより深く理解するためにモデルを作成する。

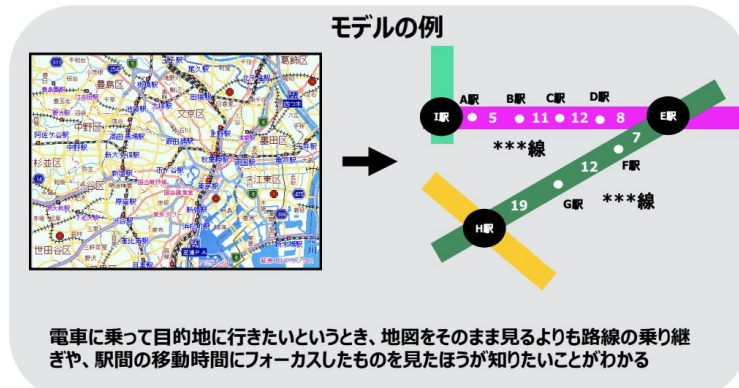
ロジャー・プレスマン; ブルース・マキシム. 実践ソフトウェアエンジニアリング (第9版) (p. 8). (Function). Kindle Edition.

[参考]モデリング＝モデルを作ること

納得できるテストをするためのモデリング講座 より引用

結局、モデルって何？

（私たちがテストをする状況において）モデルとは
物事を理解しやすく、分かりやすくするために整理して書いたもの



UML※1もデシジョンテーブルもフィッシュボーンもみんなモデル

• 書き方に決まりがないと読み手に正確に伝わらないので、書き方のルールを決めることが多い。

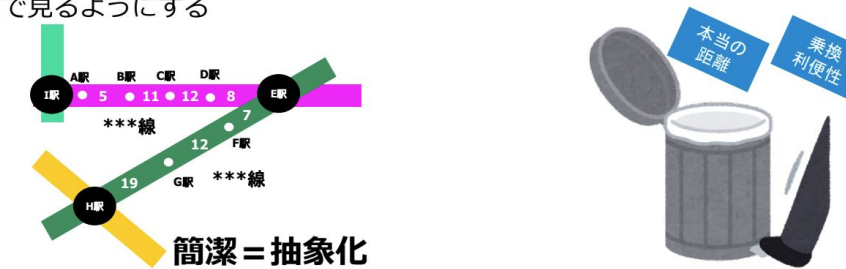
※1 UML（Unified Modeling Language 統一モデリング言語）：複雑なシステム開発を、記述方法を統一したモデルで表現することで読みやすさやコミュニケーションの効率アップに役立ってる

6 © Copyright 2021 ytte Lab Ltd. The information contained herein is subject to change without notice.

モデリングをするときの注意点

情報の多くは捨てられてしまう

- 知りたいことも捨てられてる場合があるので、知りたいことを取りこぼさないよういくつかの側面から見るようにする



共通の言葉やルールを守るだけだと、かえってわからないものになる

- 共通の言葉やルールは理解を補助するだけなので、その前に、知りたいこと（目的や観点）をはっきりさせる

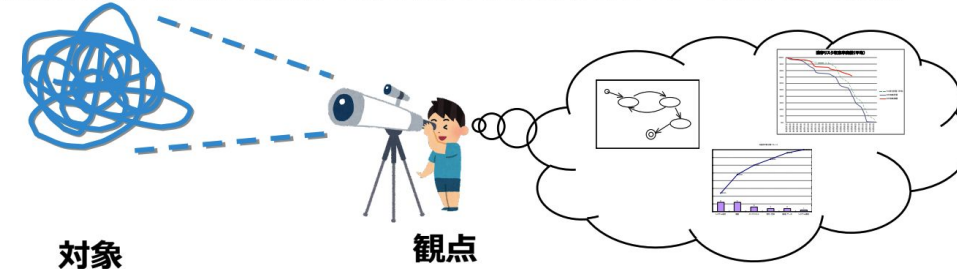


14 © Copyright 2021 ytte Lab Ltd. The information contained herein is subject to change without notice.

モデリングをする効果

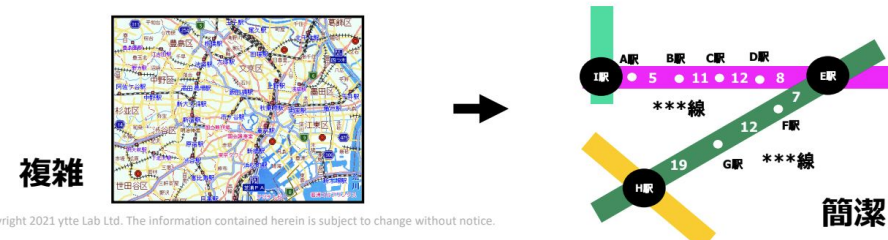
知りたいことがスーッとわかるようになる

- そのために、ある観点で対象から知りたいことを取り出して、不要なものを削る



みんなの知識や経験を共有できる

- そのために、知りたいことに対する共通の言葉やルールを作って簡潔に示す

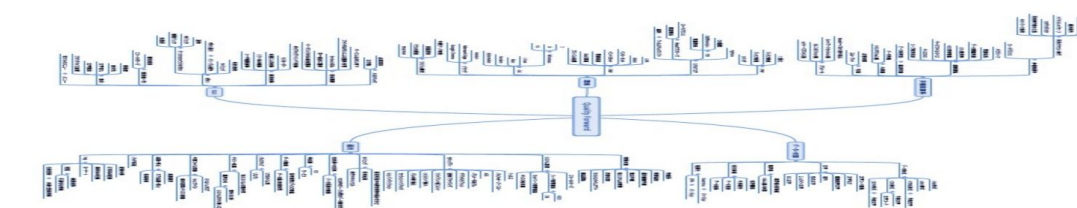


13 © Copyright 2021 ytte Lab Ltd. The information contained herein is subject to change without notice.

モデルリングの際に意識して欲しいこと

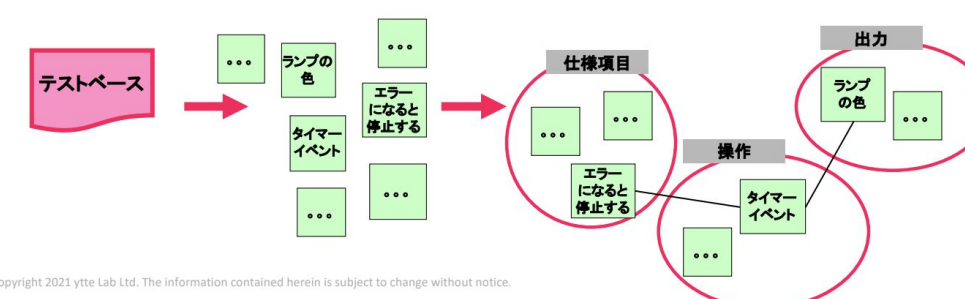
絵を描いただけではモデリングをしたとはいえない

- 絵を描いて終わりにせず、描いたものから、わかったことを説明できるようにしていく



モデリングの際は、分類して、整理して、関係を考える

- いろいろな側面に分類、整理して、知りたいことが説明できるようにしていく

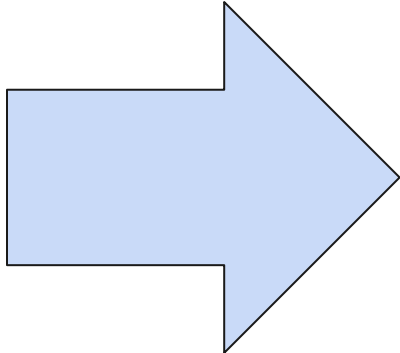


15 © Copyright 2021 ytte Lab Ltd. The information contained herein is subject to change without notice.

4.3.1 システム要求の分析

分析とは対象をしっかりと理解すること

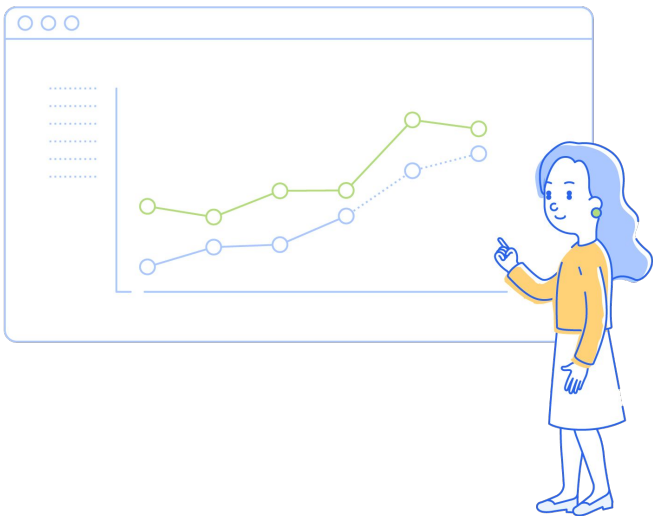
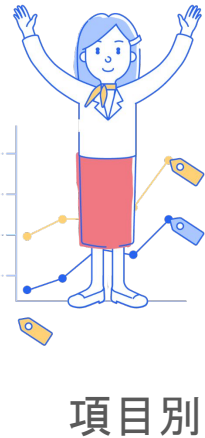
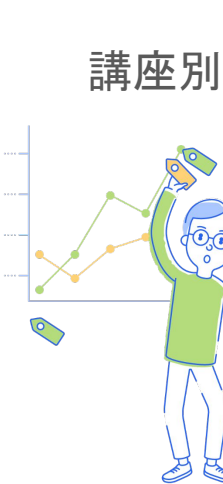
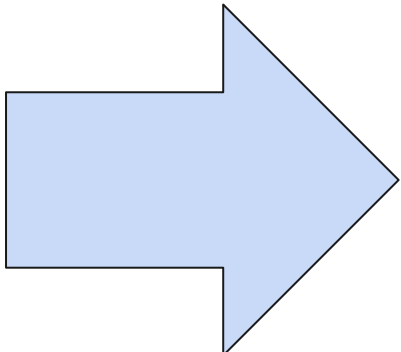
- 例:食品の成分分析
 - 食品の中に何が含まれているのかを定量的に明らかにし、食べようとしている物をよく理解すること



ピザMサイズ

重量	500g
エネルギー	872 kcal
たんぱく質	39.73 g
脂質	35.72 g
炭水化物	97.9 g
食物繊維(総量)	7.26 g
カリウム	759.2 mg
カルシウム	77.45 mg
リン	826.2 mg
鉄	2.81 m

- 例:新人教育の理解度確認テスト結果分析
 - 結果をいろいろな側面に分類、整理して、教育の効果や問題点を理解すること



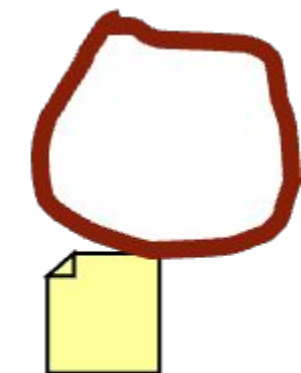
4.3.1 システム要求の分析

システム要求を理解するとは、要求を実現するのにシステムが備えるべきWhatを特定すること

システム要求を理解し、要求の不備を洗い出す(理解には背景にあるドメインや実務の知識が重要)

- 要求の抜け漏れ、重複
- 曖昧性
- 要求間の不整合

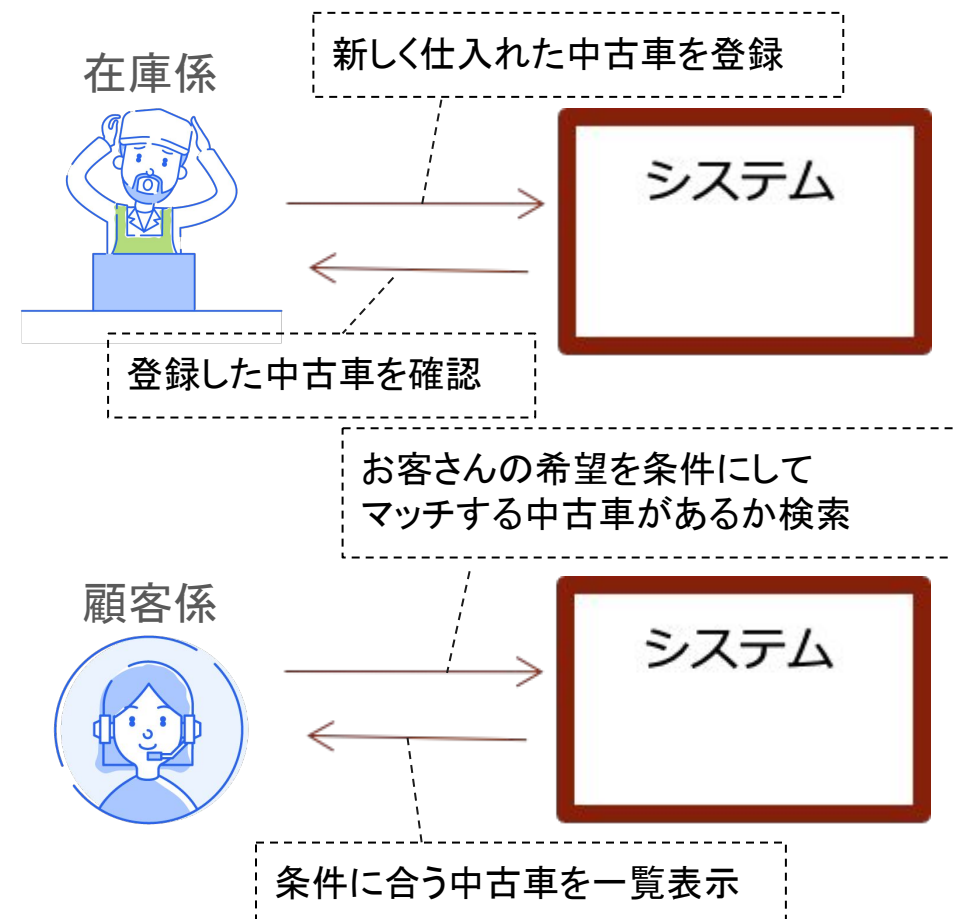
お客様のほしい中古車が
販売可能かすぐに提示したい



システム
要求

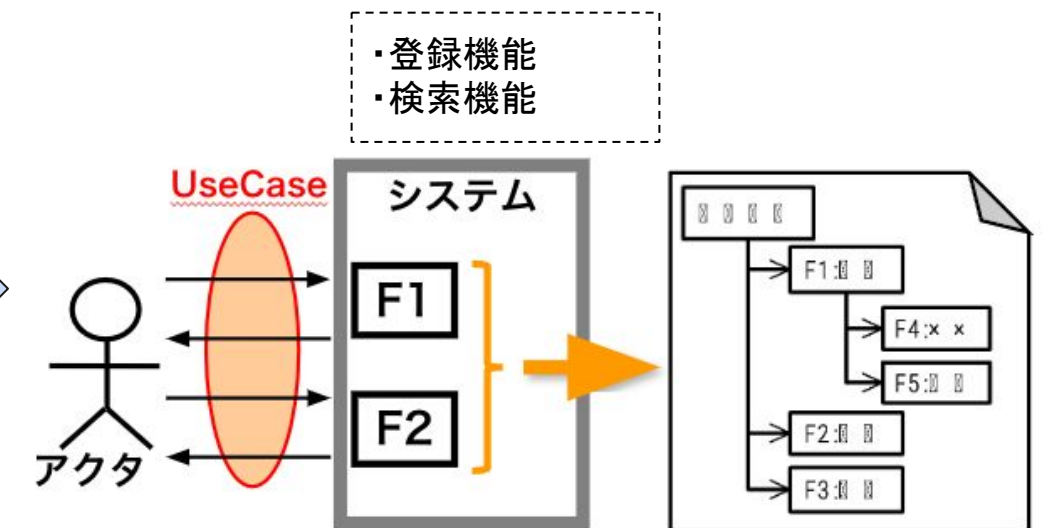
シナリオ分析

要求を具体化



機能分析

シナリオの実
現に必要な機
能を特定



モデル
ベース
分析

機能の意味を
明確にする

必要なデータ



必要な状態



4.3.2 システム要求の仕様化 (Specify)

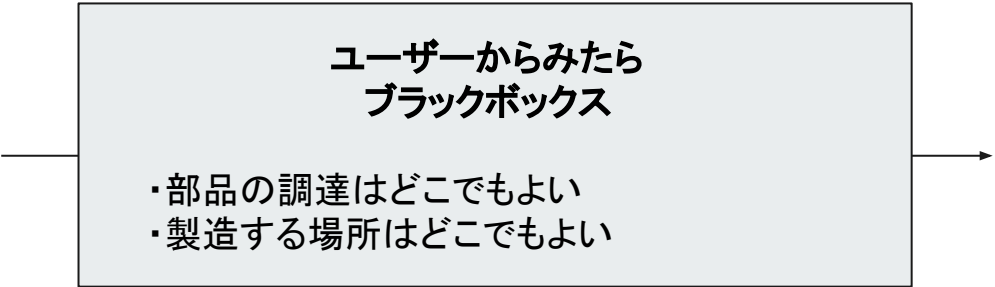
仕様 (Spesification) とは、その内部の構造の詳細でなく、システム/プロダクトの特徴や性能などを定義したもの

- 仕様化とは、要求をシステムとして実現するために提供する、機能や性能を記述すること
 - 仕様は、「設計・実装を制限しない」が、「設計・実装の確認に使える」

仕様

14インチ MacBook Pro (M4)	14インチ MacBook Pro (M4 Pro or M4 Max)	16インチ MacBook Pro (M4 Pro or M4 Max)
14.2インチ Liquid Retina XDRディスプレイ	14.2インチ Liquid Retina XDRディスプレイ	14.2インチ Liquid Retina XDRディスプレイ
Apple M4チップ	Apple M4 Proチップまたは Apple M4 Maxチップ	Apple M4 Proチップまたは Apple M4 Maxチップ
Apple Intelligence対応	Apple Intelligence対応	Apple Intelligence対応
10コアCPU	最大16コアCPU	14コアまたは16コアCPU
10コアGPU	最大40コアGPU	最大40コアGPU
16GBから32GBのユニファイドメモリ	24GBから128GBのユニファイドメモリ	24GBから128GBのユニファイドメモリ
512GBから2TBのストレージ	512GBから8TBのストレージ	512GBから8TBのストレージ
最大24時間駆動バッテリー	最大22時間駆動バッテリー	最大24時間駆動バッテリー
Touch ID	Touch ID	Touch ID

設計、実装



プロダクト

仕様通りになっている

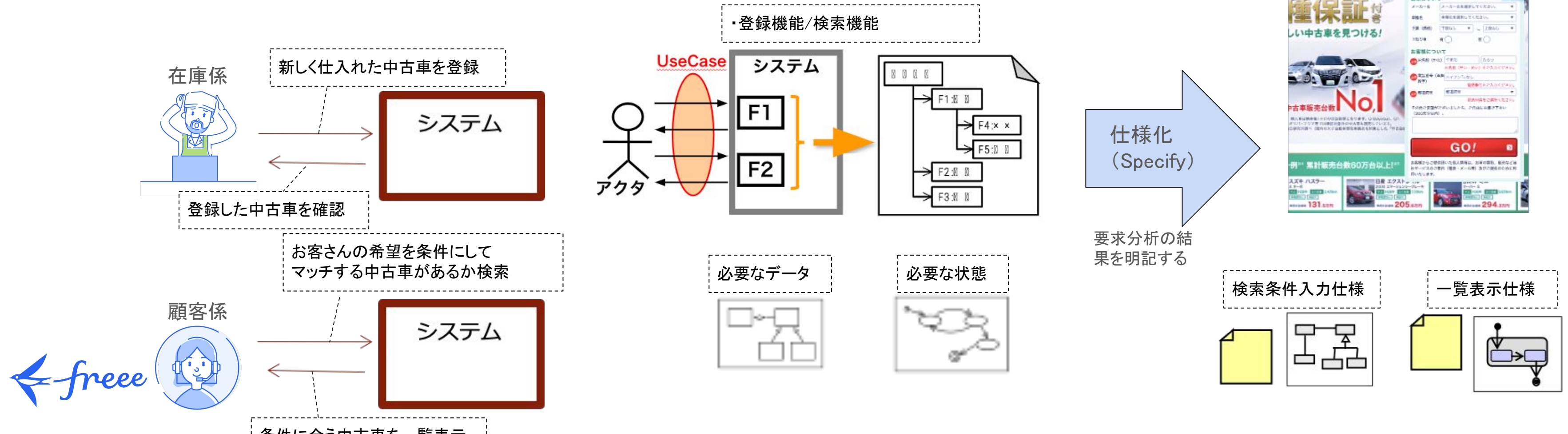


4.3.2 システム要求の仕様化(Specify)

システム要求の仕様化とは、要求分析した結果を文書化して、その後の設計、構築のインプットにすること

仕様記述の方法

- 自然言語
- 自然言語＋モデル
- 形式手法



4.3.2 システム要求の仕様化 (Specify)

仕様記述の注意点

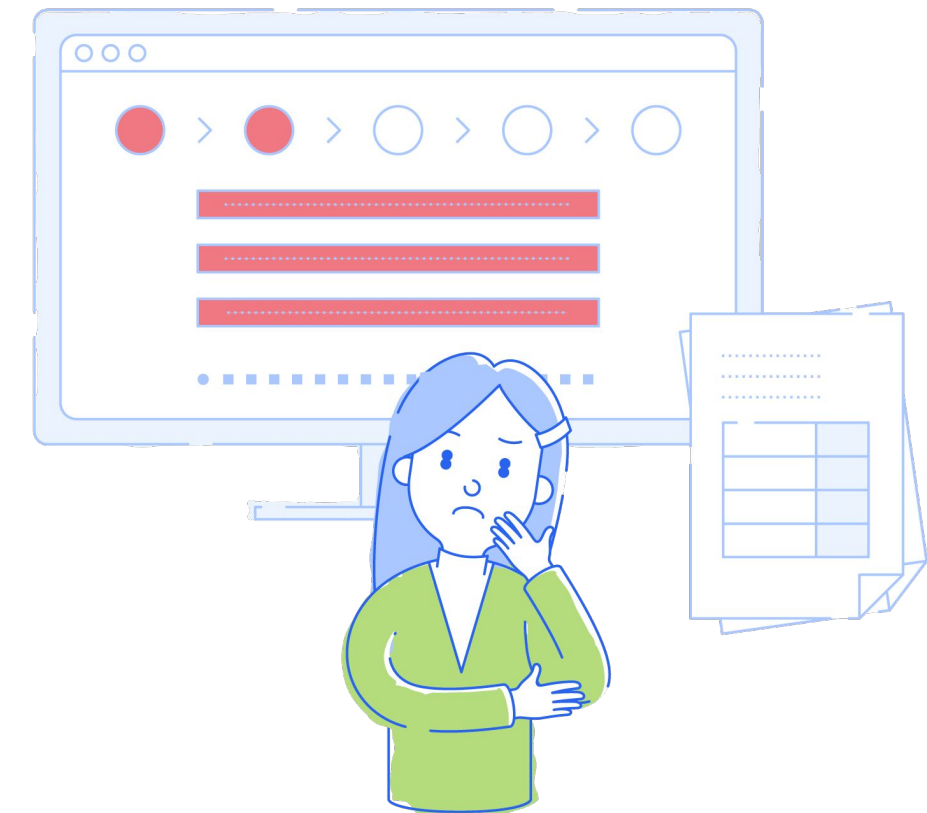
注意点	悪い例	良い例
明確であること	温度が <u>高温</u> になったらファンを回転させる	温度が <u>200度以上</u> になったらファンを回転させる
完全であること	予約機能ではシステムの現在日時より先の日時を入力できる。	予約機能ではシステムの現在日時より先の日時を入力できる。 <u>現在日時以前の値を入力した場合は、入力できないことをユーザーがわかるメッセージを表示する。</u>
一貫していること	スタートボタンを押下したら機能一覧画面を表示する。 — 閉じるボタンを押下したら <u>機能選択画面</u> を閉じる。	スタートボタンを押下したら機能一覧画面を表示する。 — 閉じるボタンを押下したら <u>機能一覧画面</u> を閉じる。



4.3.2 システム要求の仕様化(Specify)

システム要求を仕様化する必要性

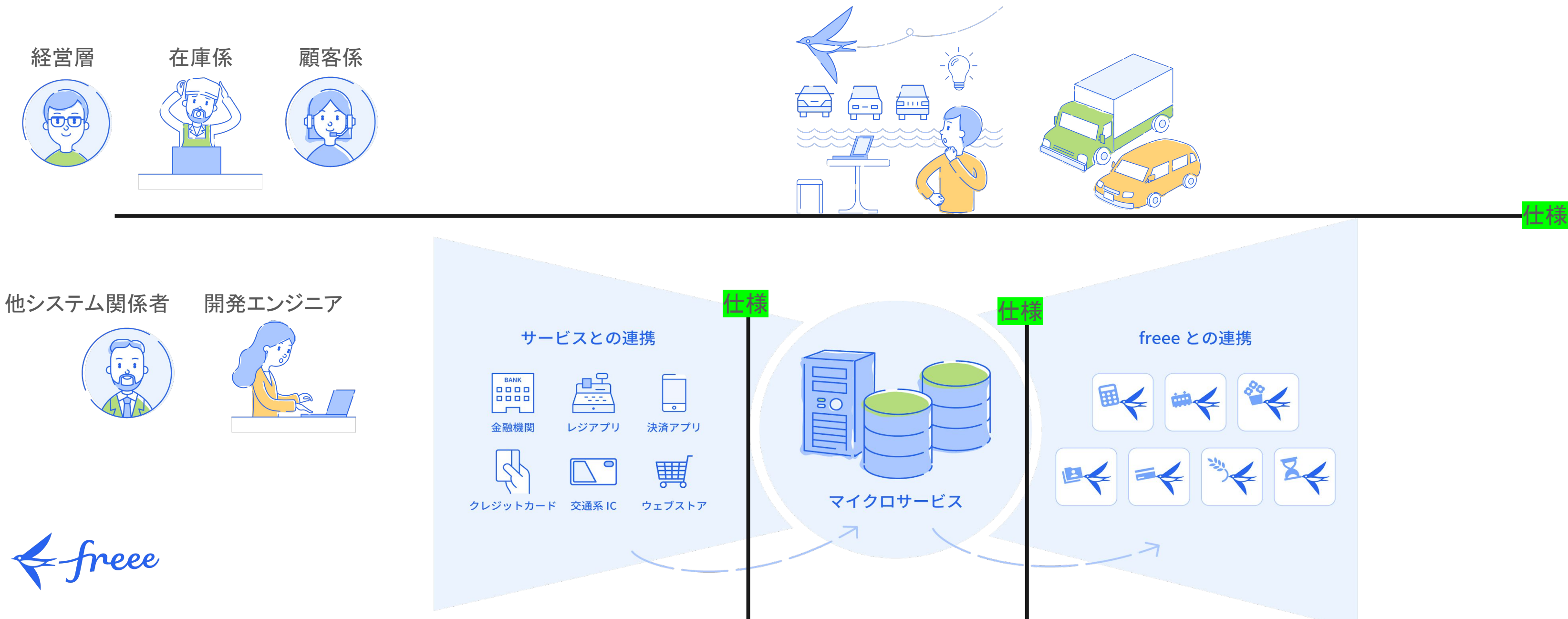
- 作る対象が明確に定義されないとするとき…
 - 設計・構築のやり直しが発生する
 - 関係者間で作る対象が共有されない、作る対象が試行錯誤で変動する、仕様の抜け漏れバグが後工程で顕在化する
 - 追加開発が間に合わない→無価値
 - 見積りができない
 - 工数、期間、費用、リスク
 - アウトソーシングができない
 - 対象が正しく設計、実装がされたか確認できない



[参考]横の仕様と縦の仕様

仕様には「横の仕様」と「縦の仕様」の2種類がある

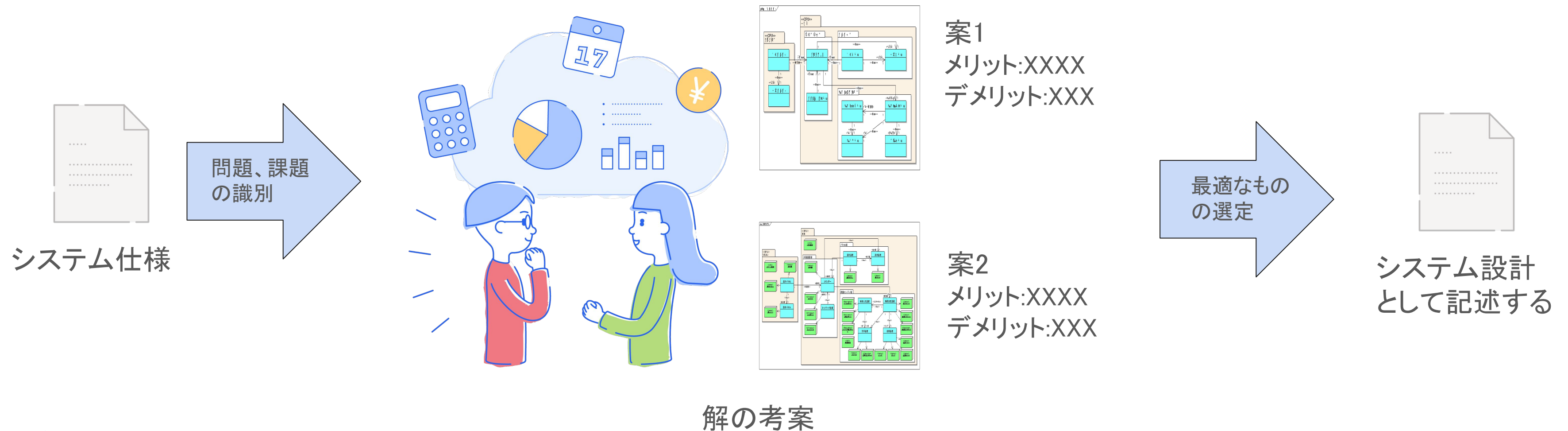
- 横の仕様＝利用者の要求を分析して仕様化したもの
- 縦の仕様＝他マイクロサービス、他社サービスとの連携を分析して仕様化したもの



4.3.3 設計(システム設計、ソフトウェア設計)

設計とは、仕様を実現するシステム、ソフトウェアの最適な構造を決めること

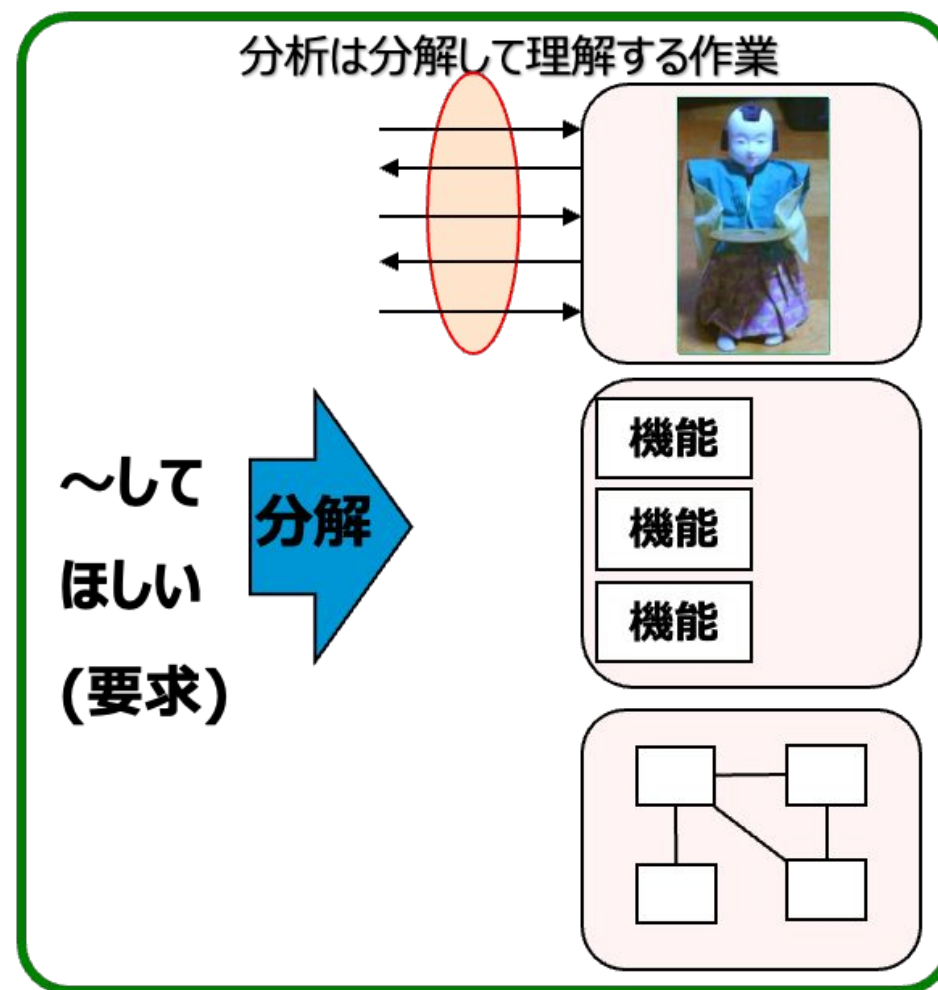
- 仕様の実現に伴う問題・課題を識別し、解を考案して最適なものを選定する



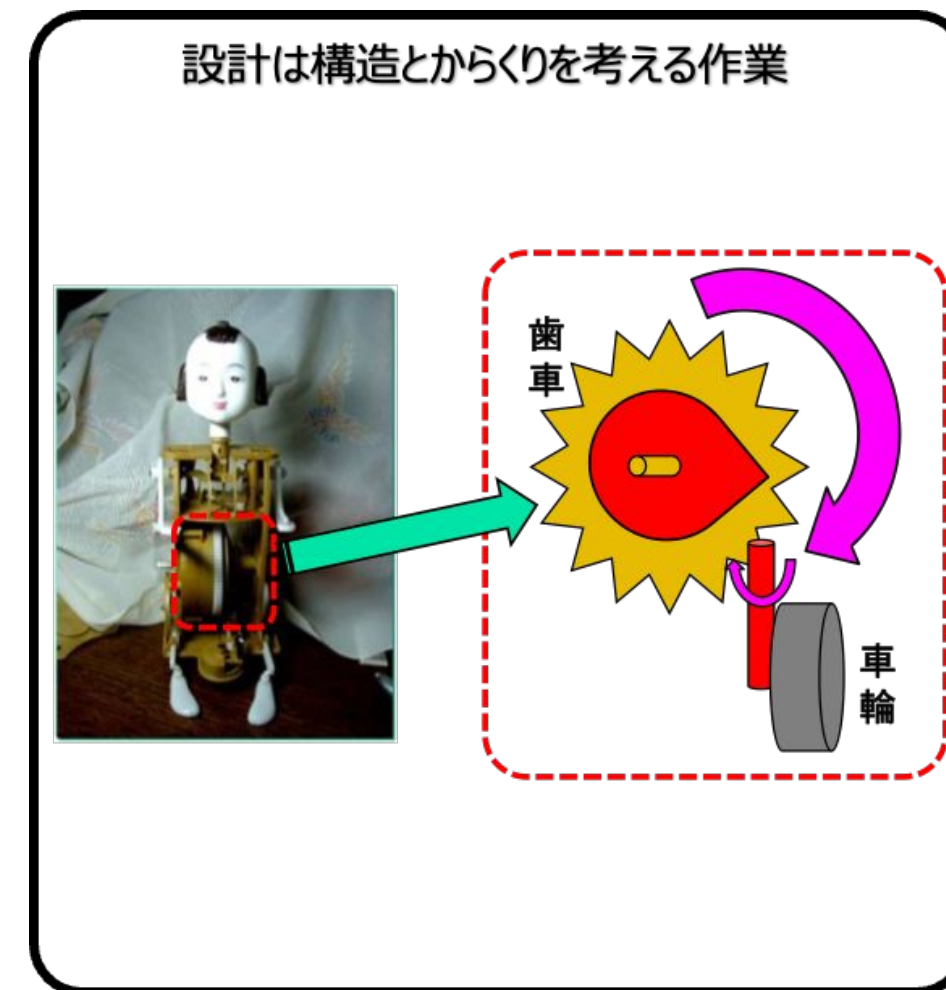
4.3.3 設計(システム設計、ソフトウェア設計)

分析と設計の違い

- 分析は分解して理解する作業、設計は作るものの構造とからくりを考える作業



分析するとは、開発する人が、いろいろな方法で「～してほしい」を分解し、その結果、「～してほしい」とは、「～ということです」と言えるようになること

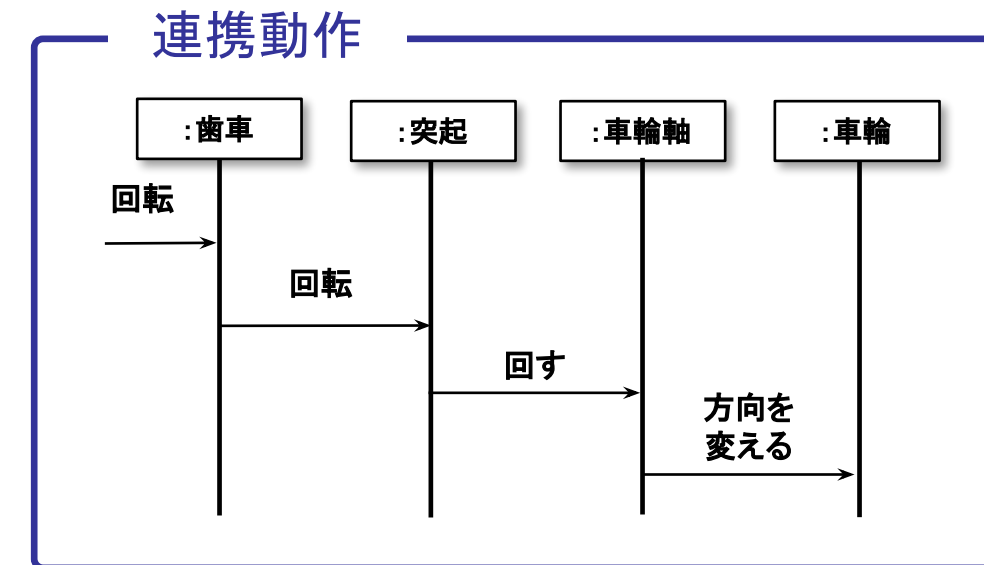
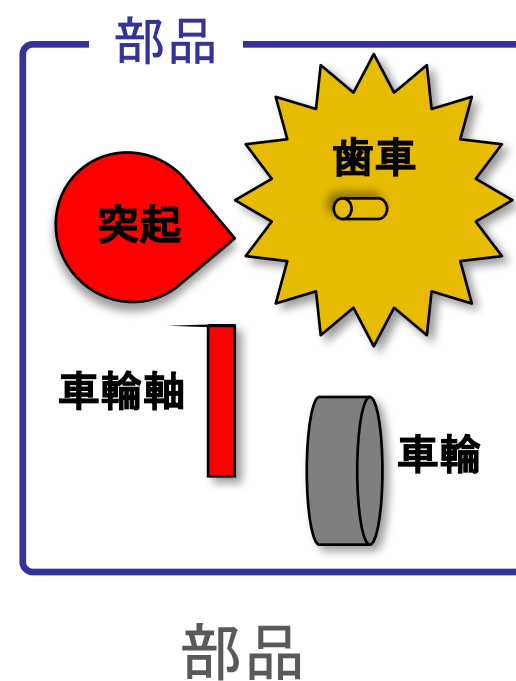
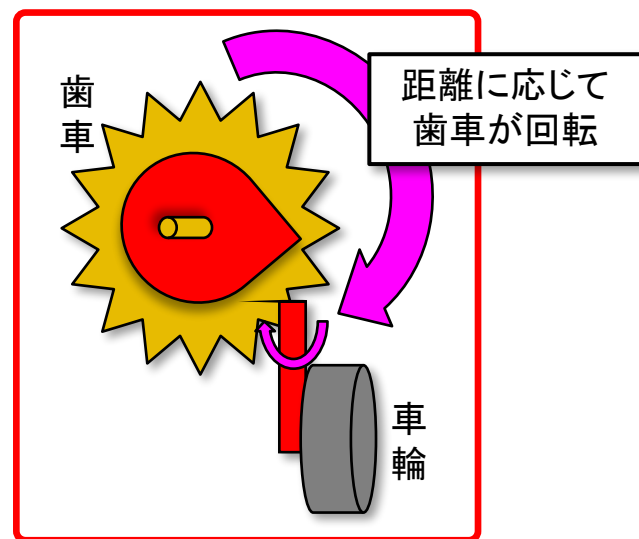


設計するとは、開発する人が、ソフトウェアの構造とからくりを決めることで、「こう作れば実現できます」と言えるようになること

4.3.3 設計(システム設計、ソフトウェア設計)

からくり(メカニズム)とは、課題を解決する中核となる機構のこと

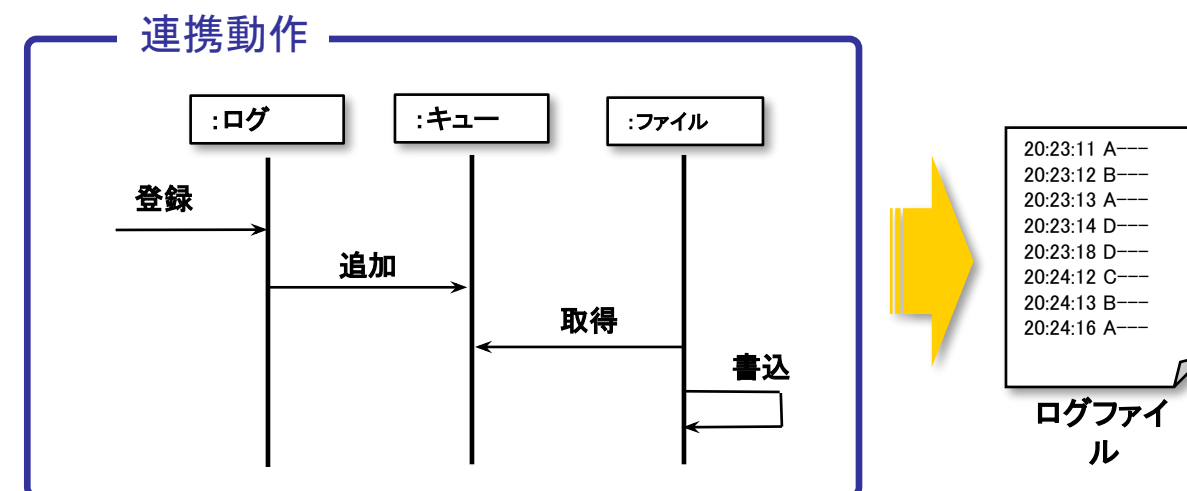
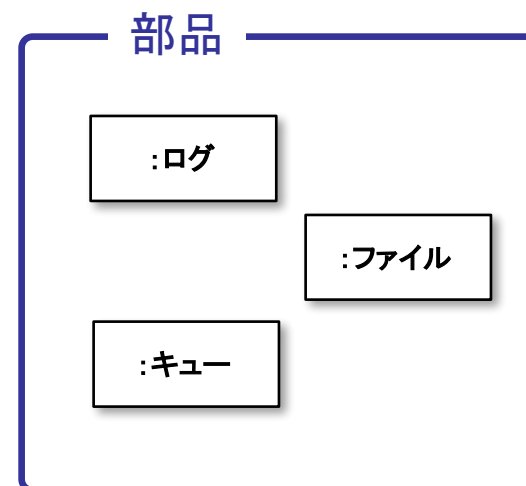
- メカニズム = 部品 x 部品の連携動作
- からくり人形がUターンをするメカニズムの場合
 - 仕様: 客にお茶を運び、Uターンして主人の元に戻る
 - 課題: 進行方向を変えるタイミングを知る方法が無い
 - 連携動作: 歯車が回転し、突起の部分が軸を回すことによって、車輪の方向を変える



4.3.3 設計(システム設計、ソフトウェア設計)

からくり(メカニズム)とは、課題を解決する中核となる機構のこと

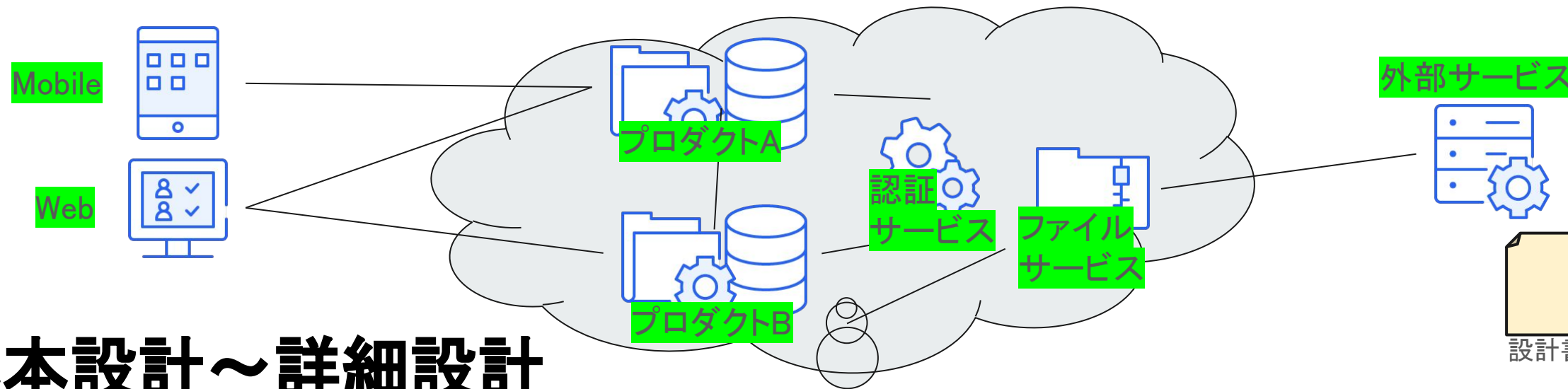
- メカニズム = 部品(コンポーネント)x 連携動作(コンポーネントの処理シーケンス)
- ソフトウェアのログ管理を行う機構
 - 仕様: システムの各機能の実行した内容をファイルに記録する
 - 課題: 同時に大量のログが発生するとログを取りこぼす
 - 連携動作: ログがキューに実行内容を追加し、ファイルに登録された実行内容を取得して書込む



4.3.3 設計(システム設計、ソフトウェア設計)

システムアーキテクチャ設計

非機能要件(性能、信頼性、保守性、セキュリティなど)やコストで決める



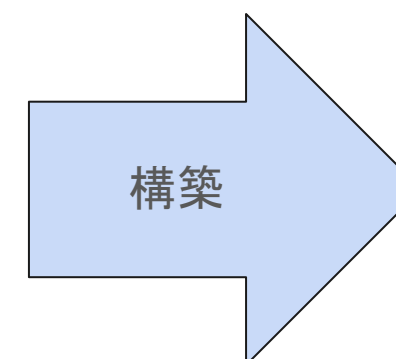
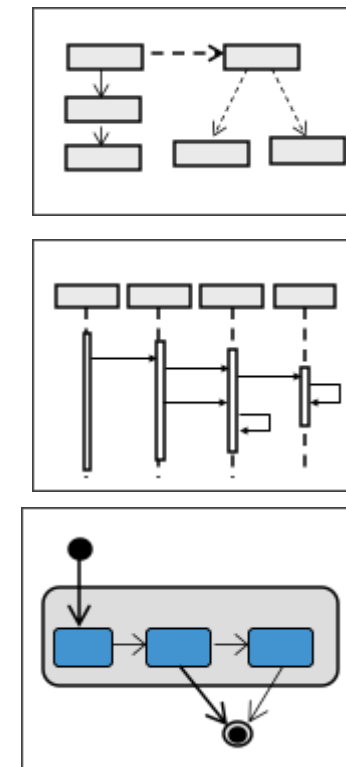
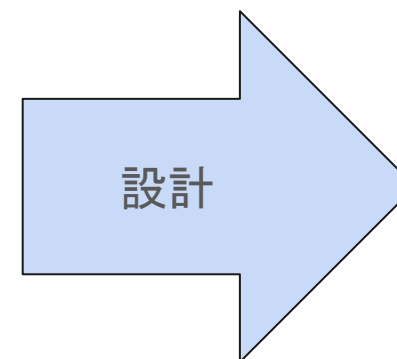
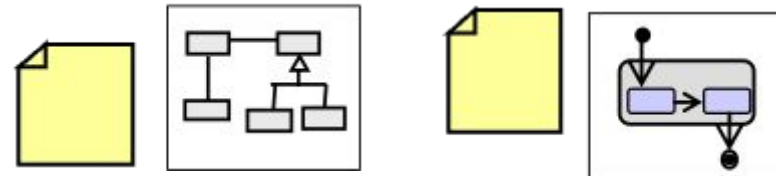
ソフトウェア基本設計～詳細設計

要求の仕様化で決めた機能を具体的にどう実現するかを決める(からくりと部品)



検索条件入力仕様

一覧表示仕様



4.3.3 設計(システム設計、ソフトウェア設計)

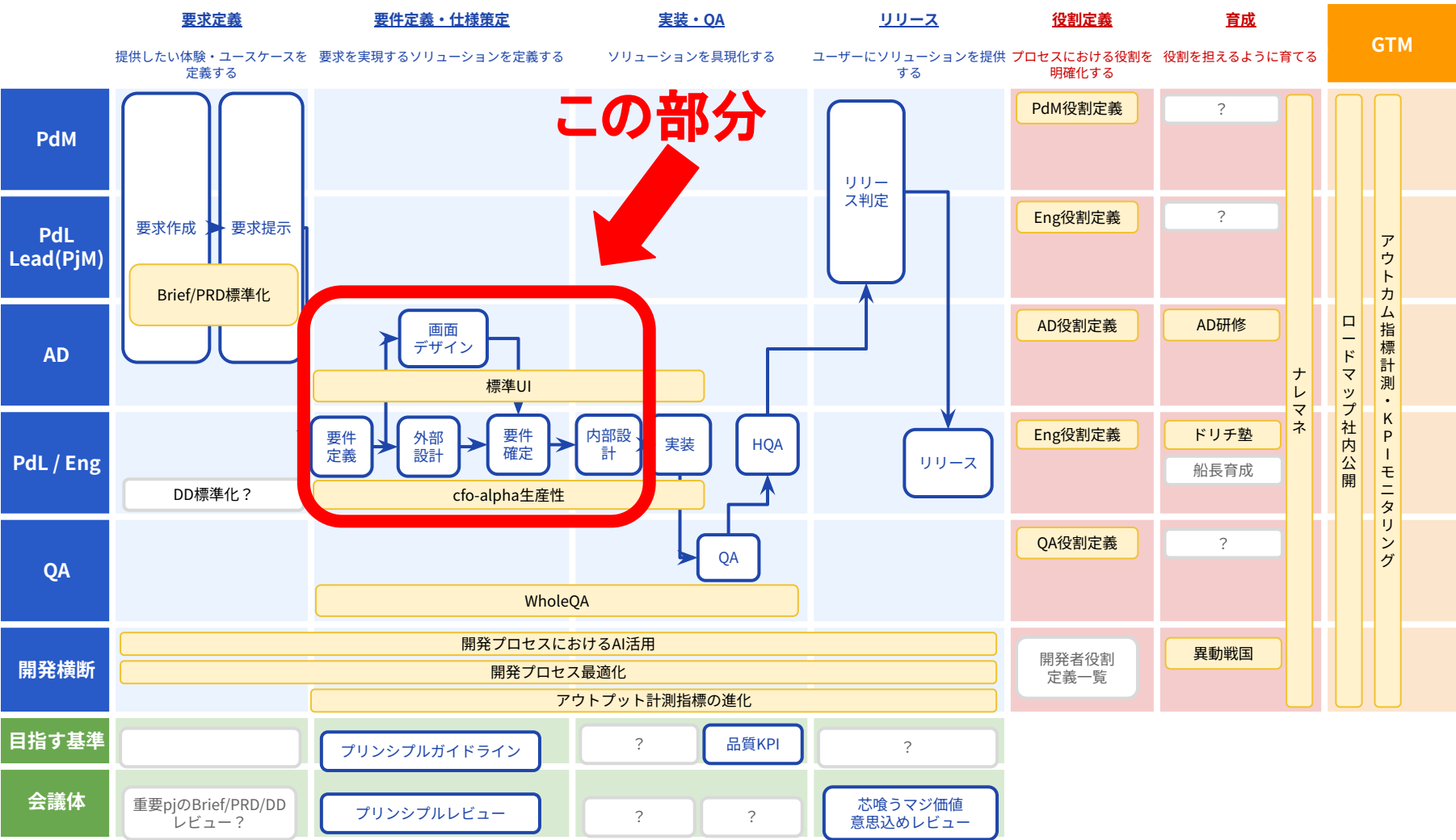
設計の必要性

- システムの機能の増大・複雑化にともない、コードレベルで構造的な問題/課題を取り扱うのは非常に困難になっている(より高い抽象レベルでの思考が必要)
 - 構造や埋め込まれている仕組みを読み取ることさえ困難になっている
- プログラム作成中にコードベースで設計をやると、場当たりの・目先の対応になってしまう
 - 結果、コードはますます複雑化
- 適切な設計がないと
 - テスティング段階で構造的な問題の顕在化により、大きな手戻りが発生してしまう
 - 機能追加・仕様変更に膨大な工数・時間が必要になってしまう



4.3.4 フリーの開発でやってること（モデリング）

FY25開發生産性PJから



2024年10月の時点でまとめた開発プロセスから

- フリーの開発プロセスを書いてみた(外部公開なし)
- 主要成果物
 - UIデザイン
 - (新規)親DesginDoc(概要)
 - (新規)GitHubリポジトリ
 - DesginDoc(詳細)
 - (新規)インフラ構成図
 - (新規)基盤で利用するもの
 - 脆弱性診断判断結果
 - (新規)グローバルリソース

フリー社内のモデリングに属する活動についてのお役立ちリンク(外部公開なし)

- DD生成の仕組み
- リスク洗い出し会

4.4 構築

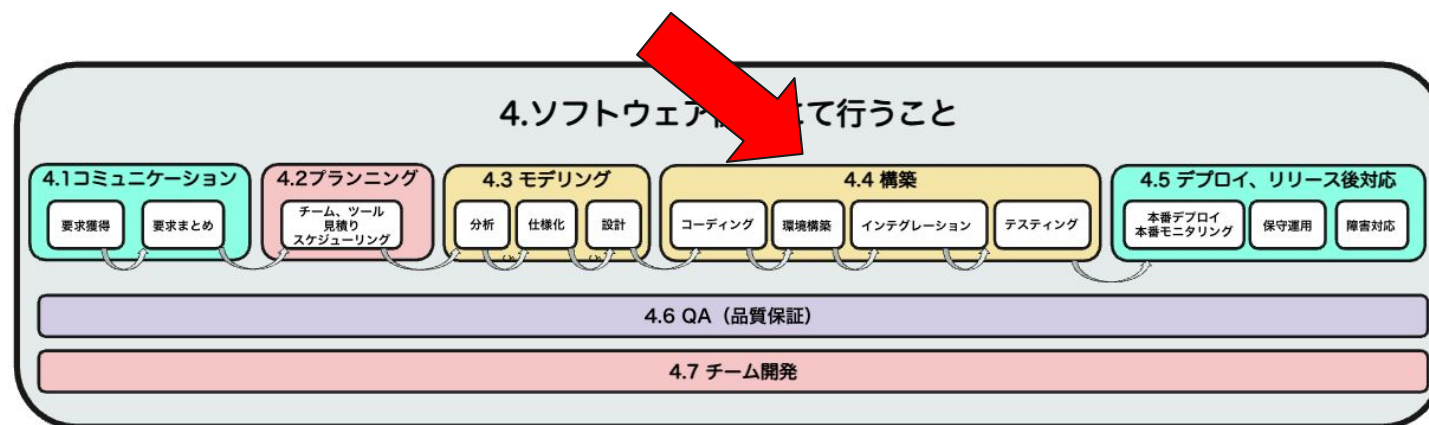
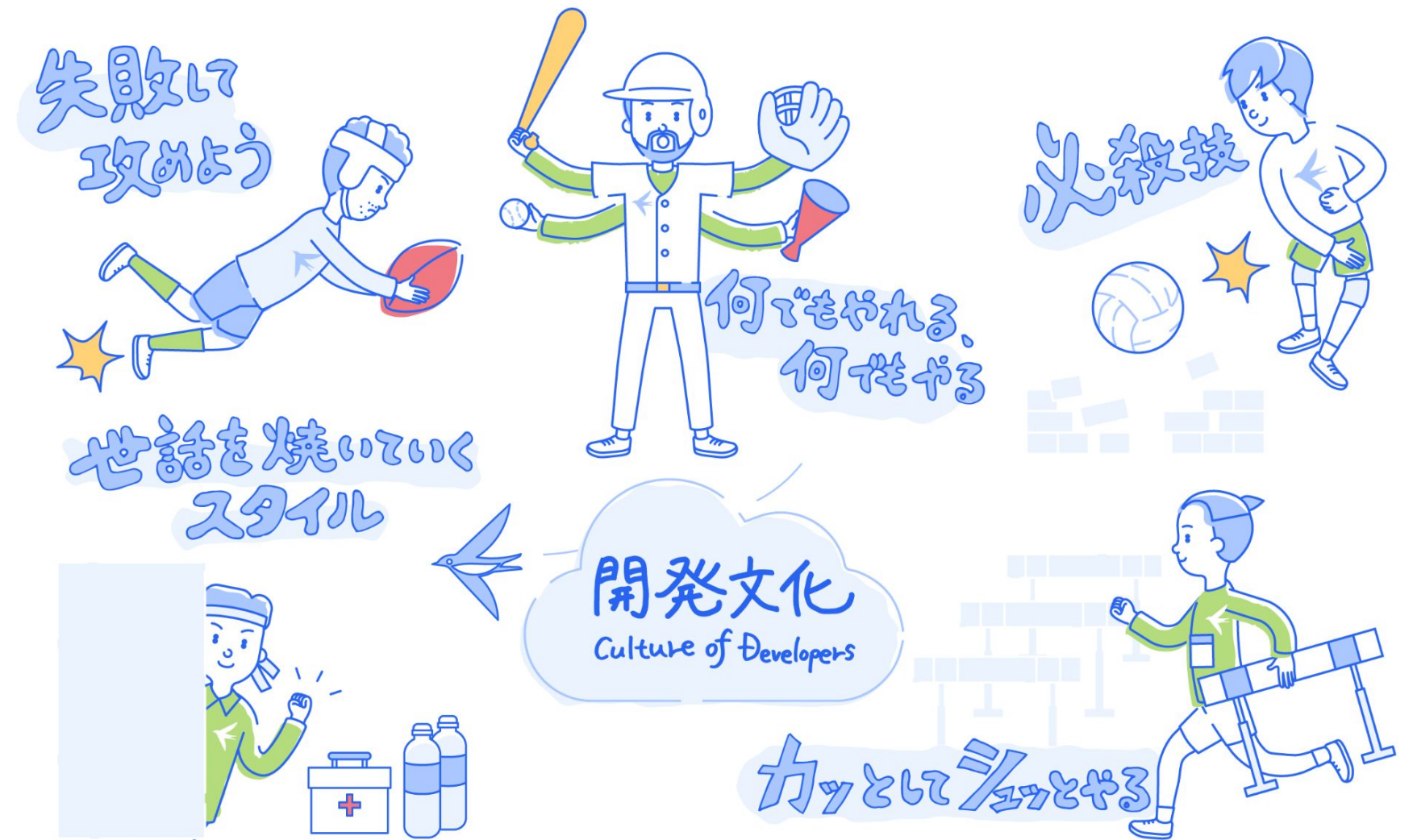
4.4.1 コーディング

4.4.2 環境構築

4.4.3 インテグレーション

4.4.4 テスティング

4.4.5 フリーの開発でやってること



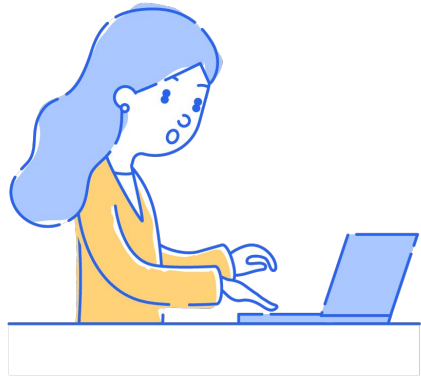
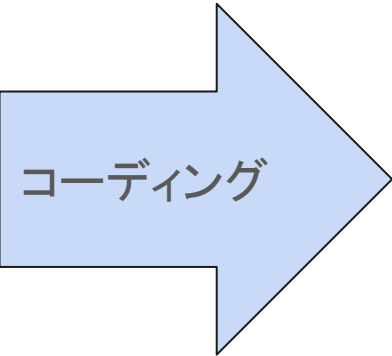
モデルをもとに行う構築では、手動または自動でのコード生成、コード内のエラーを発見するテストが含まれる。

ロジャー・プレスマン; ブルース・マキシム. 実践ソフトウェアエンジニアリング (第9版) (p. 8). (Function). Kindle Edition.

4.4.1 コーディング

コーディングとは、設計を基に動くコードを作ること

- 最終的にユーザーが直接体験する機能を具体化する重要なステップ
- ツール、規約、作法を駆使し、設計を正確にコードにする



```
package carsearch

import (
    "errors"
)

type Car struct {
    Price int
    Mileage int
}

type SearchConditions struct {
    MaxPrice int
    MaxMileage int
}

type CarRepository interface {
    FindAll() ([]Car, error)
}

type CarSearchService struct {
    repo CarRepository
}

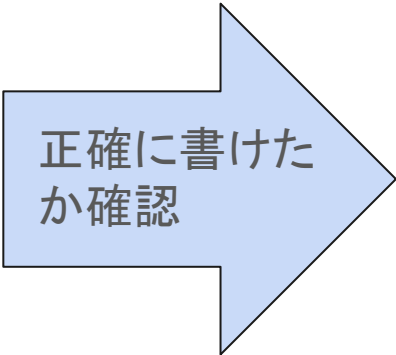
func NewCarSearchService(repo CarRepository) *CarSearchService {
    return &CarSearchService{repo: repo}
}

func (s *CarSearchService) SearchCars(cond *SearchConditions) ([]Car, error) {
    if cond == nil {
        return nil, errors.New("検索条件が指定されていません")
    }

    allCars, err := s.repo.FindAll()
    if err != nil {
        return nil, err
    }

    var result []Car
    for _, car := range allCars {
        if car.Price <= cond.MaxPrice && car.Mileage <= cond.MaxMileage {
            result = append(result, car)
        }
    }

    return result, nil
}
```



コードが想定通りに動くかテスト



コードレビュー



4.4.1 コーディング

コーディングに大事なツール、作法、規約

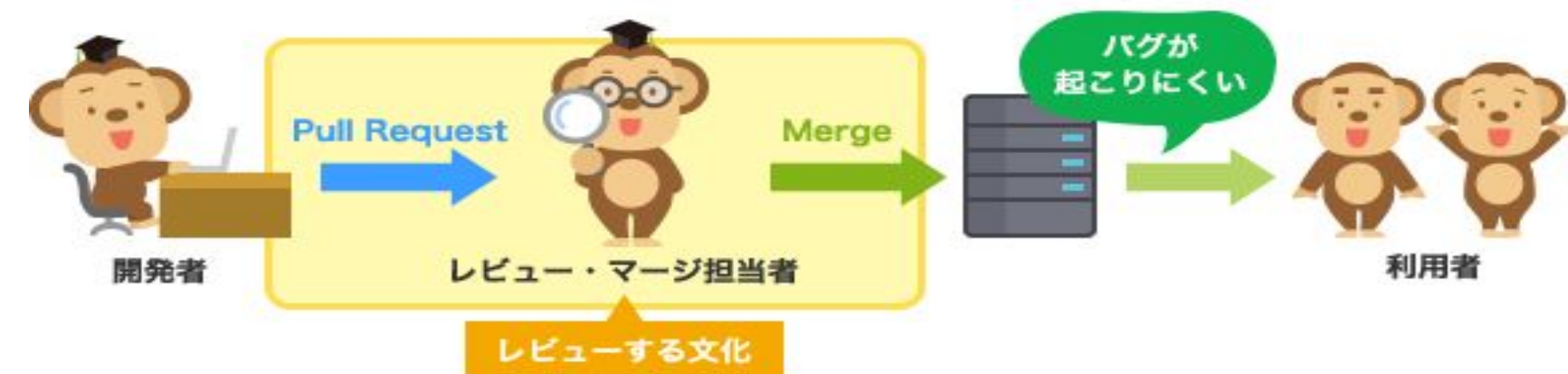
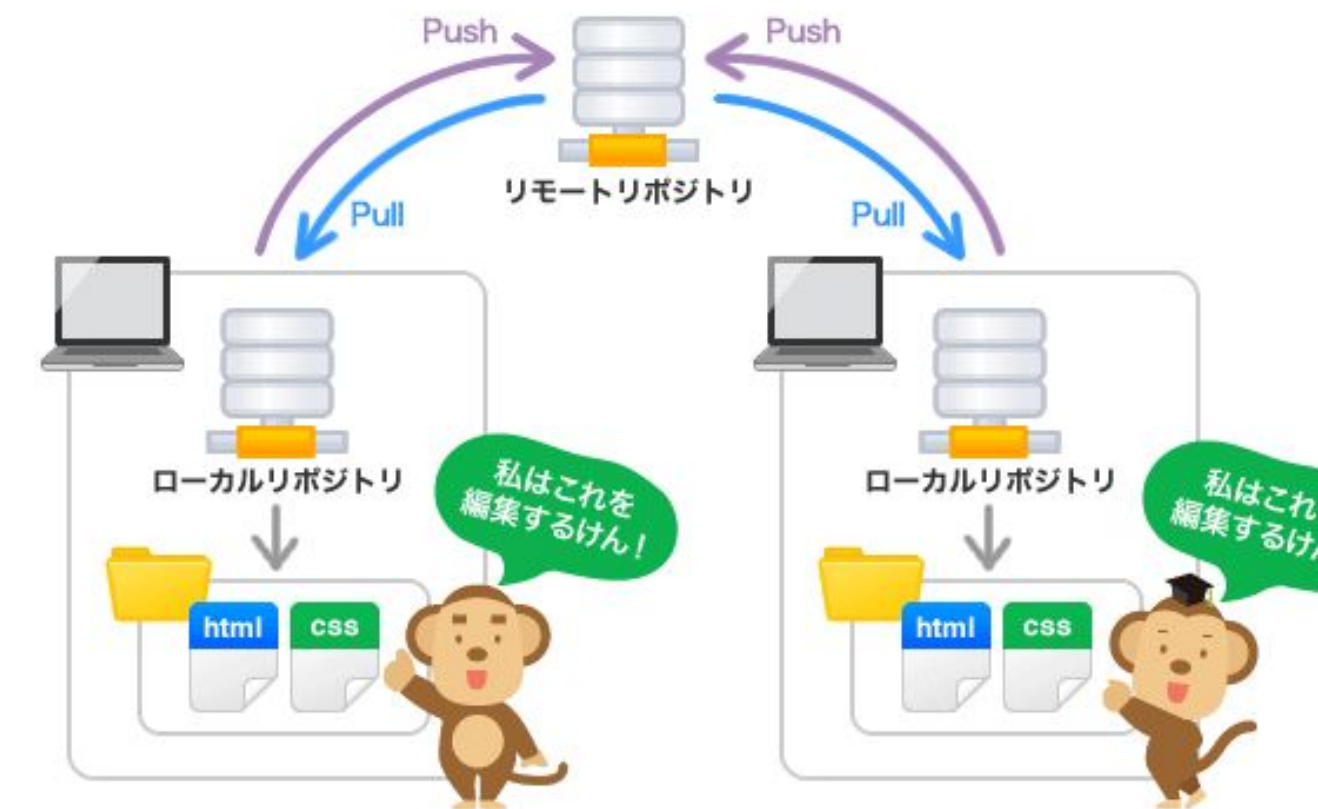
- コーディング規約 (Coding Standards) の遵守
 - チームで統一された規約を使う(変数名、メソッド名などの命名ルール、インデントや括弧の配置など)
- 可読性・保守性の高いコードを書く
 - 処理をできるだけ小さな単位に分割し、関数・メソッドの責務を明確にする
- エラーハンドリングと例外処理
 - 例外が発生した場合にどう対処するか、きちんと設計・実装する
 - 同時にログ出力やリカバリ手段を検討し、後工程(テストや運用)でトラブルシュートできるようにする
- テストコードの作成
 - 実装と同時または先行して(TDD : テスト駆動開発)テストコードを作る
 - ユニットテストで基本的なロジックのバグを早期発見する
 - ユニットテストツール(RSpec、Goテストパッケージ)
- バージョン管理ツール(GitHub など)の活用
 - バージョン管理によってコードベースを安全に分けて複数人で並行作業できる
 - コミットメッセージの書き方(何を・なぜ変更したか)
 - コードレビューのフロー



4.4.1 コーディング

コーディングにおけるバージョン管理 (GitHub) の基本

- バージョン管理の目的 – チーム開発必須ツール
 - 変更履歴の追跡: いつ・誰が・どこを修正したかがわかる
 - 複数人開発への対応: 衝突を防ぎ、安全に並行作業
 - 過去状態へのロールバック: 問題発生時にすぐ戻せる
- GitHub の基本概念
 - リポジトリ: ソースコードと変更履歴を保存する場所
 - ブランチ: 独立して機能開発や修正を行う、作業用コピーのようなもの
 - コミット: 「コード変更を論理的に意味のあるひとまとまり」で記録
 - 『Gitによるバージョン管理(岩松 信洋、上川 純一、まえだ こうへい、小川 伸一郎)』より引用
 - プルリクエスト (PR): 他メンバーにレビューを依頼し、マージする仕組み
- 開発フローのイメージ
 - ブランチ作成 → 新機能/修正の作業開始
 - 定期的にコミット & プッシュ
 - PR作成 → コードレビュー
 - マージ → リモートリポジトリへブランチを統合



4.4.1 コーディング

コーディングの典型的な流れ

- 設計ドキュメントの確認
 - 実装する機能について確認
- ブランチ作成
 - 自分の手元でコードを書けるようにブランチを分ける
- コード実装・ユニットテスト
 - 自分で書いたコードは、必ず自分の思った通りに動くかテストする
- コミット・プッシュ
 - 適宜コミットメッセージをつけてリポジトリへプッシュする
- コードレビュー
 - PRを出してチームでレビューをしてもらう
- 修正とマージ
 - レビュー指摘を反映し、ブランチを main/develop にマージ



4.4.2 環境構築

環境構築とは、ソフトウェアが動く土台を作ること

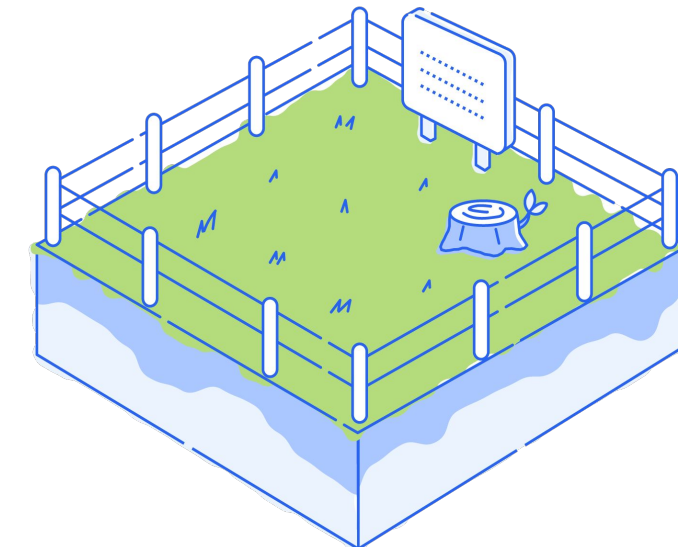
- コードが動作する「場所」を用意し、実行・保守できる状態にする
- ネットワークやサーバー、データベースなど、アプリケーションを動かすために必要なリソースを準備する

- どういうことをするか？

- サーバーや仮想マシンを用意
- ネットワークの設定（外部からのアクセスや通信経路など）
- ミドルウェア（Webサーバー、データベースなど）のインストール
- セキュリティ対策（ファイアウォールやアクセス制御）

- オンプレミス（自社サーバー） vs. クラウド

- **オンプレミス**：物理サーバーを自分たちで購入・設置・管理（初期コスト大、細かい制御可）
- **クラウド**：AWSやGCP、Azureなどのサービスを利用（初期コスト小、スケールしやすい）



4.4.2 環境構築

クラウドを使った環境構築

- 基本概念

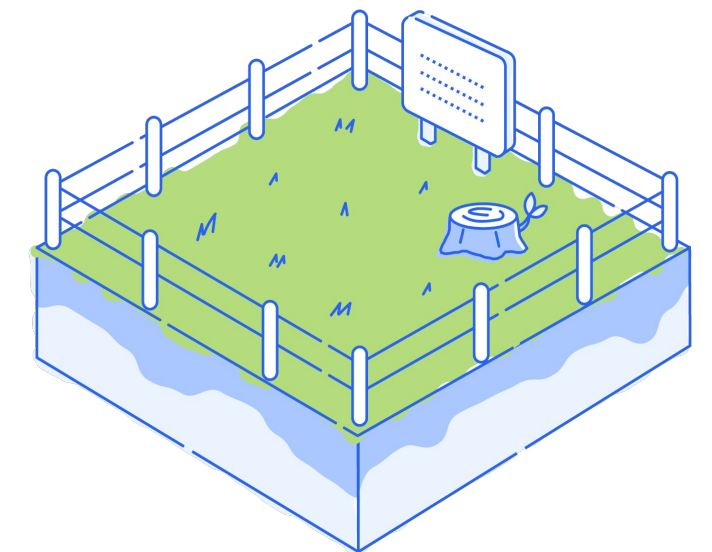
- IaaS (Infrastructure as a Service): 仮想マシンやネットワークなど「インフラ」を提供
- PaaS (Platform as a Service): データベースやアプリ実行基盤など「開発プラットフォーム」を提供
- SaaS (Software as a Service): アプリをサービスとして利用(例: Google Workspace など)

- 主要クラウド例

- AWS: EC2: 仮想サーバー、RDS: データベース、EKS: Kubernetes運用) など
- GCP: Compute Engine、Cloud SQL、GKE (Google Kubernetes Engine)
- Azure: Virtual Machines、Azure SQL Database、AKS (Azure Kubernetes Service)

- メリット

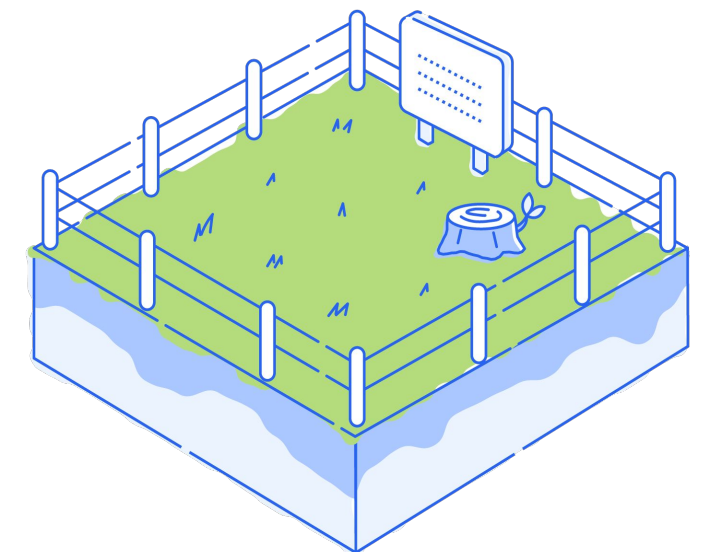
- すぐにサーバーを準備できる(環境のセットアップが短時間)
- スケールしやすい(アクセス増加にあわせて自動的にリソースを拡張)
- 保守が楽(物理的なサーバー管理が不要)



4.4.2 環境構築

環境構築の進め方: 自動化と標準化がカギ

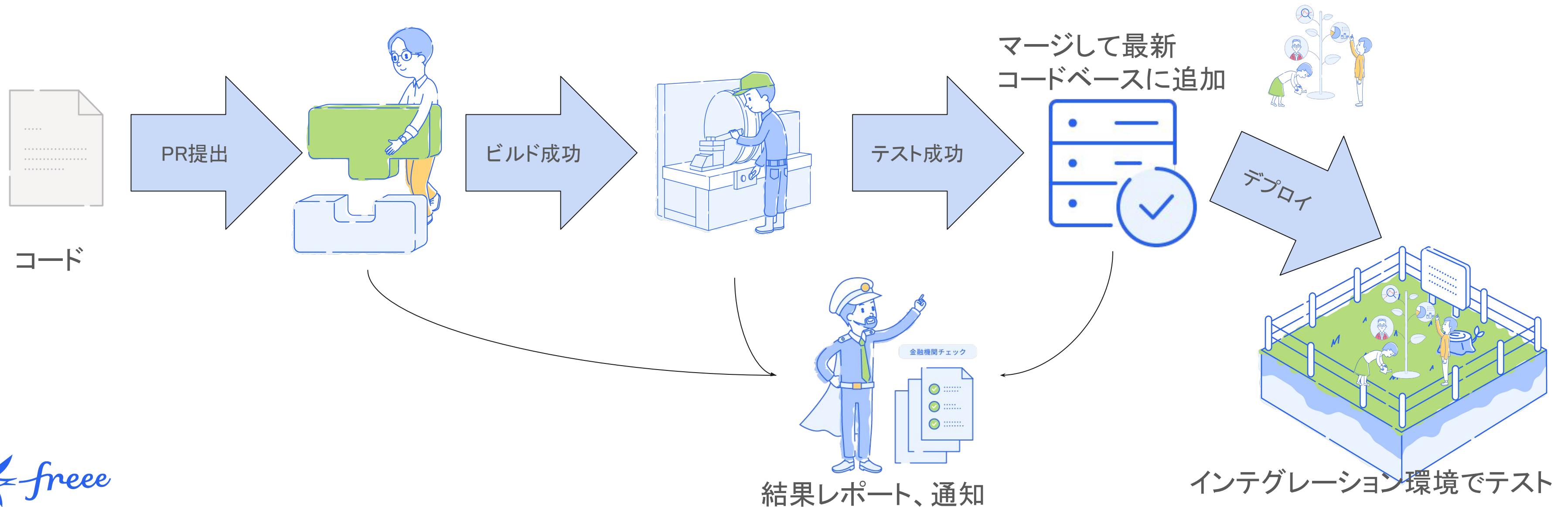
- 要求の仕様化～設計
 - どんな規模のサーバーが必要か？どのくらいの利用者を想定するか？つまり、性能要件を決めて設計する
 - セキュリティ要件(アクセス制御、暗号化など)、を決めて設計する
- Infrastructure as Code (IaC): インフラ構成のコード化
 - ツール例: Terraform, AWS CloudFormation
 - 手動設定を減らし、コードでインフラを管理 → ミス防止・再現性UP
- コンテナ活用 (Docker など)
 - 環境差異を小さくし、どこでも同じようにアプリケーションを動かせる
 - EKSなどを使うと、Kubernetesによるコンテナの自動デプロイと管理ができる
 - 複数コンテナをまとめて管理し、障害時の再起動や負荷分散などを自動化できる
 - CI/CD(継続的インテグレーション&デリバリー)と連携して効率アップ
- モニタリング / ログ管理
 - 本番稼働後もCPU、メモリ、ネットワークを監視
 - ログを収集し、障害や異常を素早く発見



4.4.3 インテグレーション

インテグレーションとは、各自が作ったコードを組み合わせで動かすこと

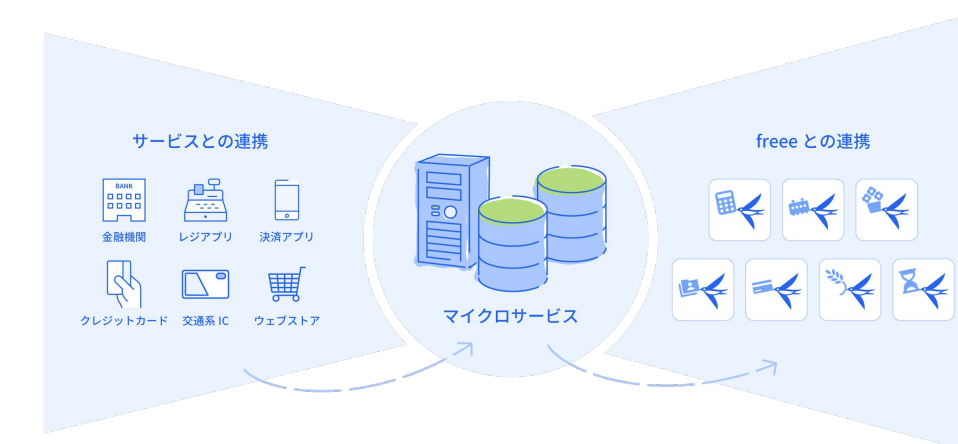
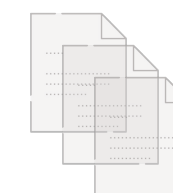
- 複数人・複数チームで開発している場合、定期的に統合すると手戻りや不整合が最小限になり、開発効率が向上する



4.4.3 インテグレーション

インテグレーションの対象

- コード
 - ブランチを分けて別々の人が作業した複数のコードをマージする
 - ブランチ戦略(次ページ参照)を決めてルールを守れる仕組みを導入するとよい
- フロントエンドとバックエンド
 - フロントエンド: ユーザーの画面、ボタン操作、表示部分など
 - バックエンド(サーバー): データ処理やビジネスロジック、データベースなど
 - フロントがバックエンドにAPIでリクエストを送り、結果を受け取って画面表示を更新する
 - 段階的に統合するアプローチがよい
- マイクロサービス連携
 - マイクロサービス: 機能ごとに分かれている小さなサービスやアプリ
 - 例: 商品情報サービス、在庫管理サービス、支払いサービス
 - APIで通信(同期的) / メッセージで連絡(非同期的)
 - どのサービスがどこことやり取りしているかを見える化して、段階的に統合するアプローチがよい



4.4.3 インテグレーション

ブランチ戦略

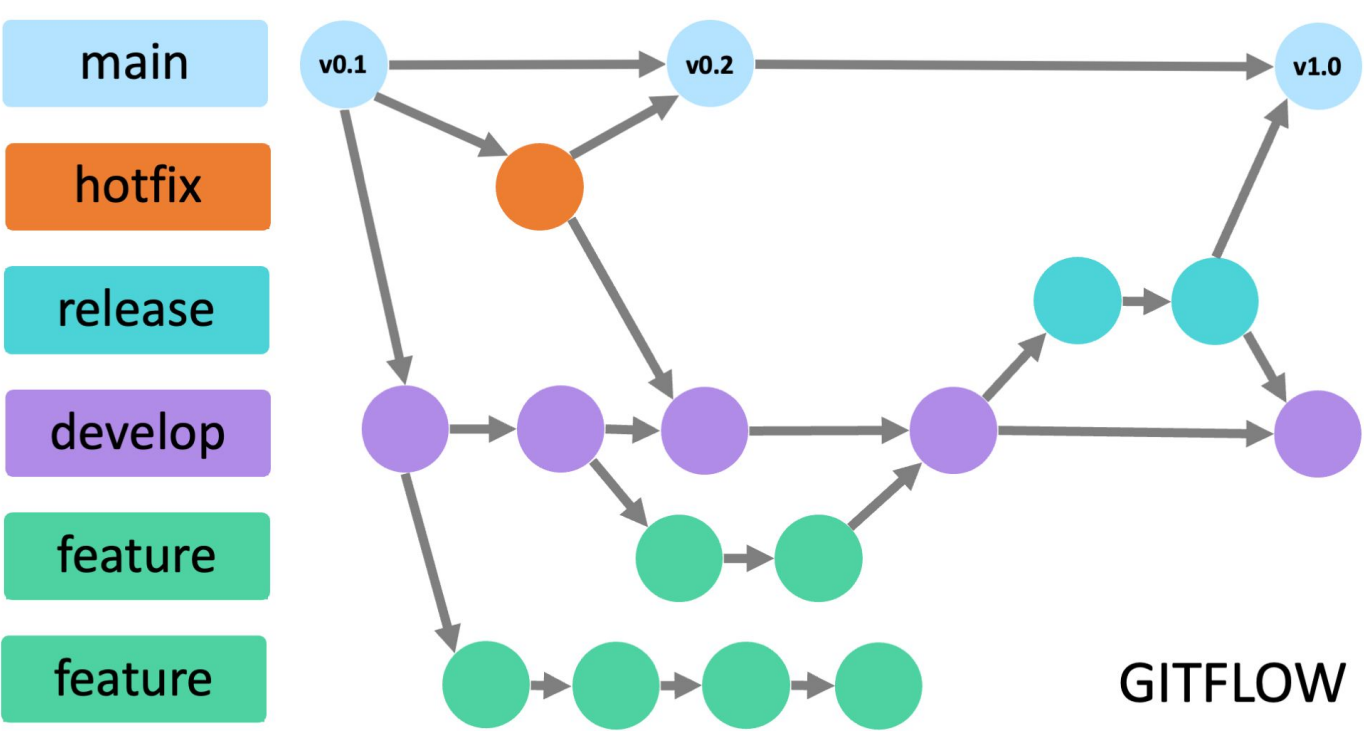
- GitHubなどのバージョン管理システムにおいて、複数の開発者がどのようにブランチを切り、どのようなタイミングでメインブランチへ統合(マージ)していくかを定めたルール
 - 大人数や長期プロジェクトで開発が進む際、闇雲にブランチを作って放置すると、互いのコードが衝突してバグや混乱が生じる可能性が高くなる
 - ブランチ戦略で一定の方針・流れを決めておくことで、効率的かつ安全にチーム開発を進められる

ブランチ戦略モデルの例

Git-Flow

Vincent Driessen 氏が提唱した、有名なブランチモデル

- mainブランチ
 - いつでもリリースできる(あるいは本番運用されている)状態のコードを管理
- developブランチ
 - 次のリリースに向けた最新コードベースを管理
- featureブランチ:
 - 個々の新機能や修正のために、develop から切り出して開発する
- releaseブランチ:
 - リリース直前の調整やバグ修正を行うために作成し、準備が完了したら main にマージ
- hotfixブランチ:
 - 本番運用中の緊急バグ修正用のブランチ。main から作成して修正後、main と develop の両方にマージする



4.4.3 インテグレーション

インテグレーションのポイント

- 継続的インテグレーション(CI)の導入
 - GitHub へのプッシュやPull Request をトリガーに自動ビルド・自動テストを行う(CIパイプライン)
 - Jenkins, GitHub Actions, CircleCI などCIツールを使う
 - CIパイプラインを成功で通過したらブランチ戦略で決めたブランチにマージする
 - 他のコードとのインテグレーションによるエラーが早期に発覚するため、原因の特定が容易になる
- 環境構築と環境間の差異の管理
 - ローカル / 開発 / ステージング / 本番 など、複数の環境を想定
 - インフラ構成をコード化 (Infrastructure as Code: IaC) して一貫性を保つ
 - コンテナ環境 (例: Docker) や仮想環境を活用し、どの環境でも同じ条件でアプリケーションを動かせるようにする
- 複数サービスとの連携・依存関係管理
 - DBや他サービスAPIとの連携がある場合、統合テスト環境で接続先やテスト用ダミーデータを明確化
 - 各モジュール間のバージョン相性やライブラリ依存関係に注意
 - 機能トグル(フリーではフィーチャーフラグと呼ぶ)の活用

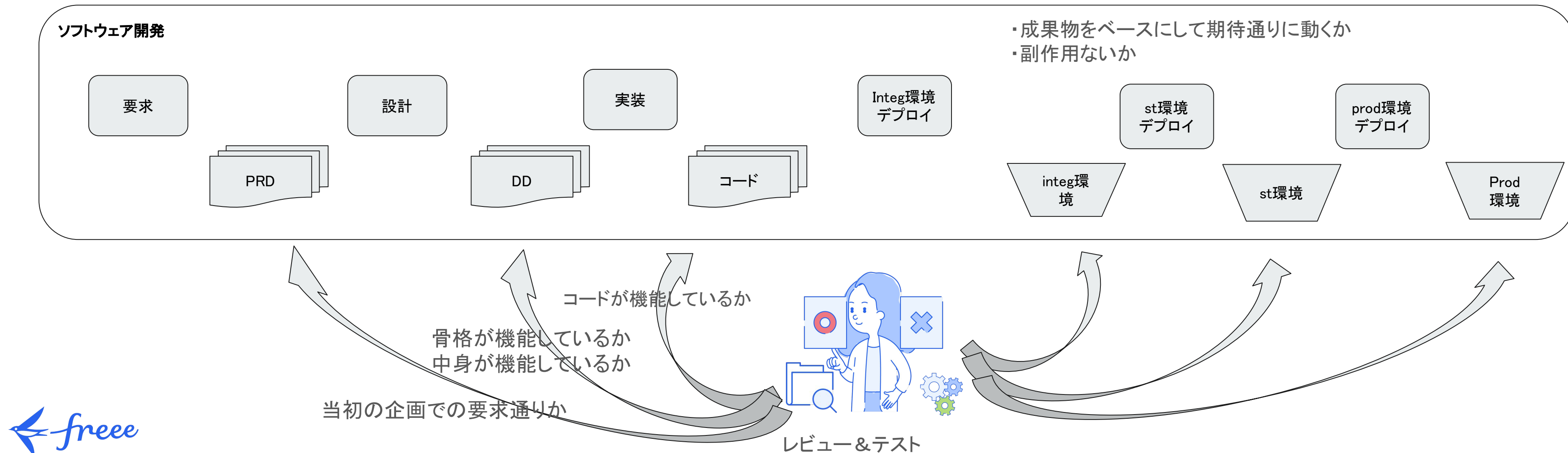


4.4.4 テスティング

対象が仕様や設計で決めた通りに振る舞うかを実際に動かして確認すること

- 「仕様化したシステム要求」「設計」「コード」と段階的に詳細化したそれぞれのものを段階的にテストするのが最も効率が高くなる

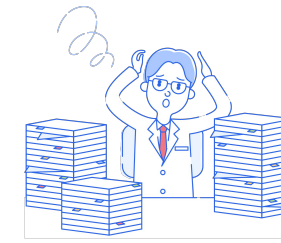
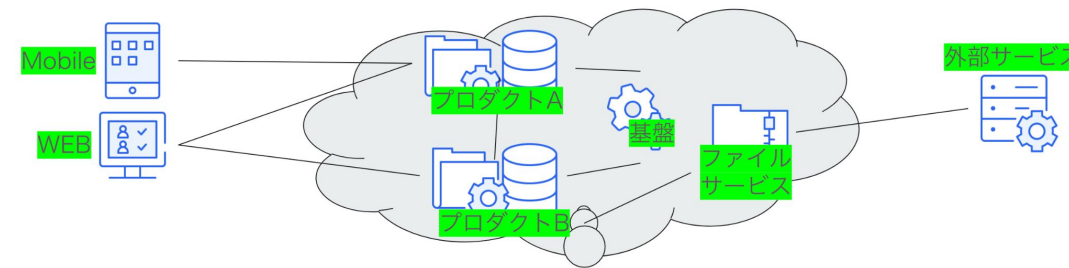
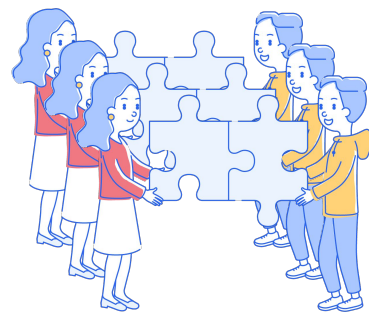
プロセスの中の活動と活動の間には成果物がある



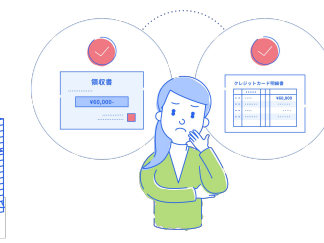
4.4.4 テスティング

テストレベル: 開発の段階に合わせて行うテストは何か？

本番運用と同様の環境へデプロイし、システムテストを行う

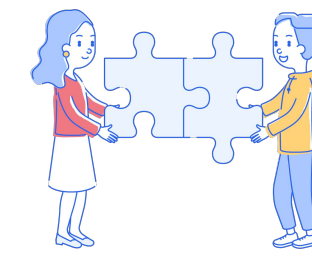
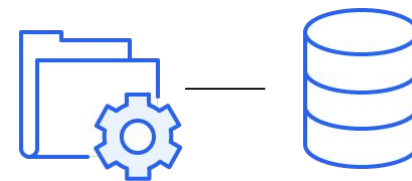


非機能(性能、信頼性など)を確認



ユーザーの利用シーンで確認

ビルドをしたら、インテグレーションテストを行う

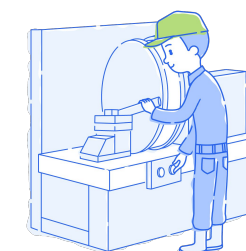
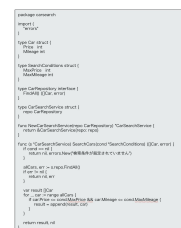


他の人が作ったものとつながる



連携した処理が実現できる

設計をもとにコーディングをしたらユニットテストを行う



エンジニア本人の理解した通りにできる

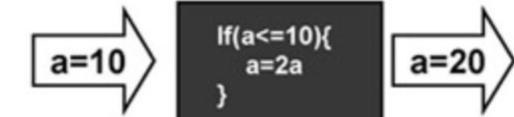
4.4.4 テスティング

テストタイプ:どこに着目してテストをするか？

- 機能テスト
 - 仕様で決められた機能が正しく動くかを確認める
- 非機能テスト
 - 性能・セキュリティ・使いやすさなど、機能以外の品質面を確認める
- ブラックボックステスト
 - コードなど内部構造を見ずに、入力と出力の結果だけで正しさを判断する
- ホワイトボックステスト
 - コードなど内部構造を把握した上で、ロジック通りに動くかを確認める
- リグレッションテスト
 - 機能拡張やデバッグによる変更の副作用で既存機能が不正にならないか再確認する

ブラックボックス

仕様: 入力値が10以下のとき2倍の値を返す



ホワイトボックス

仕様: 入力値が10以下のとき2倍の値を返す

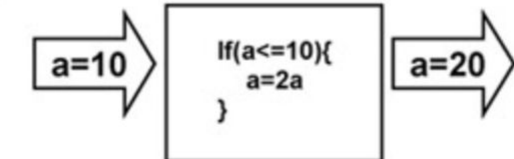
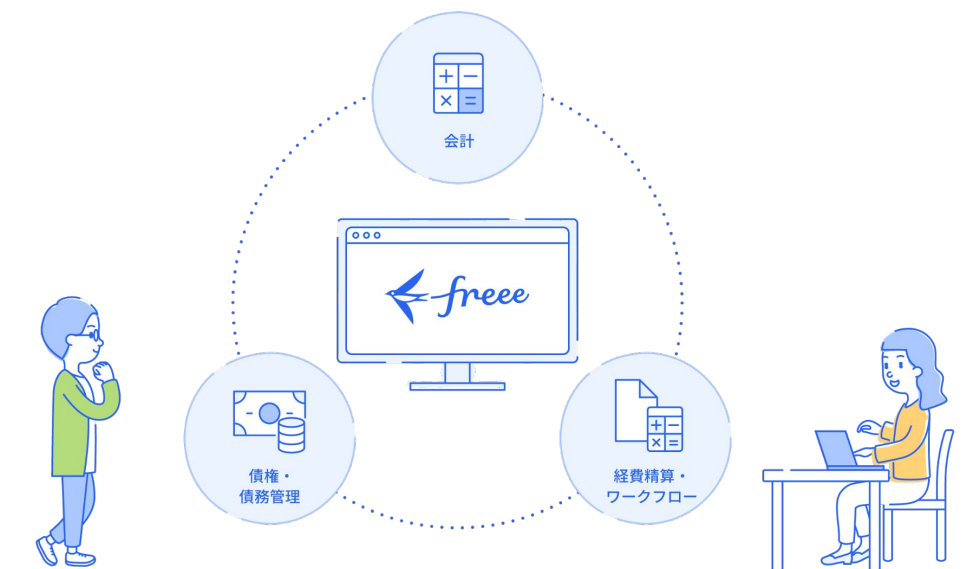


図2●ホワイトボックスとブラックボックスの違い

[テストの実践手法を理解する](#)



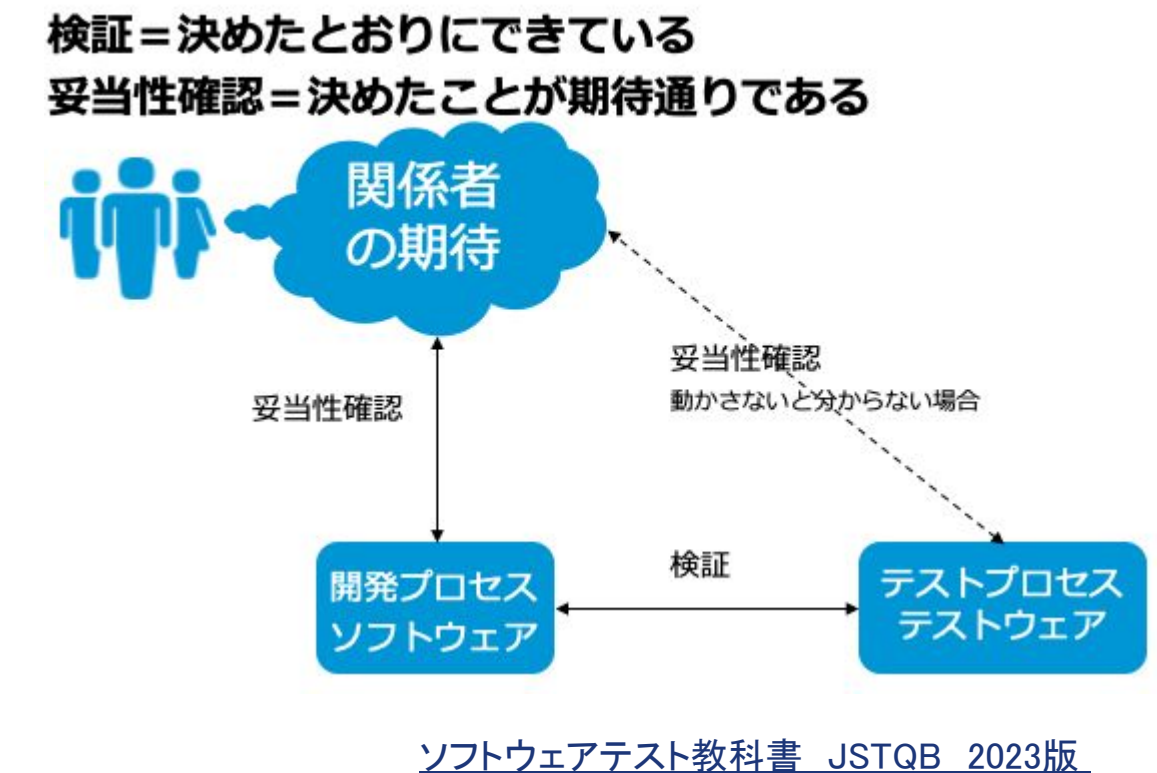
アジャイル開発ではリグレッションテストがプロジェクト成功の鍵 だということがわかった。

バートランド・メイヤー. アジャイルイントロダクション

4.4.4 テスティング

テストをする主な目的

- 開発を進めるにあたっての重要な項目の評価や確認のため
 - 要求に関して: 要求の妥当性確認、要求の獲得の推進
 - 技術課題に関連して: 解決策の妥当性確認と検証
- 仕様の充足性を確認するため
 - 仕様に対する合致性の確認
 - 仕様に対する不具合を見つける(当たり前品質の確保)
- 信頼度の向上及び評価のため
 - 意図的な欠陥識別
 - 例: ストレスをかけて壊れないか確認、脆弱性の確認
- プロダクトリスクの測定
 - 識別したプロダクトリスクに対する対応策の妥当性確認と検証



4.4.4 テスティング

テストの典型的な流れ

- テスト計画
 - テストの日程、範囲、プロダクトリスク、必要な段取りを合意する
 - どのレベルまで深くテストするか、何をもってテストを終わりにするかを合意する
- テスト分析
 - テスト対象の何をテストするか明確にする
- テスト設計
 - 「テストが無限に増えてしまうのをなんとかしたい」という課題を解決する
 - テストケースを書く
- テスト実装
 - テスト環境、テストデータ、テスト実行順書（自動テストならテストコード）を用意する
- テスト実行
 - 対象を動かして確認し、不正な動きや期待通りでない動きを修正をする
- テスト完了
 - テスト結果から、テスト対象が期待通りに動くか、課題はないかをレポートする

テスト実行の工数は、テスト全体全体の30%から40%程度になるのが典型的



4.4.4 テスティング

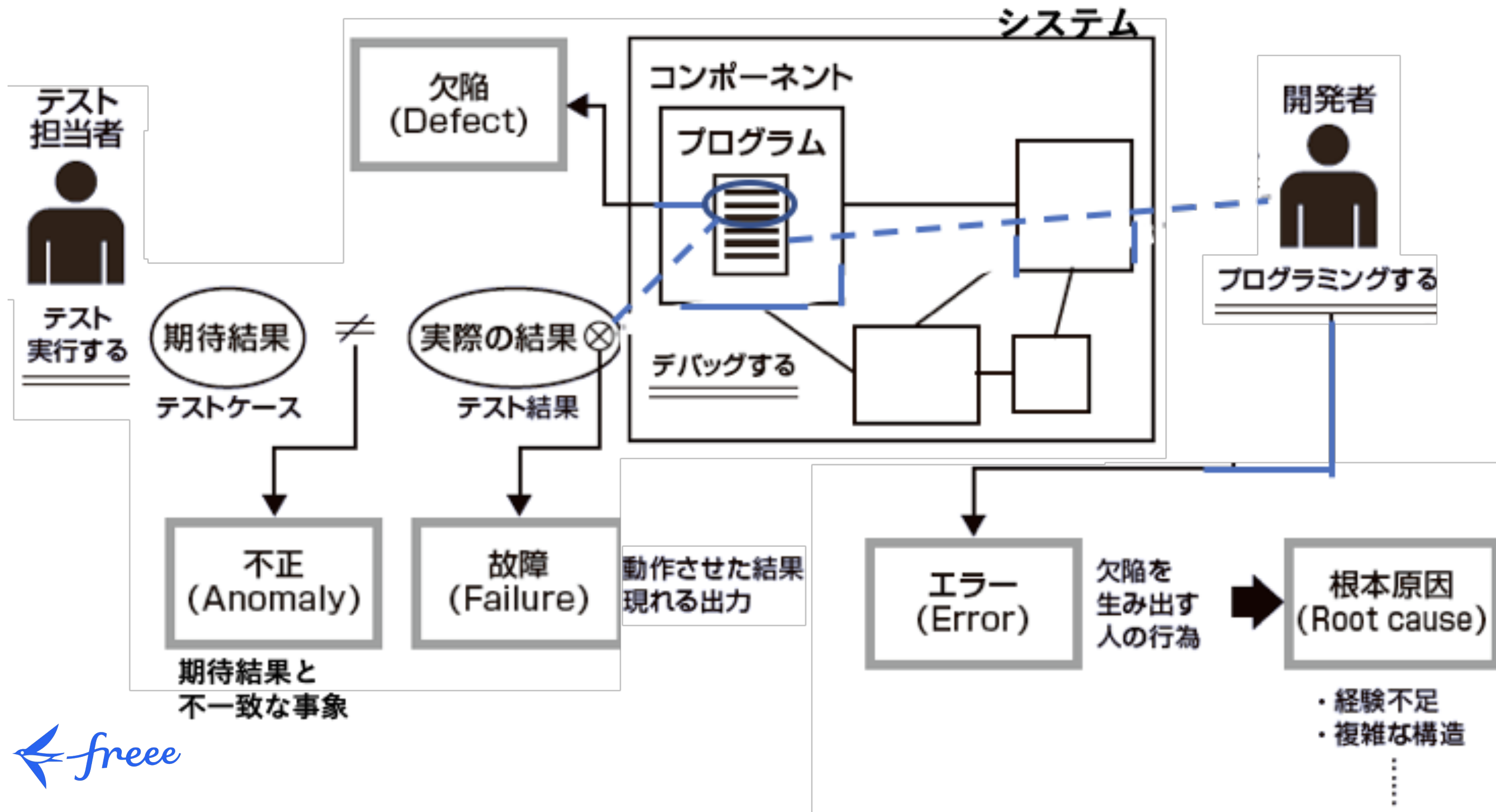
テストケースとは

- テストすべき仕様項目から特定の具体的な条件(ex.入力データ、事前状態)、操作、期待結果を抽出したもの
 - 抽出する具体的な条件は、無駄に多くなく、かつ的を射たものにする
 - テストケースは「ケース(具体例)」なので、それだけだと目的がわからないので「このテストで何を確認したいか？」がわかるようにする→テストにも分析と設計が必要

テストで確認したいこと	事前条件、入力(カバレッジアイテム)	操作	期待結果
条件に合致する販売可能な中古車があるか検索結果にリストされること	事前条件 - 入力条件に合致するデータを50件登録 入力条件 - ボディタイプ:コンパクトカー - メーカー:日産 - 価格:150万円~160万円	- 検索条件を検索フィールドへ入力する - 検索ボタンを押す - 検索結果に表れるページングボタンを押す	事前条件で登録したデータがリスト表示されている
			3ページ分の検索結果が表示される(1ページ20件) 3ページ目は10件表示される
			金額の安い順でソートされている

[参考]不正、故障、欠陥、デバッグの違い

微妙に違うのでどのことを指してるか間違えると誤解になる



不正：期待と一致しない事象
(不具合とも呼ぶことが多い)

- 例：支払い税額が不正に少なくなった
決算書が生成される

故障：誤った出力

- 例：金額不正のバリデーションチェックが働かない

欠陥：成果物上の欠点
(バグと呼ぶことも多い)

- 例：バリデーション処理をするIf文の
判定誤り

デバッグ = テストなどで見つかった不正の原因となる欠陥を探し出し、取り除くこと

4.4.4 テスティング

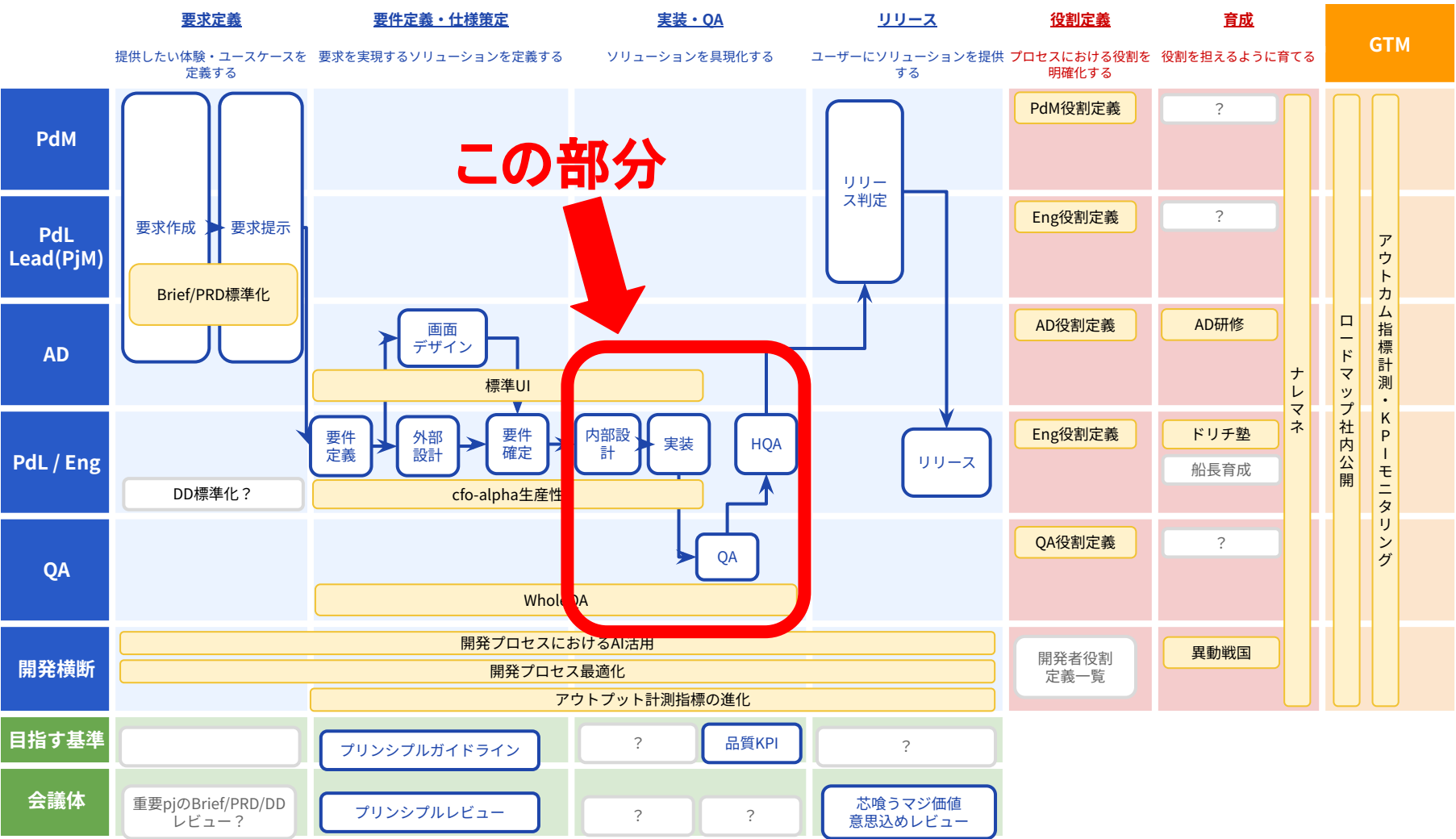
(ちゃんとした)テストの必要性

- ソフトウェアは複雑であり、開発のいたるところに不具合の発生源が潜んでいるため、テストを通して「正しく動く」ことを確認する
 - 全体像が複雑になるため、漏れや誤解が発生する
 - ソフトウェアは論理的なものなので「組み合わせ爆発」が起きやすい
 - $2^{10} = 1,024$ 、 $2^{15} = 32,768$ 、 $2^{20} = 1,048,576$ …
になるので、考えるのが面倒になる
 - 適当に確認するか、諦めるか、低賃金で多くの人を雇い無制限に耐久レースのように働かせるといった、いい加減なことをしてしまうと最悪
- テストをしないor不十分なテストだけで出荷するとどうなるか？
 - 出荷後の市場クレームが起こる可能性を秘めたまま出荷することに…
- 出荷後に市場クレームが発生するとどうなるか？
 - 不具合修正などの作業手戻りの発生で本来の作業が出来なくなる



4.4.5 フリーの開発でやってること(構築)

FY25開發生産性PJから



2024年10月の時点でまとめた開発プロセスから

- フリーの開発プロセスを書いてみた(外部公開なし)
- 主要成果物
 - (新規)freeeBootstrap雛形
 - (新規)CI環境
 - (新規)integ環境
 - (新規)BugSnag設定
 - (新規)datadog設定
 - (新規)st環境
 - (新規)ハッピーフィールド設定
 - (新規)脆弱性診断結果
 - コード、テストコード、PR
 - UIチェック結果
 - システムテスト結果、QAハッピー
 - ProductionReadinessCheck
 - 自動テストスクリプト

フリー社内の構築に属する活動についてのお役立ちリンク(外部公開なし)

- コードレビューガイドライン
- freee engineering handbook

※ハッピー(freee用語)

- 不具合のこと
- 修正するとお客さんがハッピーになるのでfreeeではハッピーと呼ぶ



4.5 デプロイ、リリース後対応

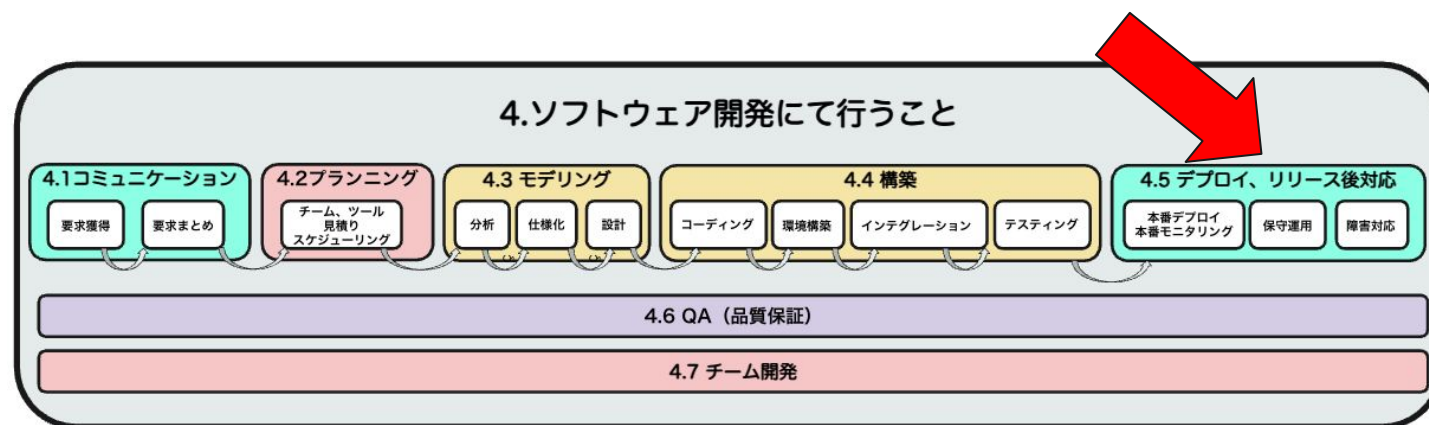
4.5.1 本番デプロイ

4.5.2 本番モニタリング

4.5.3 保守運用

4.5.4 障害対応

4.5.5 フリーの開発でやっていること



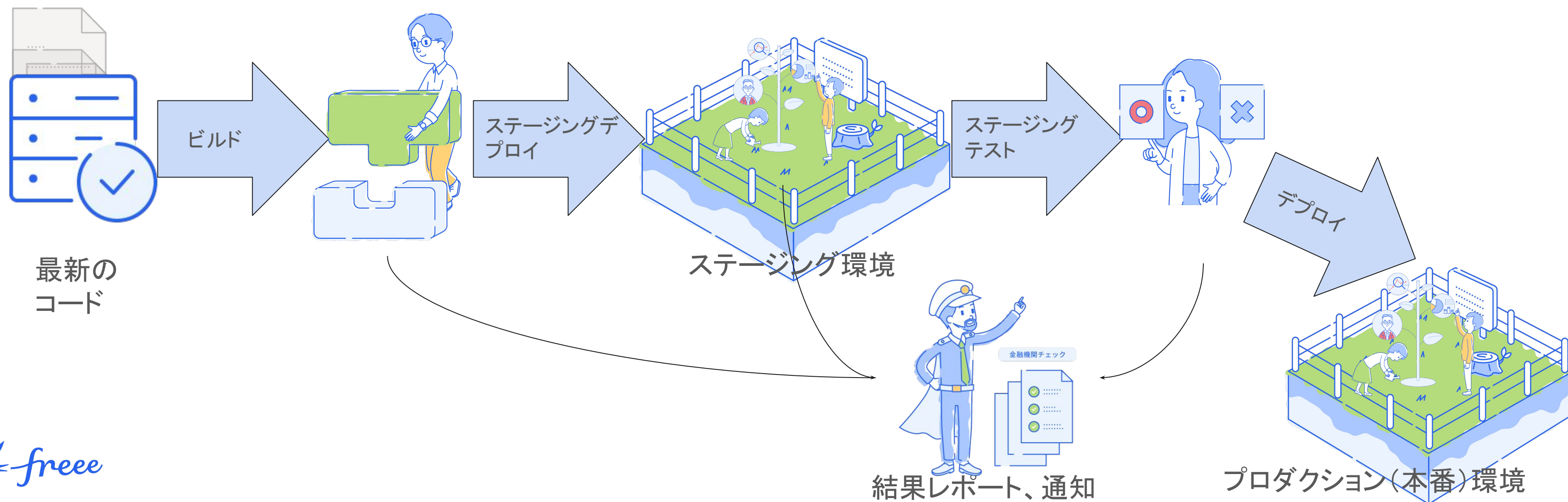
完全なプロダクトもしくは部分的なものだとしても、ソフトウェアは顧客に使われ、成果物が評価され、評価に応じたフィードバックが行われる。

ロジャー・プレスマン; ブルース・マキシム. 実践ソフトウェアエンジニアリング (第9版) (p. 8). (Function). Kindle Edition. を元に記載

4.5.1 本番デプロイ

本番デプロイとは、テスト済みのシステムをユーザーが使う環境に反映すること

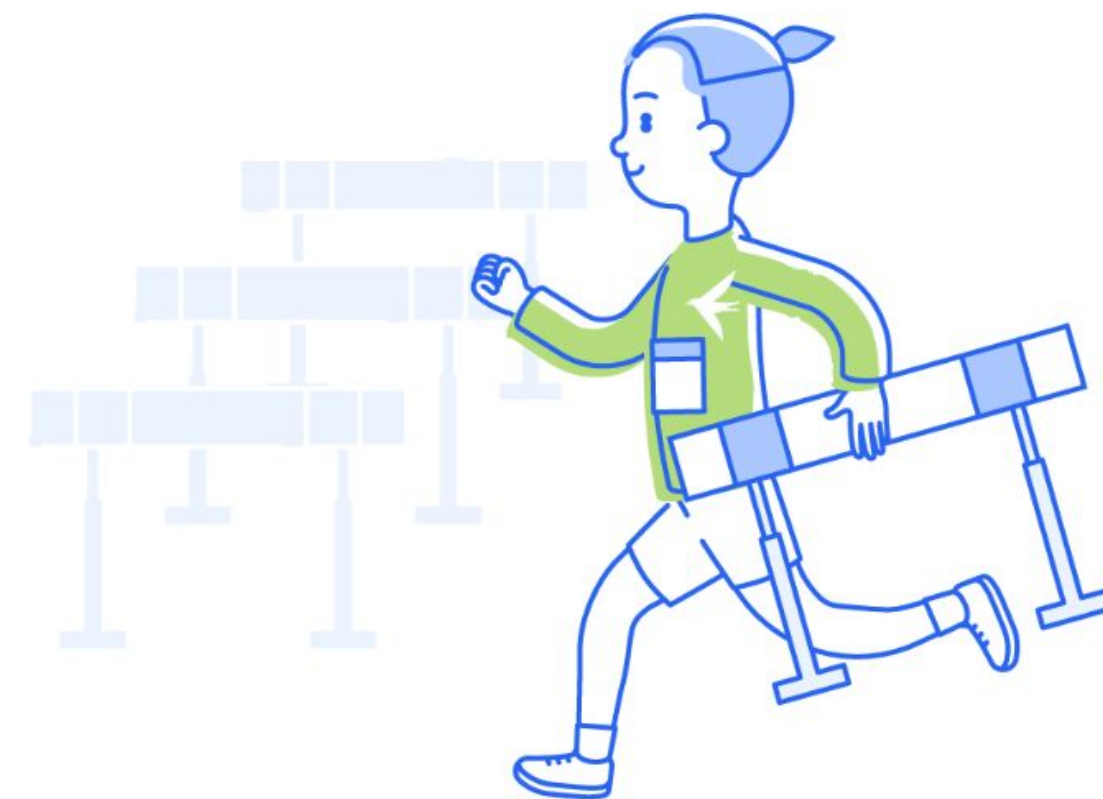
- 本番同様の設定で最終チェックする場所がステージング環境で、そこでOKなら本番に反映
- ユーザーに直接影響が及ぶため、慎重かつ安全に行う



4.5.1 本番デプロイ

本番デプロイの典型的な流れ

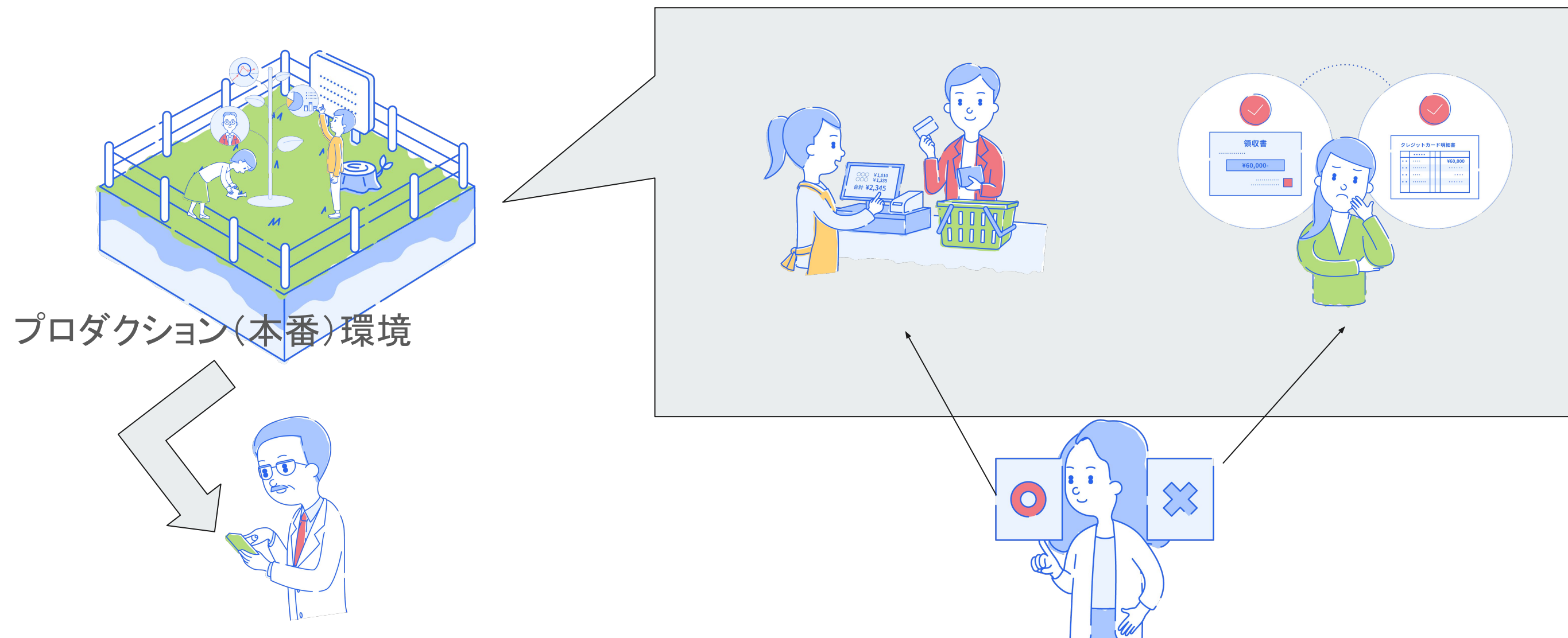
- 最終コードの確認
 - CI/CDパイプライン通過 + ステージングテストOKまで行う
 - ステージング環境ではリグレッションテストを全体的に行う
 - ユーザーに使ってもらいたくない機能が最終コードにある場合は機能トグルで制御されていることを確認
- バージョンタグ付与
 - タグやビルド番号を振って追跡可能にする
- デプロイ実施
 - デプロイツールや手動コマンド(手動の時はより慎重に安全に行う)
- ポストデプロイテスト
 - 本番環境にアクセスして基本動作をチェックするためにテストをする
- リリースノート共有
 - 変更点や注意事項を関係者へ連絡をする



4.5.2 本番モニタリング

本番モニタリングとは、ユーザーが使っている本番環境を常時チェックすること

- 本番環境で問題が起きたら、いち早く検知してユーザーへ与える被害を最小化する
- サービスは本番でユーザーに使ってもらってからが勝負なので、日々確認が必要



4.5.2 本番モニタリング

本番モニタリングの典型的な対象

- **サーバーリソース監視:** CPU、メモリー、ディスク、ネットワーク帯域
 - 例: CPU利用率が80%超えたら警告アラート
- **アプリ監視:** エラー発生件数、API応答速度、HTTPステータス
 - 例: ダッシュボードでリアルタイム可視化
- **ログ監視:** 例外ログ、アクセスログの分析
 - 例: レベルに応じてSlackやメールへ自動アラート

モニタリングをうまく回すコツ

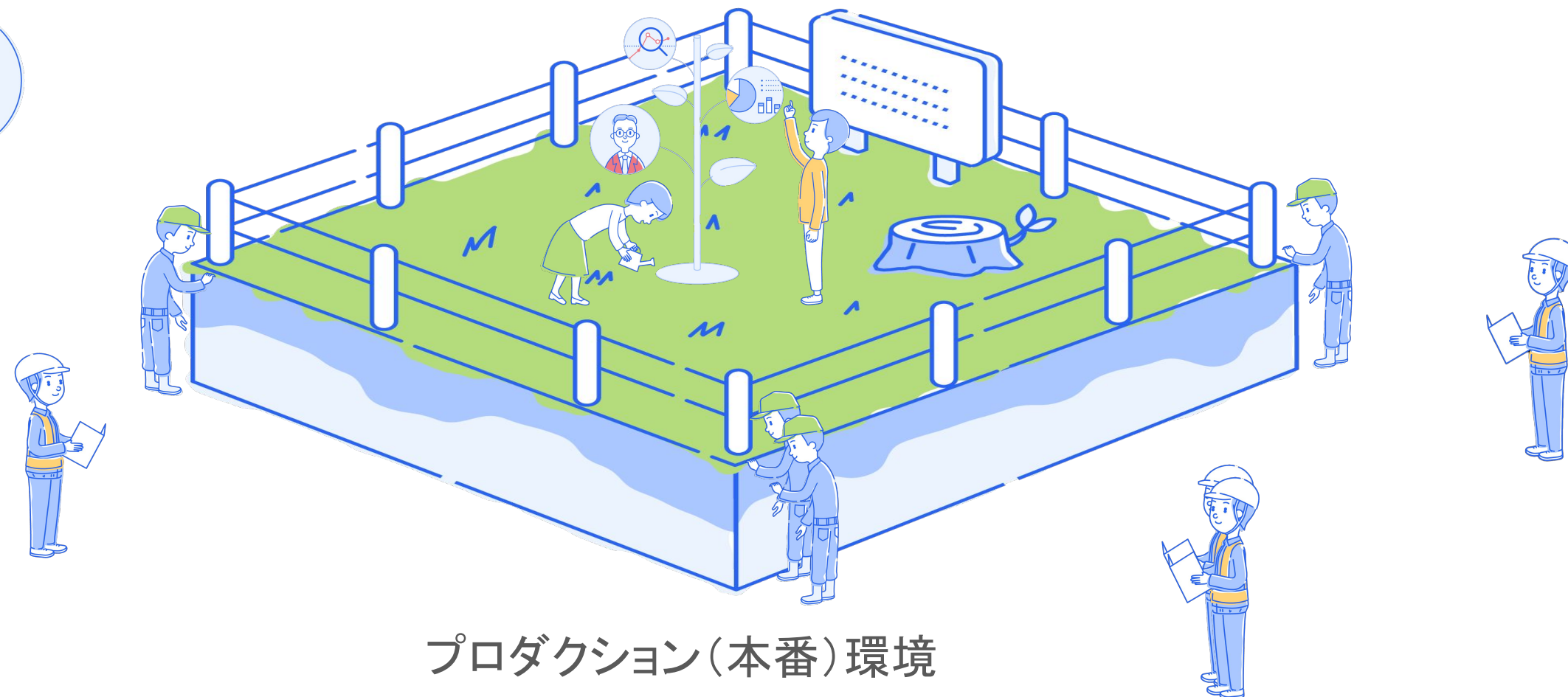
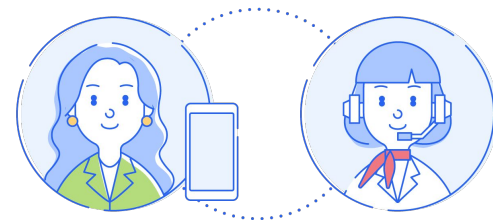
- **しきい値の適切な設定:** アラートが多すぎると見逃しやすい
- **運用プロセス化:** 毎朝の確認／問題発生時の報告フローを決める
- **継続的改善:** 新しい機能追加時に監視項目も増やし、漏れを防ぐ



4.5.3 保守運用

保守運用とは、ユーザーに対して安定したサービスを継続提供すること

- 日々の問い合わせ対応からセキュリティ対策、軽微な不具合の修正、定期メンテナンスまで幅広い作業でサービスを安定稼働させる
- トラブルや性能低下を未然に防ぎ、ユーザーに安心して使ってもらうために保守運用が必要

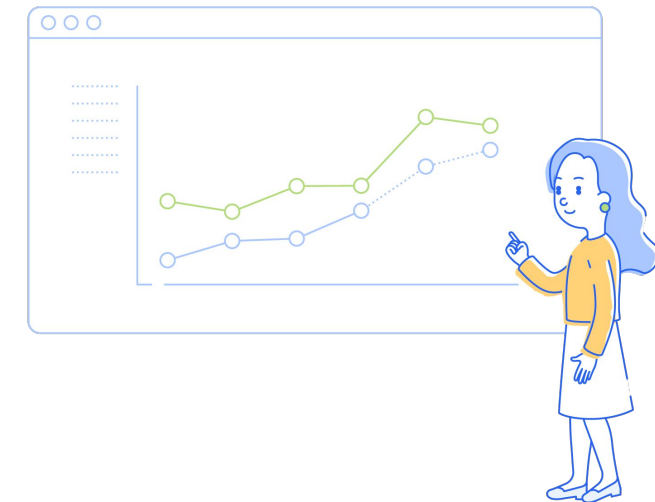
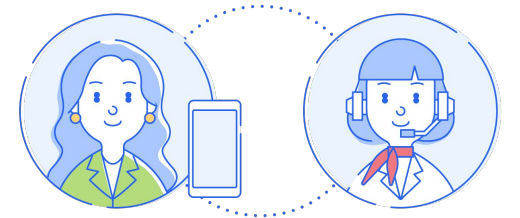


プロダクション(本番)環境

4.5.3 保守運用

保守運用の典型的なやること - その1

- 問い合わせ対応・サポート
 - カスタマーサポートが受け付けたユーザーからの質問・不具合報告の状況把握と初期対応
 - 必要に応じてコード修正 (hot fix もしくは定期メンテナンスで本番へ反映)
 - 重篤度が高い場合は障害対応 (4.5.4 障害対応を参照)
- モニタリング結果の分析
 - どの箇所でエラーが多発しているか、どの機能がよく使われているかといった情報を分析
 - 機能改善やパフォーマンスチューニング、障害予防策に生かす
- バージョンアップ／パッチ適用
 - OSやライブラリ、フレームワークのアップデート
 - 新機能追加やセキュリティ修正を取り込み、脆弱性を減らす
- データベースやストレージの管理
 - 定期的なバックアップ・リストアテスト
 - データ増加に伴うスケーリングやパフォーマンス対策
 - インデックスの見直しや不要データのクリーンアップ



4.5.3 保守運用

保守運用の典型的なやること - その2

- セキュリティ対策／脆弱性対応
 - 定期的な脆弱性スキャン、パッチ適用
 - 不正アクセス検知や権限管理の見直し
- ユーザー管理・テナント(ユーザーのグループを区別する単位)管理
 - ユーザーアカウントの登録・削除、権限設定
 - SaaSの場合はテナントごとの利用状況や請求管理
- 運用手順・ドキュメント整備
 - トラブル対応手順、定期作業の手順をマニュアル化
 - チーム内で情報共有し、担当者が変わっても円滑に運用
- 定期メンテナンス作業
 - メンテナンス日時を告知し、計画通りにアップデートやデータ移行を実施
 - 軽微な不具合の修正も含め、サービス品質を高める
 - 廃棄作業(不要リソースや古いデータの処分)
 - データ保存期間を過ぎたバックアップやログの削除
 - 使わなくなった仮想サーバー・ストレージの解放



4.5.4 障害対応

障害対応とは、システムダウンや不正処理など、重大な影響が出る不具合への対応

- サービス停止やデータ損失などは、迅速に対処しないとユーザーにとって大きな被害に繋がる
 - サービス全停止
 - 不正送金やデータ漏洩
 - 多数のユーザーが利用不能(金額がおかしくなるなど、使い物にならない)



4.5.4 障害対応

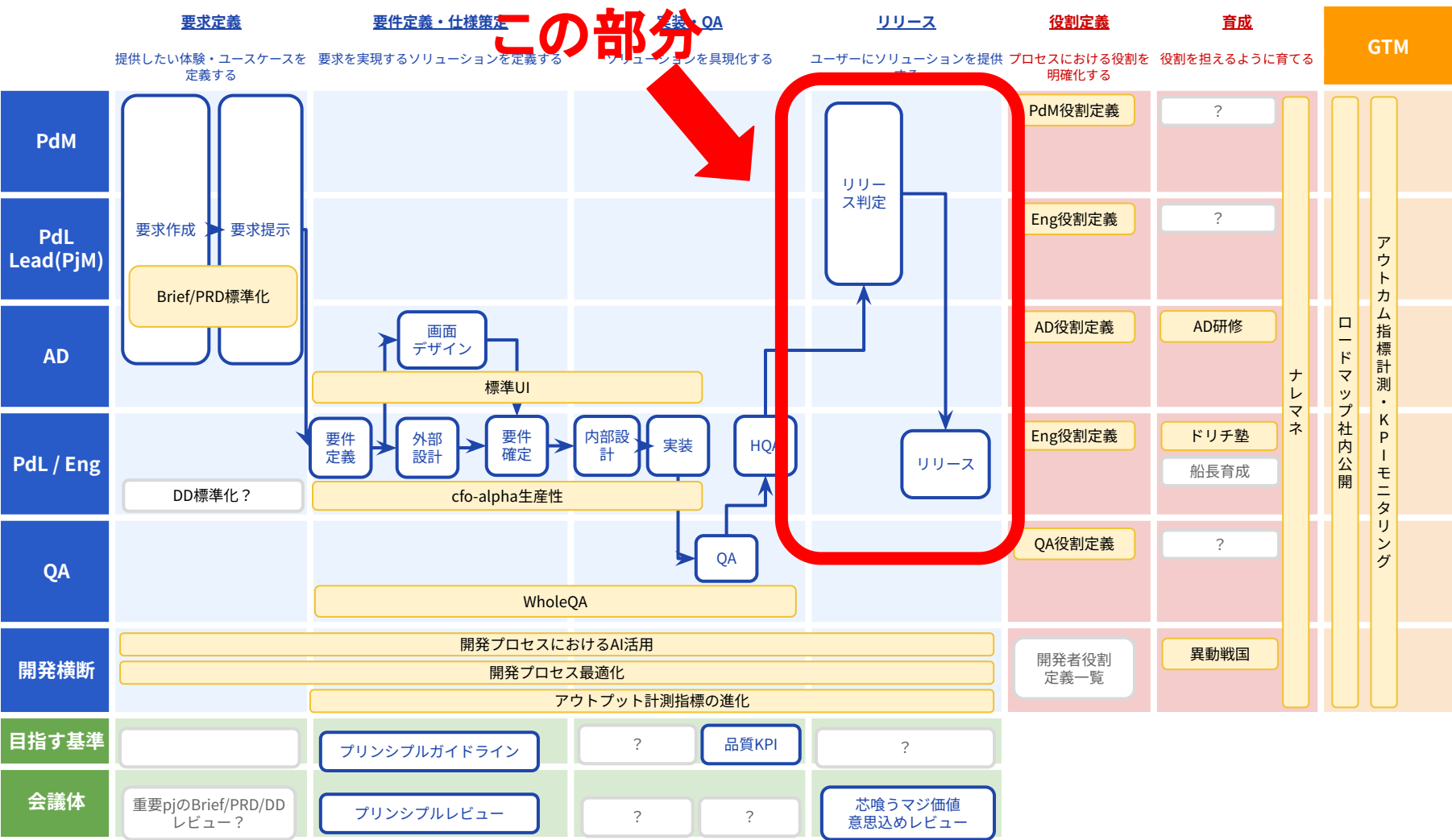
障害対応の流れ

- 検知・連絡
 - モニタリングやユーザーからの問い合わせで異常を発見
 - チームメンバーやカスタマーサポート、管理者へ速やかに連携し、緊急度合いを見極める
- 応急処置(サービス復旧)
 - 被害拡大を防ぐ一時対処やロールバック
 - 可能ならすぐ復旧を試み、サービスが使えない(役に立たない)状態を短縮
 - 修正パッチを実装し、テストをして本番に適用
- 原因分析
 - ログや監視データの確認、再現テスト
 - 欠陥箇所・引き金となった操作・コードなどを特定
- 恒久修正 & 再発防止
 - 根本的な対応を本番に適用
 - 同じ障害を繰り返さないよう、根本原因を除去



4.5.5 フリーの開発でやってること(デプロイ、リリース後対応)

FY25開發生産性PJから



- 2024年10月の時点でまとめた開発プロセスから
- フリーの開発プロセスを書いてみた(外部公開なし)
 - 主要成果物
 - (新規)prod環境
 - (新規)サポート体制
 - ステ確OK
 - リリースInfo
 - データ基盤データ
 - 工数集計結果
 - 振り返り結果
 - ユーザー問い合わせ
 - ハッピー
 - 障害報、障害振り返り、再発防止結果

- フリー社内のリリース後対応に属する活動についての
お役立ちリンク(外部公開なし)
- 障害起票から対応の作業フロー
 - ハッピー運用方針



4.6 品質保証(QA)

4.6.1 欠陥の入る仕組み

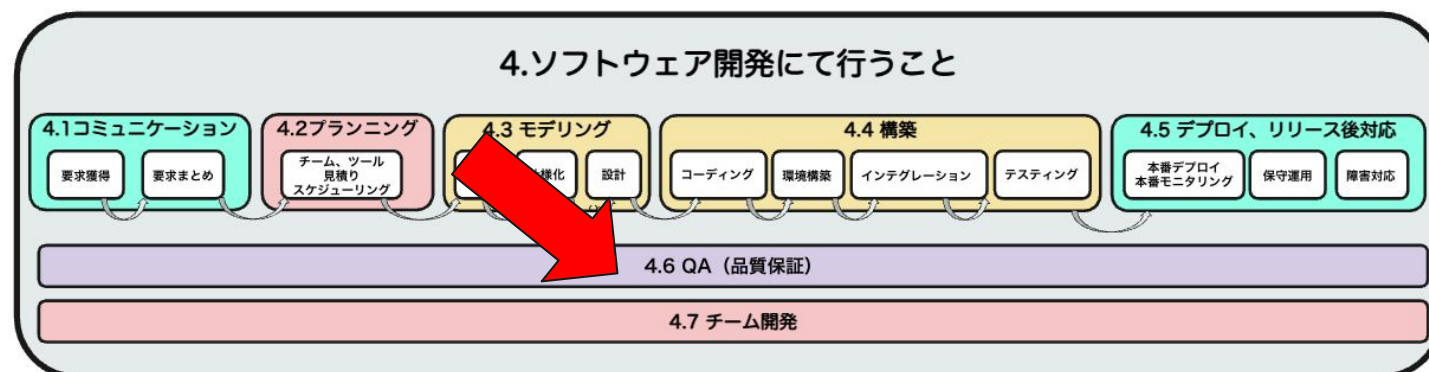
4.6.2 QAとは

4.6.3 重篤度

4.6.4 改善サイクル

4.6.5 フリーでは何をやってるか

4.6.6 フリー内での開発プロセスにおける品質のための活動

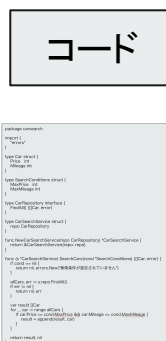
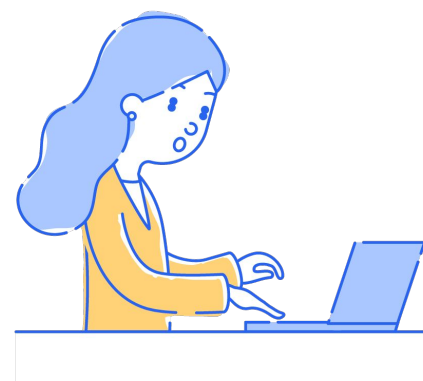


ソフトウェアエンジニアリングプロセスにおける フレームワークアクティビティは、いくつかの包括的なアクティビティにより補完される。一般的には、包括的なアクティビティはソフトウェアプロジェクト全体に適用され、ソフトウェアチームが進捗、品質、変更、リスクを管理するために役立つ。そのひとつである品質保証は、ソフトウェア品質を確保するために必要な活動を定義し遂行する。

ロジャー・プレスマン; ブルース・マキシム. 実践ソフトウェアエンジニアリング (第9版) (p. 9). (Function). Kindle Edition. を元に記載

4.6.1 欠陥の入る仕組み

1人のエンジニアによるコーディング時の間違いだけではない



- うっかり勘違いする
 - ex. 足すところを引いてしまった
 - アルゴリズムの使い方間違える
 - ex. 検索、ソート
 - 使える値を間違える
 - ex. 境界値、null値
 - 条件分岐を間違える
 - 型(整数か? 文字か?)を間違える
-
- 間違った入力があったときのことを考えない
 - ex. 在庫がない時の発注指示
 - 違うデータを使ってしまう
 - ex. 受注データに発注データを使う
 - 同時に複数の処理をする対処を忘れる
 - 大量データの処理をする対処を考えない



自分でちゃんと作ったか確認すれば間違いは見つかる

次の
ページ

4.6.1 欠陥の入る仕組み

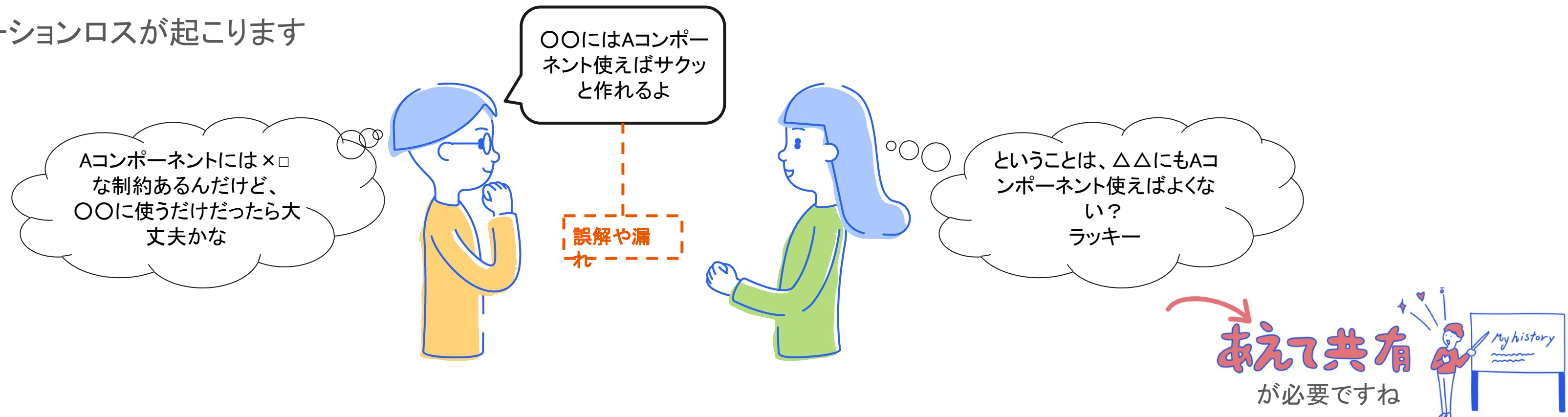
ビジネスの世界では、ソフトウェア開発は1人では行わない

規模の増大(人間の理解できる範囲を超える)、コミュニケーションロス



口頭だけで話を決めて開発してしまう

コミュニケーションロスが起こります



4.6.1 欠陥の入る仕組み

開発の段階的詳細化に対する **考え方が違う**（背景の違い／規模の違い）

- 外部設計、内部設計
 - 論理設計、構造設計
- VS 基本設計、詳細設計

※ 各個人が一度覚えてしまった考え方をひとつに合わせていくのは非常に大変（チームとして具体的なやり方バラバラ）

ソフトウェア開発で行うことは、活動としては不変であるが…

実際はこれらの活動の手順を決めて **プロセスとなるように設計** しないといけない

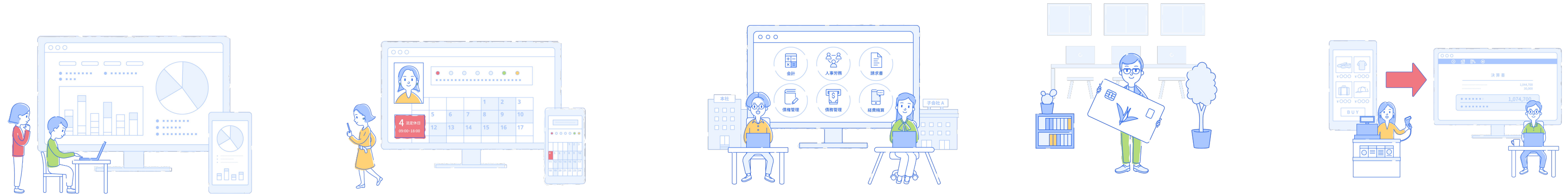
- 進化型/反復型開発（イテレイティブ型開発）
 - アジャイル開発
 - 派生開発（バージョンアップ開発）
- VS ウォーターフォール開発（シーケンシャルな新規開発）

※ それぞれの開発アプローチの意味を理解せず、言われたことだけやると **間違いに気づかない**

そもそも **原理原則として必要な活動がおざなり** のまま開発を行っている

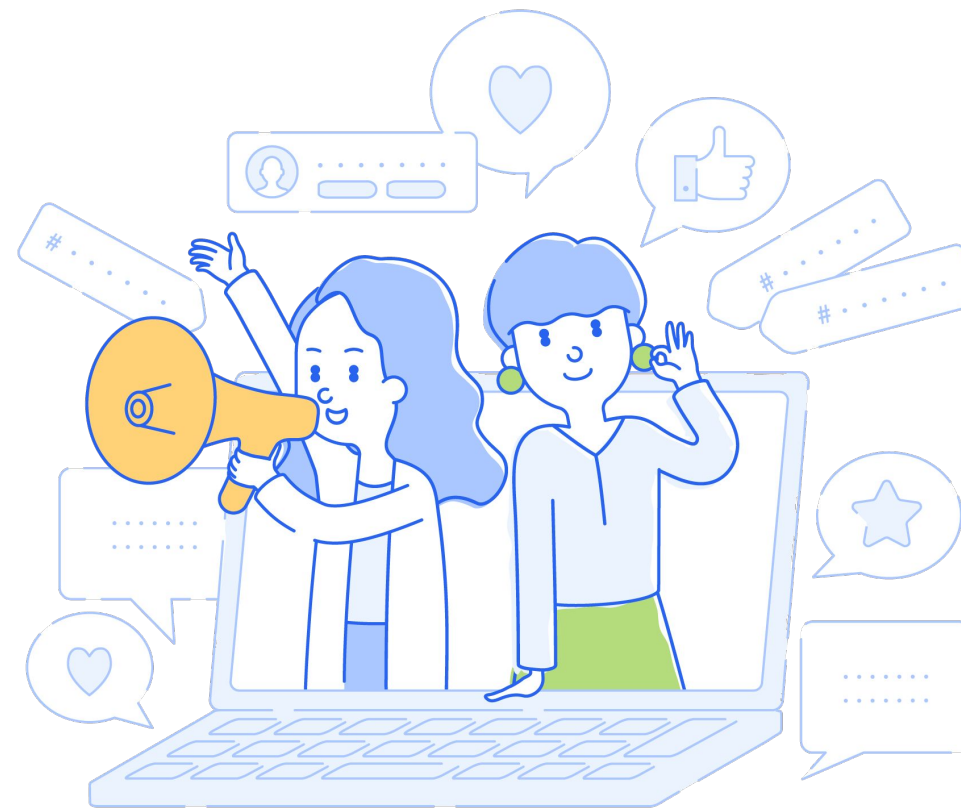
- 要求とりまとめ、要求分析をせずに開発をする（本当に作らなきゃいけないものがよくわからない）
- 設計してコーディングすること、つまり開発には段階があることを知らない（いきなり作る）
- テストが「実行だけ」ではないことを知らない（先人から学ばない）
- アウトプットして確認しあわない（他人と共有しようとしなくて、身近な人にしか伝わらない）
- マネジメント活動を行わない（整理整頓する時間を無駄だと思っている）

4.6.2 QA (Quality Assurance) とは



プロダクトやサービスをユーザーに使ってもらって大丈夫なのかな？って不安を取り除いて、**納得感を共有**する

ための活動全般のこと



4.6.2 QA (Quality Assurance) とは

QAとしてやることを取捨選択する...重篤度(次ページ)で可変させる

- 品質よく作るための活動をプロセスに埋め込む、例えば...
 - ユーザーのもとで起きたらいけないこと(プロダクトリスク)を意識合わせする
 - システム要求の仕様、設計をレビューする
 - コードレビューをする
 - ユニットテスト、インテグレーションテスト、システムテスト、ステージング環境でのテストを行う
 - ツールを使うことで人為的ミスの混入を防ぐ(例:バージョン管理、インテグレーションにツールを使う)
- 品質の可視化、例えば...
 - 重篤度を定義し、発生した不具合を分類をして傾向を把握する
 - コードカバレッジを計測する
 - 欠陥の発生件数、修正済み件数を継続的に計測して傾向を把握する
- 品質向上のために改善をする、例えば...
 - チームの定期的な振り返りで品質の向上に必要なことを見つけてトライする
 - 障害の振り返りを行い、再発防止策を決めて実行する



QAはプロセス全体を通して適用する包括的な活動

4.6.3 重篤度 (Severity)

重篤度とは、ユーザーのもとで発生する事象のひどさ(=ヤバい)の度合いのこと

- 重篤度の高い事柄を常にグリーン(正常)にしておくと、安心できる

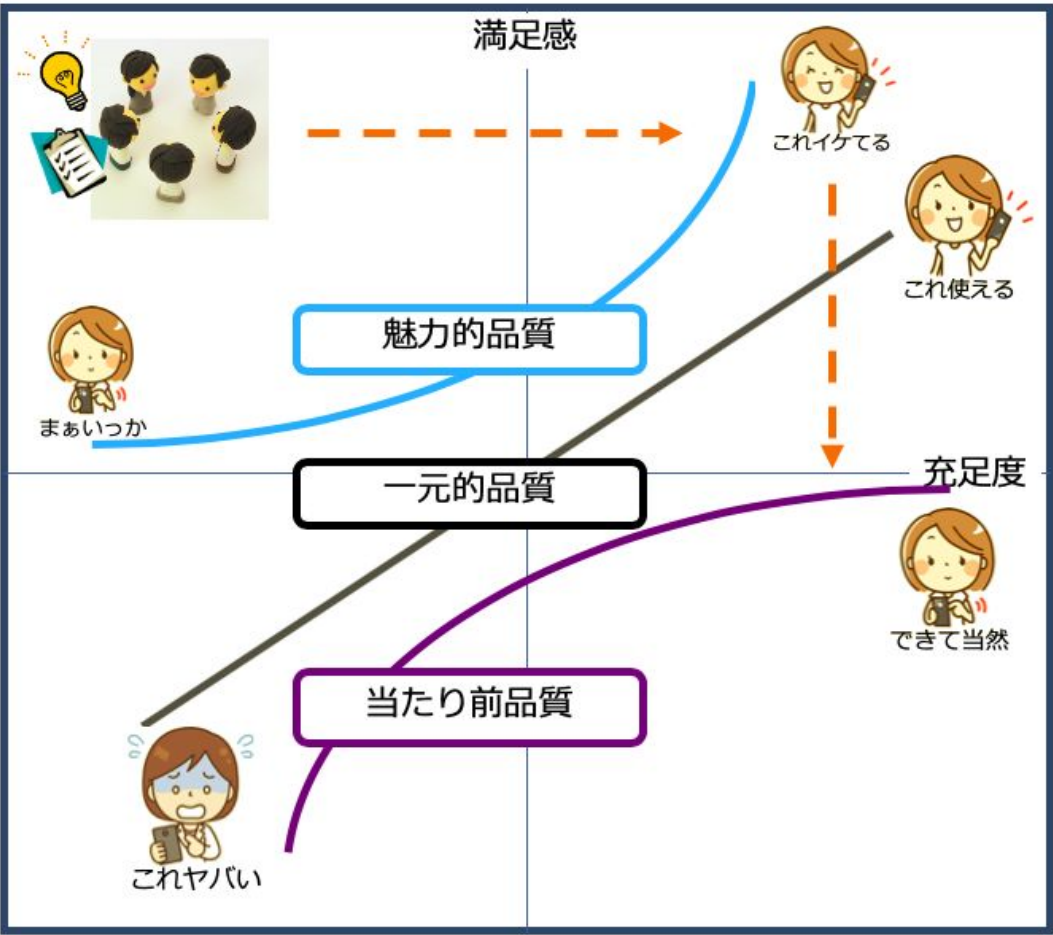
重篤度	重篤度の定義		重篤な不具合の例
Critical 即刻直せ！ Must fix ASAP.	Software is inoperable. 利用不可	・サービスが全面的に使えない ・データ消失して復旧できない ・ビジネスに深刻な影響がある機会損失、金銭損失	・システムがダウンしてて使えない ・誤った金額の決算書で確定申告する
Major 直さなきゃダメ Must fix	Major feature disabled or incorrect. 利用しても役に立たない	・サービスの主要機能が動かない、動作不正のため、サービスがユーザーの役に立たない	・取引の登録ができない
Normal 直すべき Should fix	Software is usable. 利用することはできる	・サービスの機能が動かない、動作不正があるが、サービスを継続利用できる ・対処方法がわかれば、サービスを継続利用できる	・登録前のプレビュー表示が正しくない
Minor 直したほうがいい Nice to fix	It does not affect results. 結果に影響はしない	・サービスの機能が仕様通りではないが、サービスを利用することができる。 ・ハッピーとして起票されたが、仕様だった。	・情報全て見えず横スクロールが必要



4.6.3 重篤度 (Severity)

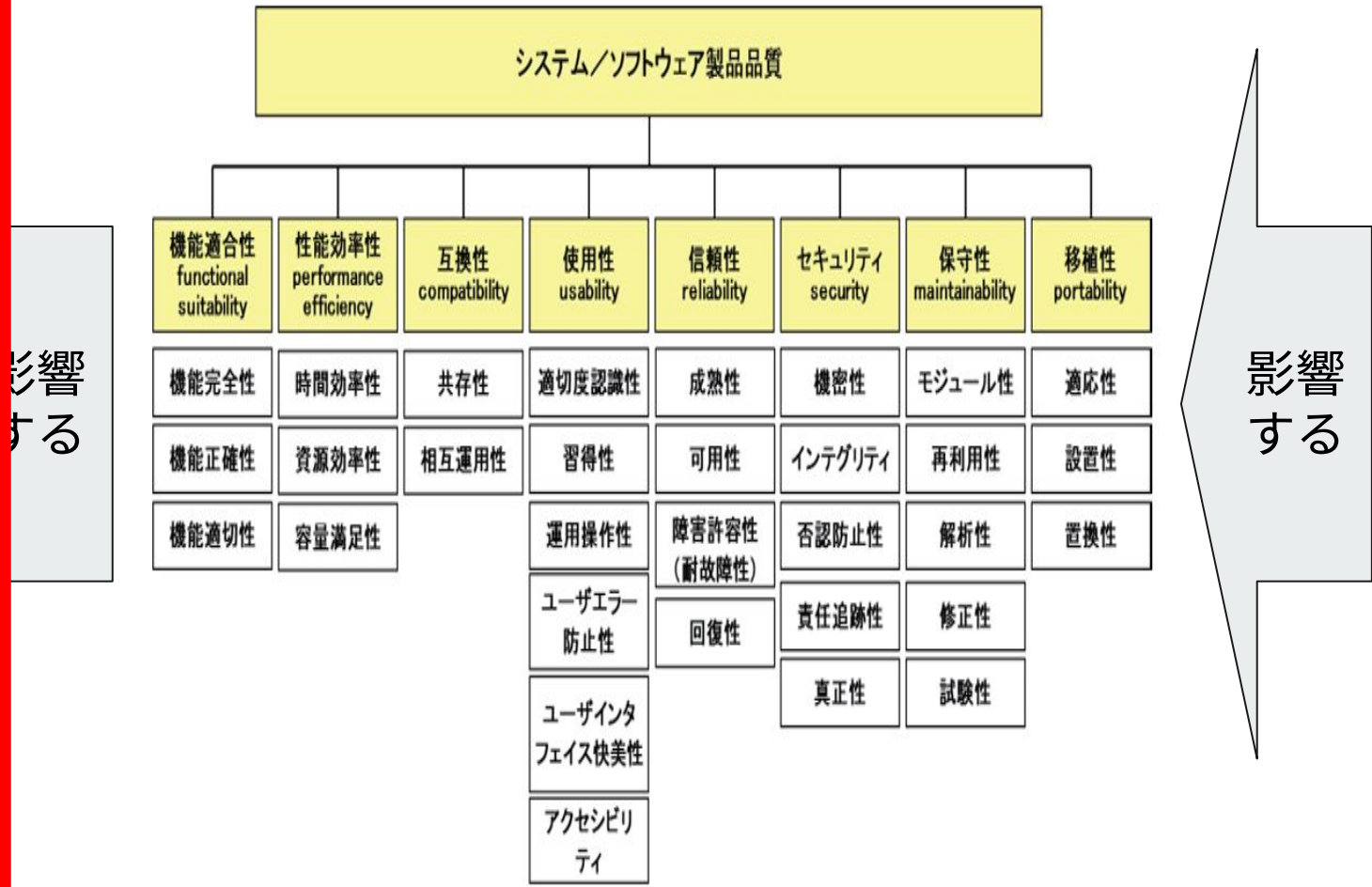
三段階の品質のうち「お客さんにとっての品質」が重篤度を測る対象

お客さんにとっての品質



狩野モデル

プロダクト品質 (品質特性)



開発段階ごと品質

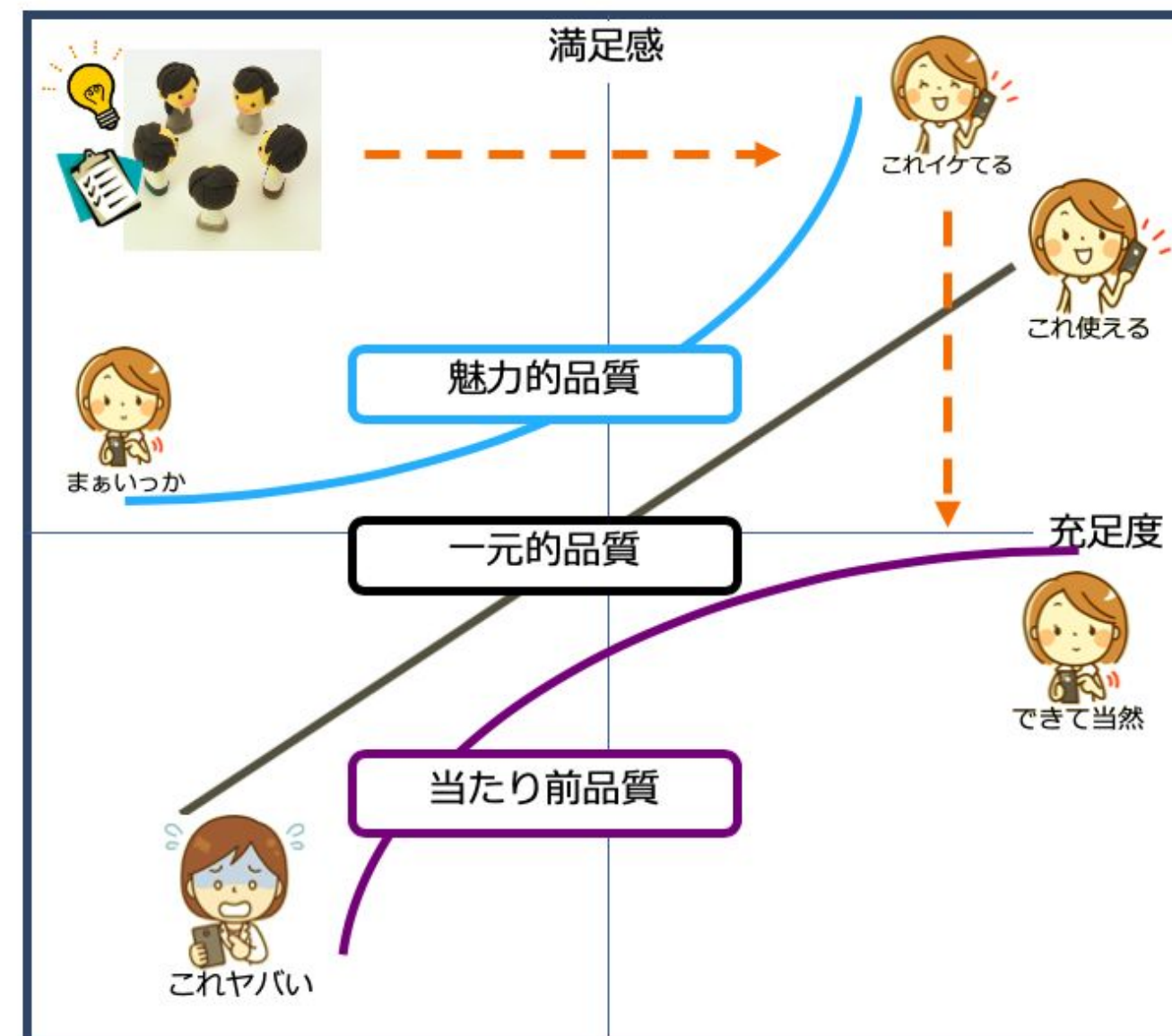
- 要求品質 (Requirement Quality):**
 - ・ サービス間で矛盾
 - ・ 非機能の定義してない
- 設計品質 (Design Quality):**
 - ・ 相互依存、循環依存してる
 - ・ 責務がばらつく
- コード品質 (Code Quality):**
 - ・ if文が多い (複雑度)
 - ・ 同じ処理のコピペ多い

ISO25010:2011 プロダクト品質モデル (最新は2023年度版だけどこの図は2011年版)

4.6.3 重篤度(Severity)

お客さんにとっての品質の全体像: 魅力、一元的、当たり前

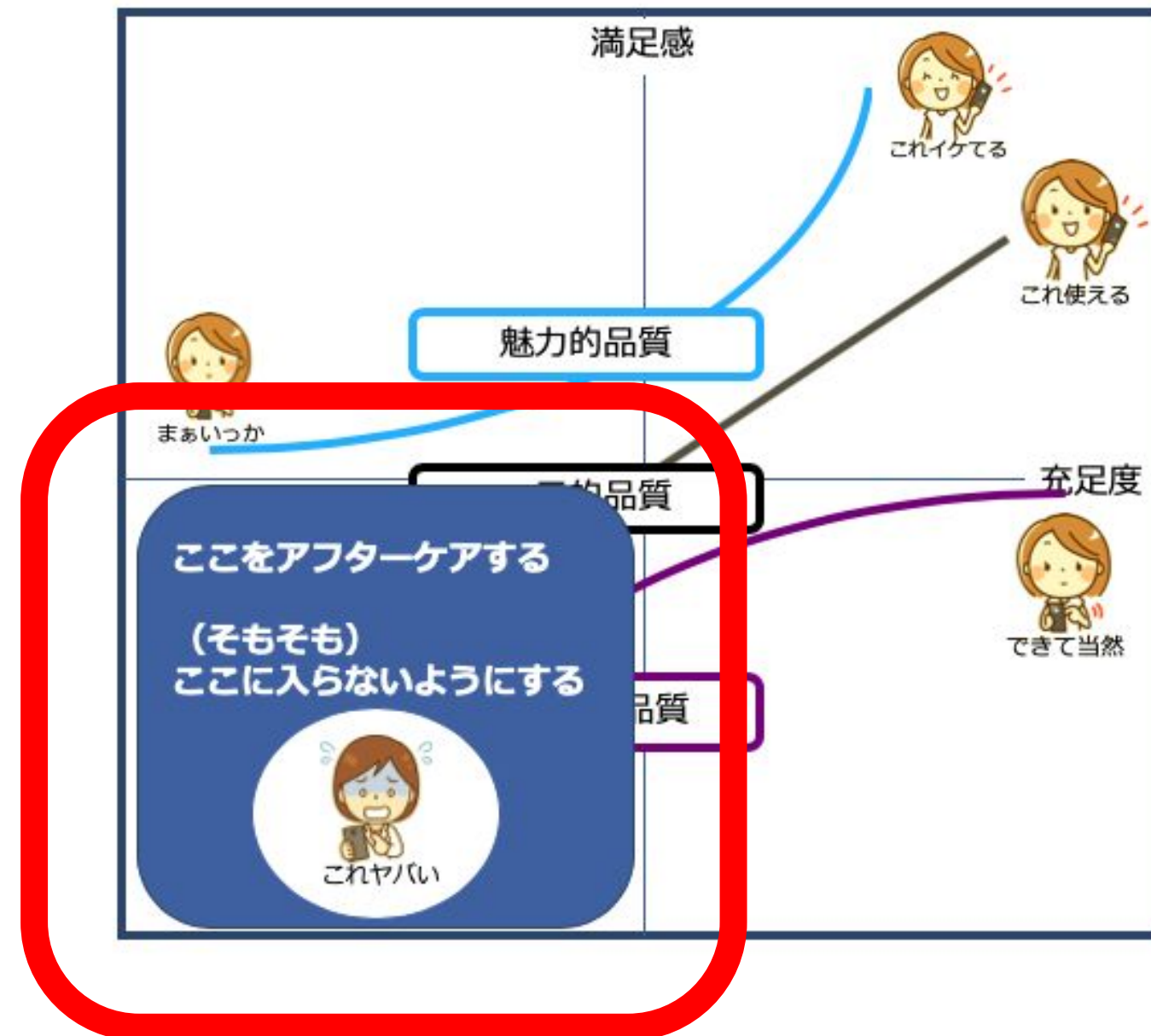
- 新サービス、コンセプトは魅力的(AIエージェント)、一元的品質を向上させる(取引検索性能)
- 魅力的、一元的品質は、時間とともに当たり前品質になっていく(銀行口座同期)



4.6.3 重篤度(Severity)

「やばいゾーン」を重篤度で表現

- 一元的、当たり前品質がヤバいゾーンに入ったら十分な対処が必要になる



4.6.3 重篤度(Severity)

病気の人がたくさんいたら誰から治療する？

- 病気を例に重篤度＝病気、優先度＝治療のスピードと考えるとイメージしやすい

病名	重篤度
癌	Critical
脳梗塞	Critical
糖尿病	Major
うつ病	Major
骨折	Normal
風邪	Normal
軽度のアレルギー	Minor
擦り傷	Minor

すぐ治療できるから先に＝優先度高くなる場合がある（可変する）

4.6.3 重篤度(Severity)

病気がたくさんいたら誰から治療する？

- 病気を例に重篤度＝病気、優先度＝治療のスピードと考えるとイメージしやすい

そもそもこうなら
ないようにする
(もし起きたら、
まずは応急措置)



こうなったら
治せばよい

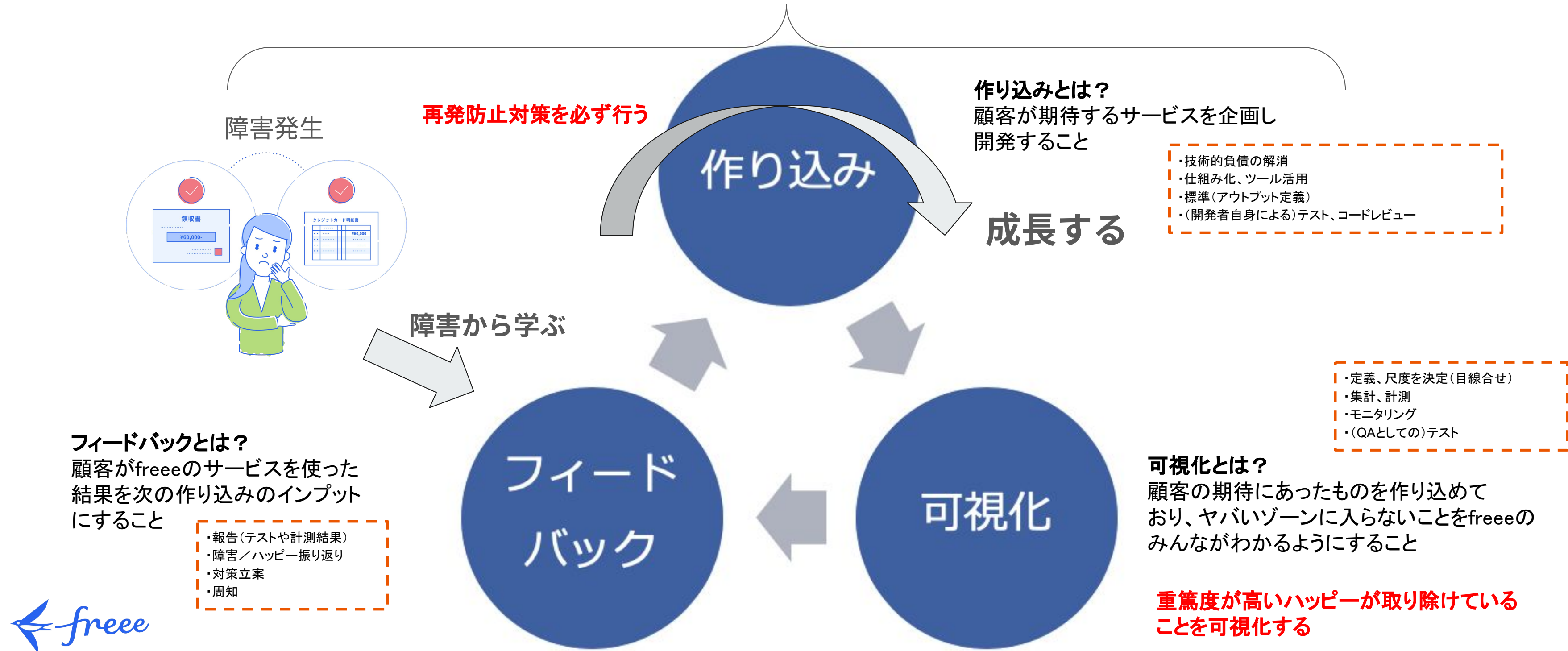


病名	重篤度
癌	Critical
脳梗塞	Critical
糖尿病	Major
うつ病	Major
骨折	Normal
風邪	Normal
軽度のアレルギー	Minor
擦り傷	Minor

4.6.4 改善サイクル

改善サイクルによって、品質が良いサービスができるように成長する

このサイクルで品質をよくしていくカルチャーの定着



4.6.5 フリーの開発でやってること (QA)

FY25開發生産性PJから



- 2024年10月の時点でまとめた開発プロセスから
- フリーの開発プロセスを書いてみた(外部公開なし)
 - 主要成果物
 - 重篤度の定義
 - リスク洗い出し
 - 品質KPIの計測結果
 - 品質目標KGI、KPIの計測結果
 - システムテストをする際のプロセスでの成果物 (QAテストプロセス)

- フリー社内のQAに属する活動についてのお役立ちリンク(外部公開なし)
- QAプロセス
 - 品質目標KPI



4.6.5 フリー内での開発プロセスにおける品質のための活動

品質チェックポイントリスト

フェーズ	コミュニケーション					モデリング			構築							デプロイ、リリース後対応									
活動名	リーガル相談	セキュリティ相談	取り扱う個人情報のリストアップ	PRD共有会	セキュリティレビュー	DD愛でる会	Security Design Review	リスク洗い出し会	テストプラン共有会	コードレビュー	ユニットテスト	脆弱性診断	テスト分析共有会	システムテスト	ぽちぽち会	ステ確	本番作業（ベアオペ）	リリース後ログ監視	振り返り	障害振り返り	再発防止策実施	SLI/SLO	SecuritySensor	内部監査（SOC1）	内部監査（J-SOX）
活動概要	企画内容に対するリーガル相談に応じる	企画内容に対するセキュリティ相談に応じる	・個人情報管理台帳の更新 ・扱う個人情報のデータフローの整理	PRDの内容を作成者からメンバーに説明する	AI開発プロセス、要求内容に対するセキュリティレビューを行う	DDをメンバーに事前共有し、ミーティング形式でレビューをする	詳細設計に対するセキュリティ観点のレビューを行う	開発の初期に関係者全員の知見をつかってリスクの識別と対処を決めて欠陥が入らないように開発を進める	テストのスコープや注力箇所をチームに共有する	PRでがされたコードをマージしても良いかを確認する	開発者が自分で書いたコードをテストする	リリース前に検証環境での脆弱性診断を実施する	テストする内容をチームに共有する	ユーザーが利用するのと同様の環境で問題ないことをテスト	リリース前に関係者全員でサービスを使ってみて、ユーザー利用が問題ないかを確認する	本番にデプロイするのと同様のコードをデプロイして問題ないことを確認する	作業手順書を書いて、一緒に作業する人を任命して操作をする。	開発の中で良かったこと、もったところをチームで共有し、今後の改善ポイントを明らかにする	関係者が集まって話し合い、原因や対策とあったことを導き出す	障害振り返りで明らかになった根本対策を実施する		実装したSLOを活用して、早期にプロダクト信頼性の低下を検知する	SecuritySensorで検出したアラートをトリアージ	・『すべての開発・変更には理由・根拠がある』状態で開発を行うこと ・監査法人への上記の全エビデンスを提供	1. プログラムの開発・変更・保守に関する内部統制 2. プログラムの運用・障害対応・アクセス権管理に関する内部統制
ロール	法務リスク管理船	PSIRT CSIRT	PdM	PdM,eng, QA	PSIRT	eng,QA	SecurityChampions PSIRT	QA,eng,PdM	QA,eng,PdM	eng	eng	PSIRT	QA,eng,PdM	QA	eng,QA,PdM	eng,QA	eng	eng	PdM,eng,QA	eng,QA	eng	PdM,eng	PSIRT	CSIRT	法務リスク管理船



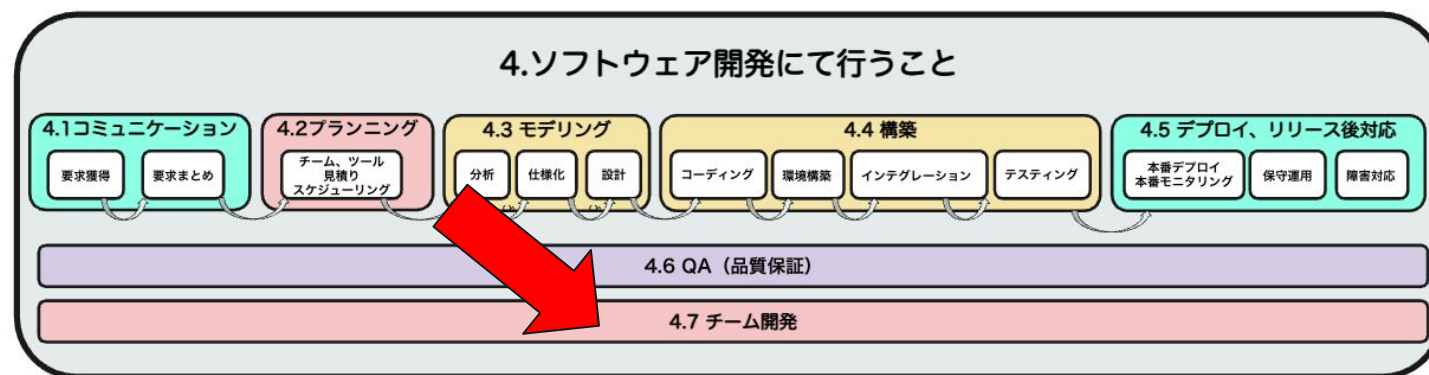
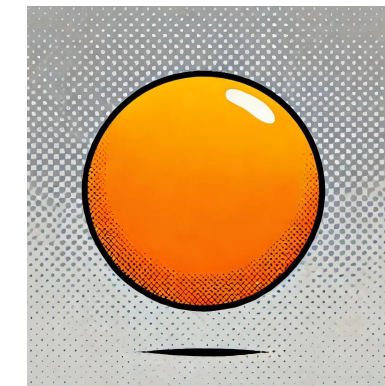
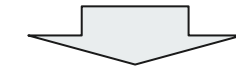
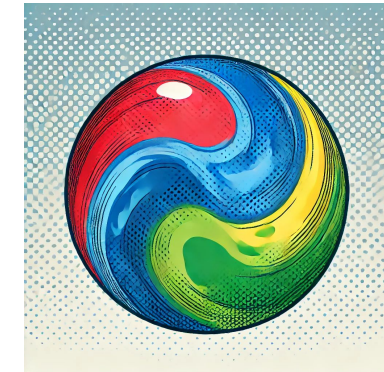
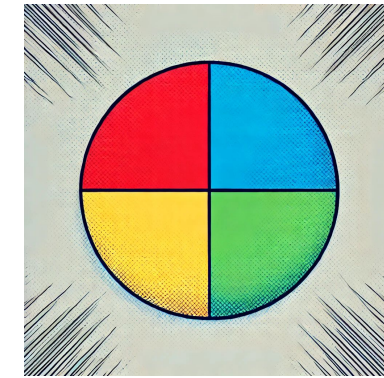
4.7 チーム開発

4.7.1 ワーキンググループ vs. チーム:何が違うのか？

4.7.2 効果的なチームの特徴

4.7.3 リーダーシップ

4.7.4 チームが成長する4つのステージ



ゼリー状になったチームは 平均よりも明らかに生産性が高く、成功に向けた意欲も高い。部分的な最適でなく全体の最適に向けて非常によくまとまったグループになる。〈中略〉いったんチームがゼリー状になり始めると成功の可能性が飛躍的に向上する。チームは成功へ向けて止まらなくなる。〈中略〉チームを従来の手法で管理する必要はなく、動機づけも不要となる。弾みがつくのだ。

ロジャー・プレスマン; ブルース・マキシム. 実践ソフトウェアエンジニアリング (第9版) (p. 57). (Function). Kindle Edition.

4.7.1 ワーキンググループ vs. チーム:何が違うのか？

チームは、共通のゴールを持ち成果物は共同で作り上げる

- ワーキンググループとチームの違いを理解することが大事

	ワーキンググループ (管理者主導的)	チーム (自律的)
定義	独立したゴールが設定され、必要な情報共有やリソースのやり取りをしっかりと行う	ゴールを共有し、役割を補完し合いながら相互依存的に進める
リーダーシップ	リーダーが単独で意思決定し、メンバーはそれぞれのゴールに基づく成果物を出す	リーダーシップを共有し、成果物はチームとしてまとめて責任を負う
プロセス	ディスカッション→決定→各自に移譲	ディスカッション→決定→チームで実行
権限	作業を実行する権限のみ	どのように仕事を進めるか決める権限があり、何を作るかに関しても裁量度が高い
説明責任	個人のゴール	チームのゴール



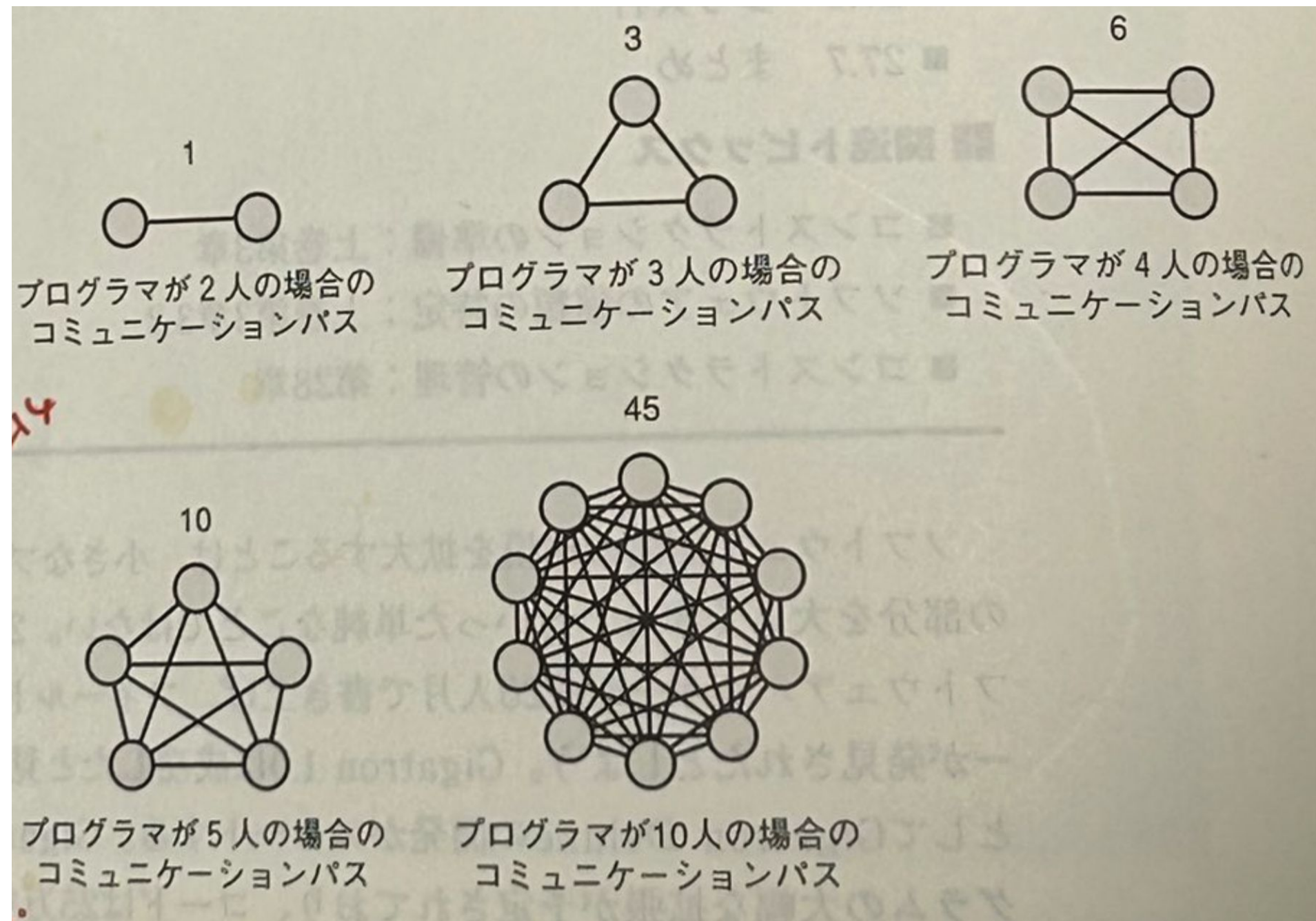
4.7.2 効果的なチームの特徴

- 少人数である (Small in number)
 - コミュニケーションの負荷を最小限に保つ
- 同じゴールを共有 (Share a common goal)
 - 何を持って成功なのかをみんなで理解して、同じ方向を目指す
 - 共通のゴールを各自が「自分ごと」として捉える
- スキルの補完 (Complementary skills)
 - プロダクトマネージャー、デザイナー、エンジニア、QAエンジニアなどが専門外にも関与し、総合力を高める
 - スクラムのようなアジャイル開発では、クロスファンクショナルかつ自己組織化したチームが頻繁なフィードバックとコラボレーションを通じて高い成果を出す
- 説明責任を共有 (Shared accountability)
 - 成果(成功も失敗も)をチーム全体で背負う



[参考]コミュニケーションとチームの人数

- チームの人数とコミュニケーションパスは同等ではなく、指数関数的にパスが増える
 - コミュニケーションパスの数だけ、誤解や漏れが起きる可能性がある
 - 例えば3人のチームであれば誤解や漏れのリスクが3つのパスに内在するが10人だと45のパスにリスクが内在する



4.7.2 効果的なチームの特徴

- 自律的に動く (Self-managing)
 - 1人のリーダーに任せきりではなく、全員がリーダーシップを発揮して主体的に意思決定やタスク管理を行う
- オープンなコミュニケーション (Open communication)
 - 意見や課題を率直に共有し合えるように、
例えば、タスクの進捗や課題は全員が見れる場所で共有し合い、可視化する
- 建設的な衝突 (Healthy conflict)
 - 異なる視点こそが新しいアイデアや問題解決を生むので、相手を尊重して建設的に議論する
- 強力なコラボレーション (Strong collaboration)
 - 一体感を持って協力しあい、知識とスキルを相互に活用する
 - 例えば、コラボレーションを促す手法(ペアプログラミングやスプリントレビューなど)を取り入れ、チーム全員でスピードと品質を両立する



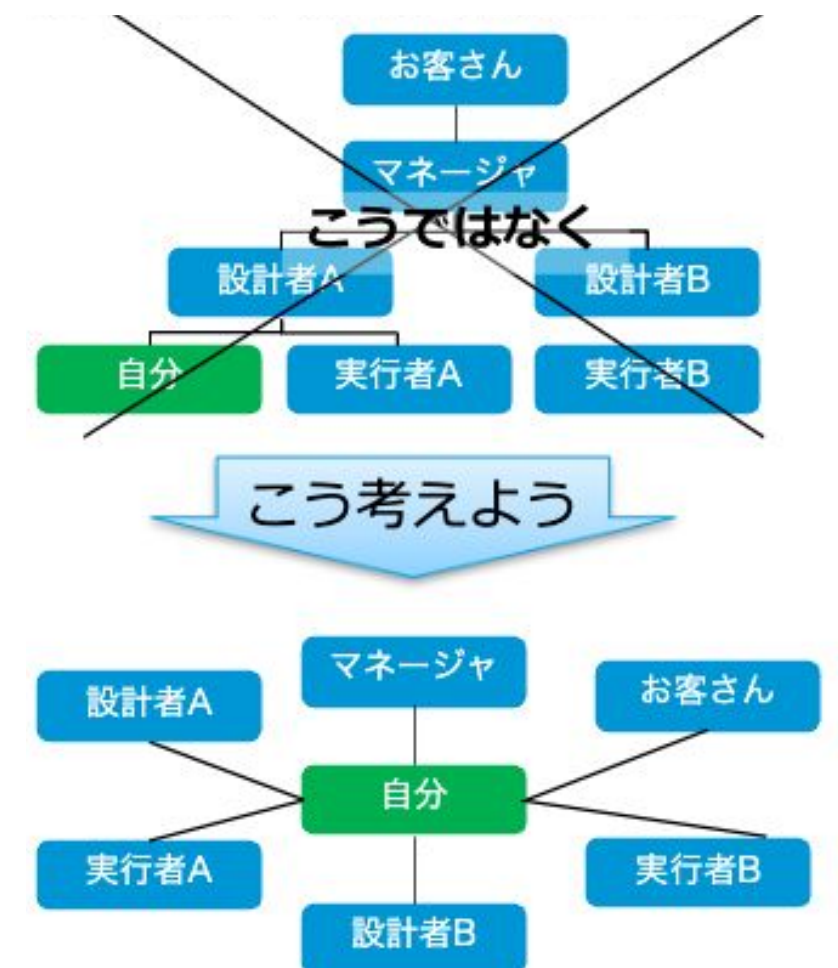
4.7.3 自律的チームでのリーダーシップ

役職についている人 ≠ リーダー

- リーダーシップとはリーダーや特定の人物だけが持つべきものではなく組織の全員が持つべきである
- リーダーシップとは、今やっていることを「自分ごと」として、考えて判断をして行動をして周りに影響をあたえること
 - 自分の仕事のリーダーは自分であり、トップダウン式の役職構造をとらず、自分中心の構造をとる
＝上の人も巻き込んでいけ、というスタンス

リーダーシップは、学んで練習して身につけることができる

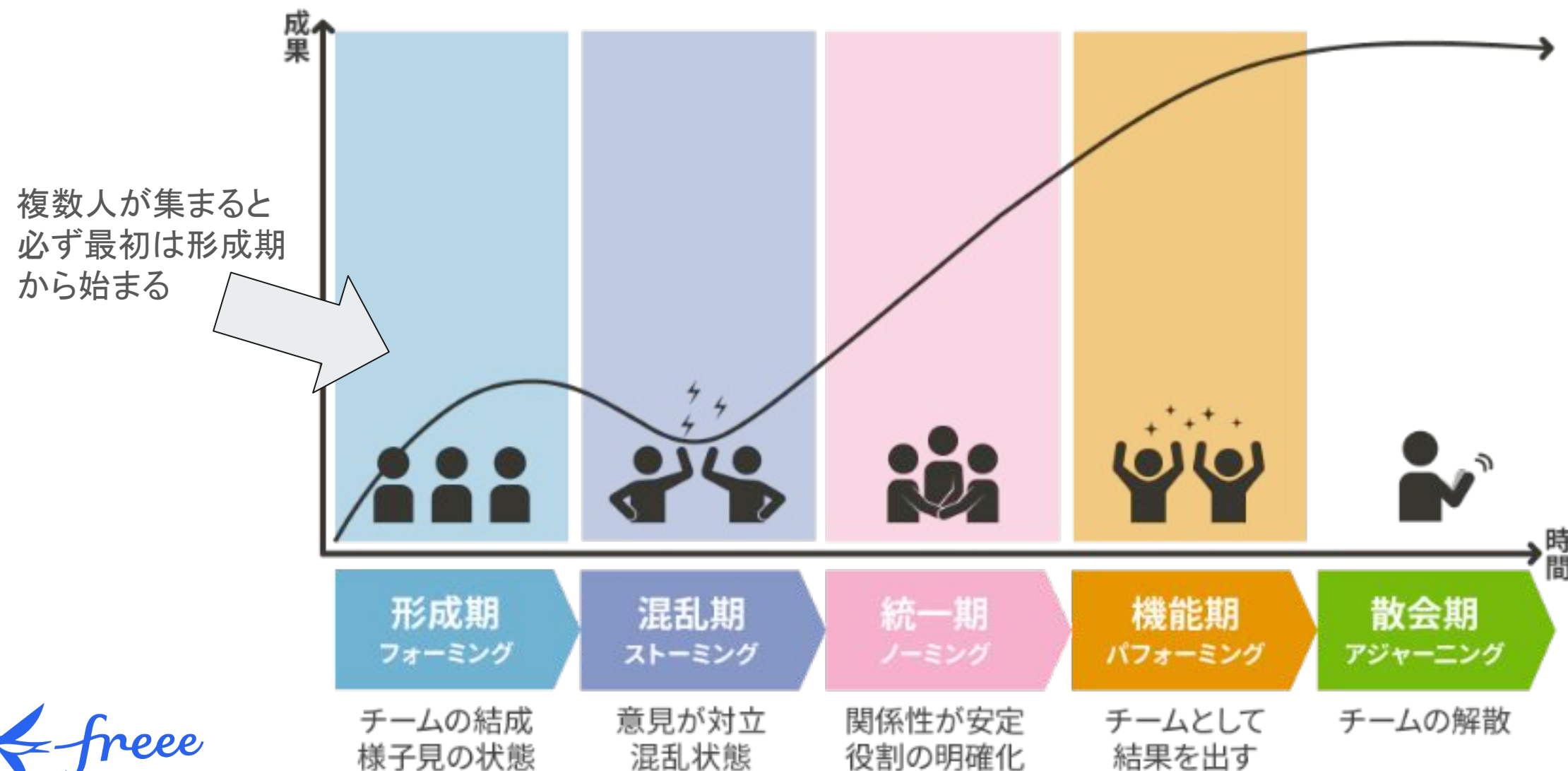
- 特定の人だけが先天的に持っているものではない
- 常日頃にリーダーシップの練習をするのが大事
 - ✕（自分のやっているテストとAさんのテスト似てるよな）
 - ○「同じテストデータで一緒に見れるので共同でやろう！」
- リーダーシップの総量＝チームの力



4.7.4 チームが成長する4つのステージ(タックマンモデル)

チームの置かれているステージに応じたコミュニケーションやサポートを意識することで、成長を加速させられる

- チームは段階的に進化するので、Stormingを上手に乗り越えることが大事
- Performingの段階に進むと、自律性や生産性が格段に向上し、チーム力を発揮できる



Forming(形成期)

- メンバーが初めて集まる段階
- 互いを探り合い、目的や役割がまだあいまい
- 1人のリーダーへの依存が高い

Storming(混乱期)

- チーム内で意見の衝突や役割のすれ違いが表面化
- 人間関係や方針で摩擦が起きやすい
- 1人のリーダーがまだ必要
- この過程を乗り越えることで、チームとしての結束が強まる

Norming(統一期)

- ルールやプロセスが固まり始め、メンバー同士の信頼感が高まる
- チーム全体で目標を共有し、協力体制が整う
- リーダーだけでなく、みんながリーダーシップを発揮し出す
- 雰囲気も安定してきて、建設的なコミュニケーションが取れる

Performing(機能期)

- チームが自律的に動き、成果を最大化できる段階
- みんながリーダーシップをもてる
- メンバーそれぞれが役割を理解し、自発的に問題を解決
- 高い生産性・柔軟性を発揮し、組織や顧客に大きく貢献する

5 グループワーク(外部公開なし)

5.1 要求分析と仕様化

要求分析と仕様化ワークショップ

5.2 重篤度の判断

重篤度を定義するワークショップ



[再掲]到達目標..到達できたでしょうか？

- ソフトウェア開発をどのように進めていけばよいのか説明することができる
- ソフトウェア開発の各活動の必要性を説明することができる
- 自分たちの開発プロセスを、一般的なソフトウェアエンジニアリングで行う活動と対比して説明できる

キーワードを覚えておいて、現場に入ってから仕事していく中で、このテキストを見直して理解を深められればOK

[再掲]目的..今後目指していきましょう！

- ソフトウェア開発を専門とする組織にて、ソフトウェア開発を実施するに当たり、開発の進め方、各活動の必要性を学ぶ
- 各自がエンジニアとして仕事をしていく中で、自社の開発の中のどの位置付けの仕事をしているのかがエンジニアリングとしてわかり、他の仕事との関係性を考慮して生産性高く高品質なアウトプットができる

エンジニアリング実践者 = エンジニア

参考文献

- 実践ソフトウェアエンジニアリング (第9版) (p. 8). (Function). ,ロジャー・プレスマン; ブルース・マキシム
- ソフトウェアエンジニアリング基礎知識体系 -SWEBOK V3.0-,IEEE
- 納得できるテストをするためのモデリング講座,ytteLab
- アジャイルイントロダクション,バートランド・メイヤー
- サル先生のGit入門 <https://backlog.com/ja/git-tutorial/>
- GIT FLOW OVERVIEW AND COMMON GIT COMMANDS
<https://dev.to/truongpx396/git-flow-overview-and-common-git-commands-1odh>
- テストの実践手法を理解する <https://xtech.nikkei.com/it/article/lecture/20070209/261590/>
- ソフトウェアテスト教科書 JSTQB 2023版,大西 建児, 湯本 剛, 町田 欣史, 福田 里奈, 角田 俊, 藤原 考功
- ISO25010:2011(プロダクト品質モデル)
- CODE COMPLETE 第2版 上 完全なプログラミングを目指して,スティーブ・マコネル.
- チームビルディングとは？目的、効果、事例まで一挙公開 <https://teambuilding.patia-kitchen.jp/column/752/>
- 採用基準 地頭より論理的思考力より大切なもの,伊賀泰代

ありがとうございました

アンケートをお願いします！